Workgroup:	Network Working Group
Internet-Draft:	draft-whelan-chio-protocol-00
Published:	30 September 2026
Intended Status:	Standards Track
Expires:	3 April 2027
Author:	C. Whelan *Backbay Industries*

The Chio Protocol: Signed Capabilities and Receipts for Mediated Tool Execution

Abstract

This document specifies the Chio protocol, which mediates the tool calls of software agents. Before a tool runs, a trusted mediator called the kernel checks that the call is authorized by a signed capability token: a time-bounded grant of named tools to a specific key. A token can be delegated in narrowed form. The authenticated attenuation profile binds delegation to the parent scope and checks the child against that scope. The kernel evaluates local policy and dispatches permitted calls. Receipt-bearing outcomes include denials, cancellations, and calls that end before completion. Signing, storage, malformed-authorization, and overload failures can produce an error without a receipt.

This document defines the capability token and receipt formats, their canonical encoding and signatures, a framed transport between agent and kernel, a binding to the Model Context Protocol, and HTTP interfaces for issuance, delegation, receipt query, and revocation. It also defines budgets and metering, governed transactions that require declared intent and approval, and signed checkpoints over batches of receipts.

The RFC Editor will remove this note

About This Document

This note is to be removed before publishing as an RFC.

Status information for this document may be found at https://datatracker.ietf.org/doc/draft-whelan-chio-protocol/.

Discussion of this document takes place on the agentproto discussion mailing list (mailto:agentproto@ietf.org), which is archived at https://mailarchive.ietf.org/arch/browse/agentproto/. Subscribe at https://www.ietf.org/mailman/listinfo/agentproto/.

Source for this draft and an issue tracker can be found at https://github.com/backbay-labs/chio.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at https://datatracker.ietf.org/drafts/current/.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 3 April 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (https://trustee.ietf.org/license-info) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction 7

1.1. Design Goals 8

1.2. Scope 8

1.3. Document Organization 9

2. Conventions and Terminology 9

3. Protocol Overview 10

3.1. Roles and Trust Boundaries 10

3.2. A Mediated Call 11

3.3. Protocol Surfaces 12

4. Encoding and Cryptography 12

4.1. Canonical JSON 13

4.2. Hashing 13
4.3. Signature Suites 13
4.4. Key and Signature Encodings 14
4.5. Signing Wrapper 14
4.6. Artifact Signing Inputs 15
4.7. Chio Key Identifiers 15
5. Capability Tokens 16
5.1. Capability Structure 16
5.2. Scope and Grants 17
5.2.1. Tool Grants 17
5.2.2. Resource Grants 17
5.2.3. Prompt Grants 17
5.3. Constraints 17
5.4. Sender Constraint 18
5.5. Delegation 19
5.6. Attenuation 19
5.7. Chain Binding 20
5.8. Verification 21
5.9. Feature Negotiation 21
6. Receipts 22
6.1. Receipt Structure 22
6.2. Receipt Kinds and Boundaries 24
6.3. Decisions 24
6.4. Content Binding 25
6.5. Receipt Identifier 25
6.6. Signing 25
6.7. Receipt Lineage 26
6.8. Child Receipts 27
6.9. Verification 27

7. Budgets and Metering 28
7.1. Monetary Amounts 28
7.2. Grant Ceilings 28
7.3. Budget Holds 29
7.4. Delegated Budget Shares 30
7.5. Aggregate Invocation Budgets 30
7.6. Guarantee Levels 33
7.7. Authoritative Spend 33
7.7.1. Operation-Owned Execution Nonces 36
7.8. Metering Metadata 36
8. Governed Transactions 40
8.1. Applying the Checks 40
8.2. Governed Intent 41
8.2.1. Intent Hash 44
8.2.2. Intent Checks 44
8.2.3. Metered Billing 45
8.3. Intent Constraints 46
8.4. Approval Tokens 47
8.5. Threshold Approval 50
8.5.1. Proposal 50
8.5.2. Verifying a Threshold Set 52
8.5.3. Kernel-Issued Proposals 53
8.6. Provenance Classes 53
8.7. Call-Chain Continuation 54
8.8. Governed Context in Receipts 56
9. Receipt Checkpoints 58
9.1. Checkpoint Statement 58
9.2. Merkle Trees 60
9.3. Predecessor and Consistency Proofs 60
9.4. Inclusion Proofs 63

9.5. Checkpoint Verification 65
9.6. Claim Limits 65
10. Native Transport 65
10.1. Native Framing 66
10.2. Native Messages 66
10.3. Tool Call Results 69
10.4. Native Errors 71
10.5. Receiver Behavior 72
11. Hosted MCP Binding 72
11.1. Endpoints and Headers 73
11.2. Session Admission 74
11.3. Initialization 75
11.4. Session States 76
11.5. Notification Stream and Replay 76
11.6. Tool Calls 77
11.7. Model Metadata 80
12. Trust-Control Interface 80
12.1. Capability Issuance 81
12.2. Delegated Issuance 82
12.3. Receipt Query 84
12.3.1. Paging 85
12.4. Revocation 86
13. Versioning and Negotiation 86
14. Error Model 87
14.1. Error Registry 87
14.2. Surface Mappings 87
15. Security Considerations 88
15.1. Assets and Trust Boundaries 88
15.2. Transport Security 88
15.3. Token Theft and Replay 89

15.4. Kernel Impersonation and Receipt Trust 90
15.5. Delegation Abuse 90
15.6. Signature Algorithm Downgrade 90
15.7. Canonicalization 91
15.8. Tool Server Confinement 91
15.9. Resource Exhaustion 91
15.10. Budgets and Approvals 91
15.11. Checkpoint Equivocation 92
15.12. Provenance Inflation 92
15.13. Attestation 92
15.14. Clock Skew 92
16. Privacy Considerations 92
17. IANA Considerations 93
17.1. Chio Error Codes 94
17.2. Chio Signature Suites 97
17.3. Chio Native Message Types 97
17.4. Chio Signed Artifact Schemas 98
18. Implementation Status 98
19. References 100
19.1. Normative References 100
19.2. Informative References 101
Appendix A. Examples 103
A.1. Capability Token 103
A.2. Tool Call Request Frame 105
A.3. Receipt 105
Appendix B. Test Vectors 107
B.1. Canonical JSON 107
B.2. SHA-256 Digests 109
B.3. Ed25519 Signatures 109
B.4. Capability Token Verification 112

B.5. Receipt Verification 116
Appendix C. Relationship to Other Work 118
C.1. Agent Communication Protocols 118
C.2. Delegated Authorization 118
C.3. Capability Tokens 119
C.4. Workload Identity and Attestation 119
C.5. Signed Evidence and Transparency 119
C.6. Verifiable Credentials 119
C.7. Candidate Companion Documents 119
Appendix D. Document History 120
Acknowledgments 120
Author's Address 120

1. Introduction

Software agents call tools. They read and write files, query services, send messages, and spend money on behalf of people and organizations. When an agent directly holds broad tool credentials, delegating a task can also delegate more authority than it needs. Unless the deployment records an authenticated authorization decision and result binding, a third party lacks that evidence of what was authorized and what happened.

Chio separates authority from execution by placing a kernel between an agent and its tool servers. A native agent sends each mediated call to that kernel, a mediator it does not control, with a capability token: a statement, signed by an issuer, that a subject key may call named tools under stated constraints and limits until a stated time. The kernel verifies the token, evaluates local policy, dispatches the call to the tool server only if both permit it, and signs a receipt for receipt-bearing protocol outcomes, including denials, cancellations, and calls that end before completion. Signing, storage, malformed-authorization, and overload failures can return an error without a receipt; a deployment cannot claim universal receipt availability for those paths.

Chio's authenticated delegation profiles enforce narrowing of authority. A holder can request or arrange trusted-issuer signing of a narrower token for another key. The resulting token signature is made by a trusted issuer; a delegation-link signature alone does not make an arbitrary holder a trusted token issuer. Each delegation link is signed. In the authenticated attenuation profile, scope hashes and a subset witness bind the child scope to the issuer-resolved parent scope. A kernel enforcing that profile rejects a claimed parent scope that is not bound to the chain. Plain delegated v1 tokens carry the per-link scope hashes but no subset witness; deployments requiring authenticated attenuation enforce the profile in Section 5.7.

This document specifies the objects and exchanges that independent implementations need in order to interoperate: capability tokens, receipts, their canonical encoding and signatures, the native transport between agent and kernel, a binding to the Model Context Protocol [MCP], and the HTTP interface of the trust-control service. It also specifies three extensions that deployments use to reserve spend exposure and to audit history: budgets and metering, governed transactions, and receipt checkpoints.

1.1. Design Goals

Explicit authority: Every mediated call names the capability token that authorizes it. The kernel grants no authority that a presented token does not carry.

Narrowing delegation: An authenticated attenuation profile rejects authority beyond its trusted parent in the checked dimensions: tools, operations, constraints, invocation counts, costs, or time.

Fail-closed evaluation: A failure to verify a token, a delegation chain, a proof, or a policy result denies the call.

A record of receipt-bearing outcomes: The kernel signs receipts for receipt-bearing evaluation outcomes. The receipt distinguishes allowed, denied, canceled, and incomplete calls.

Portable verification: A party that holds a receipt and the kernel's public key can verify the receipt without contacting the kernel.

Transport independence: The native transport and the MCP binding use the same signed capability and receipt formats. Native replies carry receipts; the hosted MCP binding records receipt-bearing outcomes for query.

Algorithm agility: Keys and signatures identify their own algorithm, so classical and post-quantum hybrid signature suites can coexist in one deployment.

1.2. Scope

This document does not specify:

- a policy language or a set of guards. Each kernel evaluates its own local policy and records the results in receipts;
- how tool servers implement tools;
- a replacement for the Model Context Protocol, the Agent2Agent protocol [A2A], or other agent communication protocols. Chio mediates calls that such protocols carry;
- an OAuth authorization server;
- payment settlement, standing relationships between operators, or portable identity credentials for agents. Appendix C lists these as candidate companion documents.

1.3. Document Organization

Section 3 describes the roles and walks through one mediated call. Section 4 through Section 9 define the signed objects: the encoding and signature rules, capability tokens, receipts, budgets, governed transactions, and checkpoints. Section 10 through Section 12 define the exchanges: the native transport, the MCP binding, and the trust-control interface. Section 13 and Section 14 define version negotiation and the error model. The remaining sections cover security, privacy, IANA registrations, and implementation status. Appendix A and Appendix B give examples and test vectors generated from the reference implementation.

2. Conventions and Terminology

The key words "**MUST**", "**MUST NOT**", "**REQUIRED**", "**SHALL**", "**SHALL NOT**", "**SHOULD**", "**SHOULD NOT**", "**RECOMMENDED**", "**NOT RECOMMENDED**", "**MAY**", and "**OPTIONAL**" in this document are to be interpreted as described in BCP 14 [RFC2119] [RFC8174] when, and only when, they appear in all capitals, as shown here.

This document uses the following terms.

Agent: A software actor that requests tool calls. The kernel does not trust an agent; an agent acts only through the capability tokens it presents.

Kernel: The trusted mediator that verifies capability tokens, evaluates policy, dispatches permitted calls to tool servers, and signs receipts.

Tool server: A service that implements tools, resources, or prompts, to which a kernel dispatches permitted calls.

Trust-control service: The service that issues and revokes capability tokens and answers queries over stored receipts.

Capability: The authority to call a set of tools, read a set of resources, or use a set of prompts, under stated constraints and limits.

Capability token: A signed, time-bounded JSON object that grants a capability to a subject key (Section 5).

Issuer: The key that signs a capability token.

Subject: The key to which a capability token is bound. Only the holder of the subject key can exercise the token when the token requires a sender proof.

Grant: One entry in a capability token's scope: a tool grant, a resource grant, or a prompt grant.

Delegation: Issuing a capability token from the authority of an existing token.

Delegation link: One signed step in a capability token's delegation chain.

Attenuation: A narrowing of authority along one or more dimensions: the grants themselves, their operations and constraints, invocation counts, cost ceilings, budget shares, or time.

Guard: A check that the kernel evaluates before dispatch or before releasing output, as part of its local policy.

Receipt: A signed record of a call or operation, carrying a mediated decision, trace observation, or advisory evaluation (Section 6).

Decision: The outcome that a receipt records for a mediated call: `allow`, `deny`, `cancelled`, or `incomplete`.

Checkpoint: A signed statement by a kernel that commits to a batch of receipts in a Merkle tree (Section 9).

Code-formatted words such as `expires_at` are protocol element names and values, which are case-sensitive. JSON is defined in [RFC8259].

3. Protocol Overview

3.1. Roles and Trust Boundaries

Figure 1 shows the roles and the four flows between them.

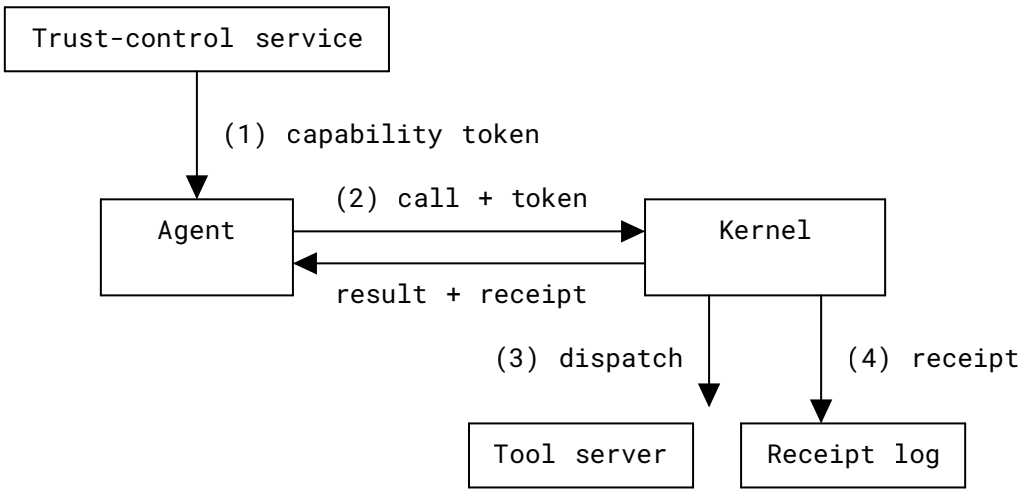


Figure 1: Roles and Flows

The trust-control service issues a capability token to the agent (1). The agent sends a call, with the token, to the kernel (2). The kernel verifies the token, evaluates policy, and, if both permit the call, dispatches it to the tool server (3). The kernel signs a receipt for the receipt-bearing outcome

and records it (4). The illustrated native exchange returns that receipt with the result. Hosted MCP clients receive the result without the receipt, which an operator can query when retained by trust-control (Section 11.6).

The kernel is trusted to mediate the call path. It holds the key that signs receipts and the configuration of trusted issuer keys. The agent is untrusted: it can present any token it holds, and the kernel verifies each one. Preventing calls that bypass the kernel depends on deployment isolation and credential controls (Section 15.8). A tool server role does not itself confer token-issuing authority. The trust-control service is trusted to issue and revoke tokens under the operator's policy.

3.2. A Mediated Call

Figure 2 shows one call over the native transport (Section 10).

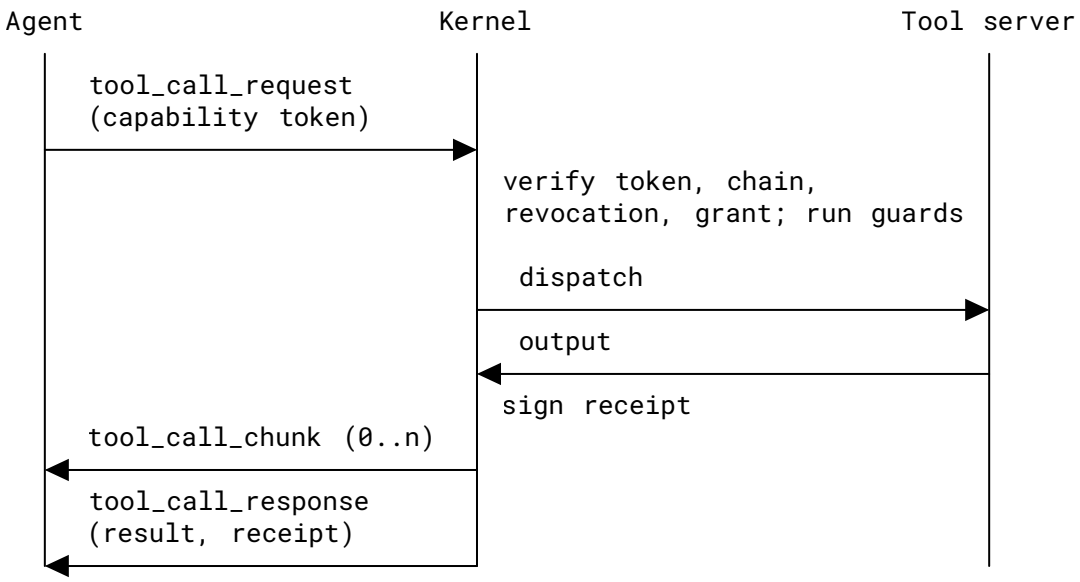


Figure 2: One Mediated Call

The figure shows a receipt-bearing native exchange. The reference driver can buffer all output and sign the receipt before sending the first chunk; chunk delivery does not establish when the tool produced it.

An ordinary receipt-bearing native call proceeds as follows:

1. The agent sends a `tool_call_request` that carries a capability token, the target tool server and tool, and the call's parameters.

2. The kernel verifies the token: its signature, its validity interval, its delegation chain and any applicable attenuation proof, the revocation state of the token and of each delegation ancestor, the grant that matches the target, and a sender proof when the grant requires one (Section 5.8). The native request has no sender-proof field, so a grant requiring that proof is denied on this binding.
3. The kernel evaluates its guards for the call. When budgets apply, it reserves the call's authorized exposure before dispatch (Section 7).
4. If every check passes, the kernel dispatches the call. Output may stream back to the agent as `tool_call_chunk` messages.
5. On a receipt-bearing path, the kernel signs the outcome and returns the receipt in the terminal `tool_call_response`. A failed check prevents dispatch and ordinarily produces a denial receipt. The signing, storage, malformed-authorization, and overload exceptions in Section 1 can instead return an error without a receipt; Section 10.3 describes native diagnostic keys and response failures.

3.3. Protocol Surfaces

Capability tokens and receipts use the same object formats across the three surfaces. Their delivery differs:

Surface	Carries	Encoding	Defined in
Native transport	Calls from agent to kernel	Length-prefixed canonical JSON	Section 10
MCP binding	Calls from MCP clients to a hosted kernel	JSON-RPC over HTTP with server-sent events	Section 11
Trust-control interface	Issuance, delegation, receipt query, revocation	JSON over HTTP	Section 12

Table 1: Protocol Surfaces

The native transport has no initialization exchange. A native agent obtains its capability tokens out of band, for example from the trust-control service, and then sends calls.

4. Encoding and Cryptography

Chio signs typed JSON projections. A verifier reconstructs the projection specified for that artifact, canonicalizes it, builds its signing wrapper (Section 4.5), and verifies its signature. It does not sign the original transport bytes, whitespace, or unknown members discarded during typed parsing. Consumers therefore distinguish authenticated fields from other data in the incoming envelope.

4.1. Canonical JSON

Canonical bytes are UTF-8. Object members are ordered lexicographically by their UTF-16 code-unit sequences; a proper prefix sorts first. Arrays retain order. The encoding emits no whitespace between tokens. It escapes quotation marks, reverse solidus, and U+0000 through U+001F, using the short escapes for backspace, form feed, line feed, carriage return, and tab. Other control escapes use four lowercase hexadecimal digits. Other Unicode characters are emitted without normalization.

Floating-point values use the shortest round-trip decimal representation. Negative zero becomes `0`. Values with magnitude at least 0.000001 and less than 1e21 use decimal notation; other nonzero values use exponent notation, with + on a non-negative exponent. Non-finite values cannot be encoded. These rules follow [RFC8785].

The v1 typed encoding additionally preserves signed 64-bit and unsigned 64-bit integer values as exact decimal integers. It does not round those values to binary64. For example, `9007199254740993` remains those decimal digits. This compatibility rule differs from unrestricted application of RFC 8785's I-JSON number model. An implementation using only binary64 numbers cannot verify such an artifact after rounding its integer fields. For interoperable inputs within the shared I-JSON domain, the encodings agree. Producers can keep integers within the safe binary64 range when interacting with consumers that cannot preserve full-width integers.

Duplicate-member handling and unknown-member rejection belong to the artifact parser. The typed compatibility path does not provide universal duplicate-member rejection. An application requiring rejection at ingress uses a strict JSON parser before constructing its typed object. This document does not turn that stricter profile into a claim about every v1 decoder.

4.2. Hashing

Unless a structure specifies a domain prefix or Merkle construction, Chio uses SHA-256 over the specified bytes [FIPS180-4]. Content-addressed receipt identifiers and scope hashes are lowercase hexadecimal strings without `0x`. Capability and other application-assigned identifiers are strings; they are not required to be hashes. Merkle hashes are serialized as `0x` followed by 64 lowercase hexadecimal digits. A hash of canonical JSON means the hash of its UTF-8 bytes, rather than a hash of an application language's object storage.

4.3. Signature Suites

This document defines Ed25519 [RFC8032], ECDSA P-256 with SHA-256, ECDSA P-384 with SHA-384 [FIPS186-5], and a composite of each classical suite with ML-DSA-65 [FIPS204]. ECDSA signatures use ASN.1 DER encoding [RFC3279]. ML-DSA-65 uses an empty context string. Composite verification requires both components over identical message bytes and matching encoded algorithm sets.

A verifier dispatches from the parsed key and signature material. It **MUST** reject a key/signature algorithm mismatch. A composite verifier **MUST** reject a different algorithm set on the key and signature, an invalid component pairing, or either failed component signature. An unsupported suite fails verification. Capability and receipt v1 verification uses ordinary Ed25519 verification; it does not imply the additional weak-key checks of a strict-verification profile.

Cryptographic floors are `allow_classical`, `allow_hybrid`, and `pq_required`. They respectively accept classical suites only, classical or composite suites, and composite suites only. A floor-aware verifier **MUST** reject an artifact disallowed by the configured floor. On artifacts with an `algorithm` envelope hint, a verifier **MUST** reject a present hint inconsistent with the signature encoding, whether or not it applies a floor. A missing hint does not override a self-describing signature.

4.4. Key and Signature Encodings

Ed25519 public keys are 32 octets encoded as 64 lowercase hexadecimal characters; signatures are 64 octets encoded as 128 such characters. These encodings have no algorithm prefix. ECDSA keys use uncompressed SEC1 points [SEC1]: `p256:` followed by hexadecimal of `04 || X || Y` (65 octets), or `p384:` followed by the corresponding 97-octet point. ECDSA signatures use the same prefix followed by hexadecimal of DER signature bytes.

The composite format for keys and signatures is:

```
hybrid:<classical-encoding>:<pq-hex>:<algorithm-set>
```

The classical component retains its own prefix when it is ECDSA. The algorithm set is exactly `ed25519+mldsa65`, `p256+mldsa65`, or `p384+mldsa65`. ML-DSA-65 public keys occupy 1952 octets and signatures 3309 octets. Components are separated from the right, since an ECDSA classical encoding contains a colon. Composites cannot nest. This version defines no standalone ML-DSA wire encoding.

Producers emit lowercase hexadecimal and no `0x` on classical material. Classical decoders also accept uppercase hexadecimal and an optional `0x`; typed serialization normalizes those spellings before verification.

4.5. Signing Wrapper

Every Chio signature over bytes Chio defines signs an 88-byte signing wrapper instead of the body.

```
wrapper = "chio.signing-wrapper.v1" || 0x00          (24 bytes)
       || SHA-256(utf8(schema_id))                     (32 bytes)
       || SHA-256(canonical_body)                      (32 bytes)
```

`schema_id` is the signed artifact's registered schema id, never the id of a payload or body schema. Each signed artifact has exactly one id, and a request and its response are two artifacts with two ids. Where an artifact carries a `schema` member, it equals the wrapper id. A verifier takes `schema_id` from its verification context, meaning the artifact type it expects at that call site. It never reads `schema_id` from a member of the artifact being verified. A signature over one artifact type therefore never verifies as another.

`canonical_body` is the RFC 8785 canonical JSON of the artifact's signing body (Section 4.1).

The schema id is the only signature domain separation. No signing input carries a domain prefix of its own. A signature that would cover a digest of a Chio object covers the wrapper over that object's canonical body instead. Domain strings inside hash and commitment inputs (digests, identifiers, commitments) are not signature prefixes and are unaffected.

Ed25519 signs the 88 wrapper bytes. ML-DSA-65 signs the 88 wrapper bytes with an empty FIPS 204 context string. P-256 signs with ECDSA over SHA-256 of the wrapper. P-384 signs with ECDSA over SHA-384 of the wrapper. Hybrid signatures sign the same 88 bytes with both halves. Signature encodings are unchanged (Section 4.4).

Both inner hashes are SHA-256, which caps the collision resistance of every signature at 128 bits, including P-384 and ML-DSA-65. Changing either hash or the layout requires a new prefix string.

Exempt formats, a closed list: JWS signing inputs ([RFC7515], including local authorization server access tokens, other JWTs, OpenID4VP request and response objects and SD-JWT) and DSSE pre-authentication encoding. An external standard fixes their signing bytes, so they do not use the wrapper. Every other signature does.

4.6. Artifact Signing Inputs

The projection is part of the protocol. Each signed artifact defines its signing body and its schema id, and its signature covers the signing wrapper (Section 4.5) of that body under that id. Capability tokens include their schema and exclude their signature and algorithm hint. A receipt's signing body holds the computed identifier and identity projection. Delegation links, approvals, child receipts, and checkpoints use their own body projections. Copying a signing rule from one artifact to another is not valid. The relevant sections define the exact bodies and schema ids.

4.7. Chio Key Identifiers

The `did:chio` form in this version is `did:chio:` followed by the 64 lowercase hexadecimal characters of an Ed25519 public key. It does not encode an ECDSA or composite key. Resolution constructs a local key document; it does not perform an online trust lookup. The key method identifier is the DID plus `#key-1`, with type `Ed25519VerificationKey2020`. Its `publicKeyMultibase` is `z` followed by base58btc of `0xed 0x01` and the raw key. Authentication and assertion references identify that method. Knowing the DID does not establish permission to issue capabilities or sign receipts for an operator.

5. Capability Tokens

A capability token is an issuer-signed grant to a subject key for a bounded time. The issuer's trust relationship, the grant's scope, local policy, and the runtime's enforcement profile jointly determine whether the kernel can dispatch a request.

5.1. Capability Structure

Field	Type	Presence
`schema`	`chio.capability.v1` string	Always
`id`	String	Always
`issuer`, `subject`	Encoded public keys	Always
`scope`	Scope object	Always
`issued_at`, `expires_at`	Unsigned 64-bit Unix seconds	Always
`delegation_chain`	Array of links	When nonempty
`aggregate_invocation_budget`	Budget object	When present
`caveats`	Array	When nonempty
`scope_attenuations`	Array	When nonempty
`attenuation_proof`	Proof object	When present
`budget_share_bps`	Unsigned 16-bit integer	When present
`algorithm`	Algorithm hint	When non-default
`signature`	Encoded signature	Always

Table 2: Capability Token Fields

The signing input is one flattened object containing every typed member above except `algorithm` and `signature`, with the same omission rules. The `schema` member equals `chio.capability.v1`. A token that omits it or carries another value fails validation. The signature covers the signing wrapper (Section 4.5) of the canonical JSON of that object under `chio.capability.v1`. The issuer **MUST** reject a signing operation whose actual key differs from `issuer`. Appendix A.1 gives exact generated bytes.

Verification uses only this signing body. A signature over the token without the `schema` member does not verify.

5.2. Scope and Grants

A scope has `grants` (tool grants), `resource_grants`, and `prompt_grants`. Each array defaults to empty and is omitted when empty. An empty scope is `{}`. Operations are `invoke`, `read_result`, `read`, `subscribe`, `get`, and `delegate`. An operation name is not proof that a runtime implements an enforcement point for it.

5.2.1. Tool Grants

A tool grant requires `server_id`, `tool_name`, and `operations`. `constraints` is omitted when empty. `max_invocations`, `max_cost_per_invocation`, `max_total_cost`, and `dpop_required` are included when present. Section 7 defines the ceilings.

A grant matches an invocation when the server and tool each equal the request value or the literal `*`, the operations contain `invoke`, and all argument constraints match. Tool names have no prefix-glob rule. Matches are ordered by server exactness, tool exactness, then constraint count, each descending; ties keep the original grant index. A constraint evaluation error on a target-matching grant aborts resolution rather than allowing a less specific grant to hide that error. The portable tool profile selects the first match. The hosted kernel can try matched grants in order through governance, policy, runtime, and budget admission.

5.2.2. Resource Grants

A resource grant requires `uri_pattern` and `operations`. Resource reading checks `read`; subscription checks `subscribe`. Patterns use raw-string equality, `*` for all values, or a trailing `*` for a prefix. No URI normalization is implied by that comparison.

5.2.3. Prompt Grants

A prompt grant requires `prompt_name` and `operations`. Prompt retrieval checks `get` using the same exact, universal, or trailing-prefix pattern rules. Resource and prompt matching are outside the portable tool-only profile.

5.3. Constraints

Constraints use `{"type":name,"value":payload}` with case-sensitive snake_case names. A unit constraint omits `value`. Unknown variants or unknown members of the constraint object fail parsing. The portable argument profile evaluates these constraints:

- `path_prefix` (string): identify path-like string leaves, normalize slash and reverse-solidus separators, remove empty and `.` segments, resolve `..` without escaping the root, and require matching absolute or relative form and the stated segment prefix for every candidate.
- `domain_exact` and `domain_glob` (strings): extract host-like values from string leaves, remove scheme, user information, port, path, query, and fragment, trim dots, and lowercase. Exact compares hosts; the glob's `*` matches any run including dots. At least one candidate is needed, and every candidate has to match.
- `max_length` (non-negative integer): limit the UTF-8 byte count of each argument string leaf.

- `max_args_size` (non-negative integer): limit the bytes in compact JSON of the entire arguments value, rather than its canonical form.
- `custom` (two-string array): require a recursively found object member whose name and string value match. Reserved delivery and recovery spellings are errors, not custom predicates. They are `output_digest_sha256`, `require_finding_purchase`, `require_finding_recovery`, `recovery_of_receipt_id`, and `recovery_of_capability_id`.
- `audience_allowlist` (string array): check members named `recipient`, `recipients`, `audience`, `to`, `channel`, or `channels`, ignoring case in the names. If present, each value must contain at least one string and contain only strings or recursively nested arrays. Every string value is allowlisted, including an empty string only when explicitly allowlisted. Absent audience members pass.
- `memory_store_allowlist` (string array): the same rule for `store`, `memory_store`, `collection`, and `namespace`.

Path candidates have a case-insensitive key containing `path` or equal to `file`, `filepath`, `dir`, `directory`, `root`, or `cwd`; path-like values without `://` are also candidates when they begin with `/`, `./`, `../`, or `~/`, or contain a path separator. A missing path candidate does not match. Host candidates are `localhost` or strings containing a dot with only ASCII alphanumerics, dots, and hyphens after extraction.

Governed constraints are defined in Section 8.3. Other constraint families require an explicitly supported runtime profile. When constraint evaluation encounters an unsupported constraint, the portable kernel **MUST** fail grant resolution. A marker that a hosted matcher defers to a guard is not evidence that the guard enforced it. Section 18 identifies that profile boundary.

5.4. Sender Constraint

The Chio invocation proof object below is distinct from the hosted HTTP DPoP proof. The native `tool_call_request` has no field carrying it; the ordinary session bridge supplies no invocation proof. The hosted HTTP edge accepts no DPoP sender proof (Section 11.2). A grant requiring invocation-level DPoP therefore needs a surface that carries the object; neither transport silently supplies it.

When any matching tool grant has `dpop_required: true`, the hosted kernel requires a valid proof before invocation. The native proof is `{body, signature}`. Its body has `schema` (`chio.dpop_proof.v1`), `replay_authority`, `capability_id`, `tool_server`, `tool_name`, `action_hash`, `nonce`, `issued_at` (Unix seconds), and `agent_key`. `replay_authority` names the durable replay domain the verifier selects independently of the proof: `destination_store_uuid`, `dpop_authority_id`, `expectation_id`, `proof_ttl_secs`, and `max_clock_skew_secs`. All members are required. The subject signs the signing wrapper (Section 4.5) of the canonical JSON of that body under `chio.dpop_proof.v1`. `action_hash` hashes canonical arguments only. This is capability/action proof of possession; it is distinct from the HTTP-method and URI JWT binding in [RFC9449].

The kernel checks schema, that `replay_authority` equals its activated authority, subject key, capability/server/tool/action bindings, freshness, signature, and durable nonce replay state. Freshness uses the authority's `proof_ttl_secs` and `max_clock_skew_secs`; the expiration comparison is inclusive. The nonce's replay identity is `(nonce, capability_id)` within that authority and is retained through its signed expiry. Preview and dispatch revalidation can be stateless; dispatch credential acquisition claims the nonce in durable custody. A kernel with no activated replay authority refuses every required proof.

5.5. Delegation

A delegation link requires `capability_id`, `delegator`, `delegatee`, `timestamp`, `scope_hash` (Section 5.7), and `signature`. It includes `attenuations` when nonempty, and `aggregate_budget` and `cumulative_approval` when present. The delegator signs the signing wrapper (Section 4.5) of the canonical JSON of those typed fields except `signature` under `chio.delegation-link.v1`; the link has no schema member. The signer has to match `delegator`.

A chain verifier checks each link's signature and `scope_hash` presence, adjacent key connectivity, nondecreasing timestamps, and final delegatee equal to the token subject. A configured maximum depth is also enforced. A portable verifier has no universal default depth limit. The hosted kernel additionally validates stored parent snapshots, delegator authority, validity windows, expiry narrowing, and delegation permissions. It checks revocation for the token and every named ancestor.

5.6. Attenuation

Signed attenuation arrays use internally tagged objects. Members below are beside `type`, rather than nested inside a `value` member:

type	Additional required members
`remove_tool`	`server_id`, `tool_name`
`remove_operation`	`server_id`, `tool_name`, `operation`
`add_constraint`	`server_id`, `tool_name`, `constraint`
`reduce_budget`	`server_id`, `tool_name`, `max_invocations` (unsigned 32-bit integer)
`shorten_expiry`	`new_expires_at` (unsigned 64-bit integer)
`reduce_cost_per_invocation`	`server_id`, `tool_name`, `max_cost_per_invocation` (monetary amount)
`reduce_total_cost`	`server_id`, `tool_name`, `max_total_cost` (monetary amount)

Table 3: Signed Attenuation Variants

Target names are strings; `operation`, `constraint`, and monetary amounts use the types defined above. Unknown variants fail parsing. The compatibility decoder can discard unknown variant members, which therefore supply no authenticated authority.

A cumulative-approval constraint in a child scope **MUST** preserve the parent's approval-budget identifier, epoch, currency, and canonically equal root binding, or have both root bindings absent. Its threshold **MUST NOT** exceed the parent's threshold. A child **MUST NOT** introduce a cumulative-approval marker absent from its covering parent grant.

Every child grant fits a parent grant of the same kind. For tools, the parent covers the child server and tool, child operations are contained, parent constraints remain present, invocation and monetary caps remain or narrow, and `dpop_required: true` remains true. Monetary caps preserve currency. Constraint preservation uses equality except the supported cumulative-approval narrowing rule in Section 8. Resource and prompt narrowing use their pattern-coverage rules. Delegating authority additionally requires a covering parent grant with `delegate`.

The attenuation proof uses camelCase members `parentScopeHash`, `childScopeHash`, and `normalizedSubsetProof`. The witness contains `normalizedParentScope` and `normalizedChildScope` (JSON strings), `subsetRelations` and `restrictedPredicates` when nonempty, and `aggregateBudget` and `cumulativeApproval` when present. A relation has `grantKind`, `childIndex`, `parentIndex`, and boolean `subset`. Scope hashes are SHA-256 of the canonical typed scope bytes. Array order is preserved; normalization does not sort grants or operations.

The verifier hashes the supplied scope strings, parses both scopes, and recomputes actual grant subset relationships. It rejects a false declared subset relation, child-hash mismatch, or a widened actual scope. Relation indices and textual restricted-predicate descriptions are not independent authorization evidence. The token's `scope_attenuations` is signed but does not substitute for recomputing the subset. Nonempty caveats fail closed in this version; this document defines no caveat evaluator.

5.7. Chain Binding

Every delegation link **MUST** carry `scope_hash`, the canonical hash of the scope that the delegator authorized at that hop. The link signature covers it, binding that scope to the delegator's key. A verifier **MUST** reject a delegation chain in which any link omits `scope_hash`. This applies to every delegated token, including a plain chain that carries no attenuation proof, no `scope_attenuations`, and no budget share.

A proof, nonempty `scope_attenuations`, or a budget share additionally binds the attenuation proof to that lineage. The verifier **MUST** require an attenuation proof and an issuer-resolved trust-root scope hash for such a token. For an empty chain, the proof's parent hash equals that root hash. For a delegated token, the last link's hash equals the proof's parent hash and the first link's hash equals the root hash. This proof-bearing profile accepts at most one link. Aggregate and cumulative profiles impose their own authenticated root-snapshot checks.

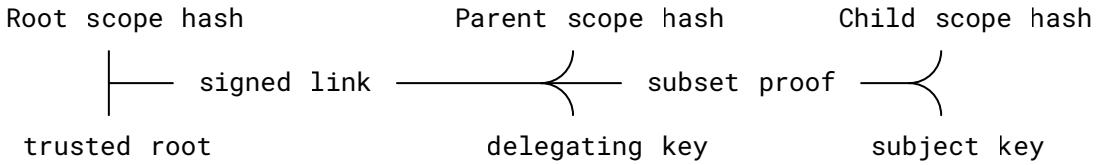


Figure 3: One-Hop Scope-Bound Delegation

Open issue: multi-hop across independent keys. The one-link limit is a restriction of this profile, flagged here rather than resolved. A proof-bearing token with two or more delegation links fails the chain-binding step of Section 5.8, and the verifier resolves a single trust-root scope hash from the token issuer. Authority that passes through more than one delegating key, such as keys held by independent operators, therefore cannot be carried as a scope-bound chain. Lifting the limit needs per-hop child-scope witnesses, a set of partner keys distinct from the issuing authority's keys, and a chain-binding resolver at each receiving boundary. This document does not specify them; cross-operator delegation is candidate companion work (Appendix C), and the reference implementation's roadmap defers it (lane COOP, After the test).

5.8. Verification

Portable full verification checks, in order: peer profile validity; issuer trust; optional-feature negotiation; algorithm-hint consistency and cryptographic floor; schema and signature; time; supplied direct-root capability, if present; aggregate and cumulative budget bindings; chain shape; chain binding; and sibling-share admission. Time is valid exactly when `issued_at <= now < expires_at`, with no capability clock-skew allowance. The chain-binding step rejects a proof-bearing token with more than one delegation link; see the open issue in Section 5.7.

Failures deny admission. They distinguish untrusted issuer, invalid signature, rejected cryptographic floor, not-yet-valid, expired, attenuation violation, and budget split rejection. Some schema or proof errors occur inside signature reconstruction and surface as internal verification failures. The wire mapping is specified in Section 14.

Hosted admission adds token/ancestor revocation, stored delegation lineage, exact subject binding (`agent_id` equals normalized subject-key encoding), scope resolution, proof of possession, governance, guards, runtime and budget admission, credential reservation, and immediate dispatch revalidation. Checks can repeat across that boundary. A token verified earlier is not exempt from later revocation or changed state.

5.9. Feature Negotiation

A capability-feature profile is `{"schema":"chio.capabilities.v1","features":{"name":true}}`. `schema` defaults to this value on parsing and is always serialized; an empty `features` map is omitted. Unknown envelope members are rejected. Feature names contain 1 to 96 ASCII bytes from `[a-z0-9_.-]`; values are booleans. The flags relevant here are

`delegation_chain_binding`, `aggregate_invocation_budget`, `cumulative_approval_budget`, and `threshold_governed_approvals`. Other defined flags describe companion profiles and do not establish support for this document's checks.

The intersection of two valid profiles contains true only when both sides declare true, false when either explicitly declares false, and omits a flag declared true by only one side. The chain-binding rule's absent-flag compatibility default still applies to that intersection.

Optional-feature negotiation can gate chain binding, aggregate invocation budgets, cumulative approval, and threshold approval. An absent chain- binding flag defaults to enabled; explicitly disabling it rejects tokens that require it. A feature flag describes supported validation behavior, not permission to skip a constraint whose evaluation is unsupported. The reference local kernel configures this profile locally; the separately specified federation handshake carries signed capability negotiation and a pinned intersection. MCP initialization advertises its own transport features, not the complete `chio.capabilities.v1` negotiation object.

6. Receipts

A receipt binds a kernel key to a recorded outcome. Its signature permits a verifier to detect changes to the signed record. The verifier separately decides whether to trust that key and what authority the recorded boundary supports. A trace of an observed call does not establish that the kernel authorized that call.

6.1. Receipt Structure

The receipt is one JSON object. It has no top-level `schema` member. `chio.receipt.v1` names the artifact and signing projection, rather than a field to insert into the wire object. All names below are case-sensitive.

Field	Type	Presence
`id`	Lowercase SHA-256 hex string	Always
`timestamp`	Unsigned 64-bit Unix seconds	Always
`capability_id`	String	Always
`tool_server`, `tool_name`	Strings	Always
`action`	Action object	Always
`receipt_kind`	Kind string	Always
`boundary_class`	Boundary string	Always
`tool_origin`	Origin string	Always
`redaction_mode`	Redaction string	Always
`decision`	Decision object	Mediated kind
`observation_outcome`	Outcome string	Other kinds
`content_hash`	SHA-256 hex string	Always
`policy_hash`	Policy identifier string	Always
`actor_chain`	Array of actor objects	When nonempty
`evidence`	Array of guard evidence	When nonempty
`trust_level`	Trust string	Always
`metadata`	JSON value	When present
`tenant_id`	String	When present
`kernel_key`	Encoded public key	Always
`algorithm`	Algorithm hint	When non-default
`signature`	Encoded signature	Always

Table 4: Receipt Fields

The action has `parameters` (any JSON value) and `parameter_hash`. The latter is lowercase hexadecimal SHA-256 of the canonical parameters. An actor has `actor_id` and, when supplied, `actor_kind`. Guard evidence has `guard_name`, `verdict` (boolean), and, when supplied, `details`. `policy_hash` identifies the evaluated policy; a symbolic policy identifier is also representable, so this field is not necessarily a SHA-256 digest.

`tool_origin` is `caller_executed`, `host_executed_provider_reported`, or `host_executed_unmediated`. `redaction_mode` is `none`, `summary`, or `redacted`. The origin describes the execution path; it does not change the boundary's authority. Redaction describes the producer's representation of the record, not permission to modify an already signed receipt.

6.2. Receipt Kinds and Boundaries

The kernel or other producer **MUST** check the following coherence rules before signing. A receipt verifier **MUST** reject a record that violates them.

Kind	Boundary	Trust	Outcome member
`mediated_decision`	`prevent`	`mediated`	`decision`
`trace_observation`	`detect_only`	`verified`	`observation_outcome`
`advisory_evaluation`	`advisory_only`	`advisory`	`observation_outcome`

Table 5: Receipt Coherence

A mediated receipt carries a decision and no observation outcome. A non-mediated receipt carries an observation outcome and no decision. Observation outcomes are `observed`, `evaluated`, and `dropped`. `cannot_see` is not a signable receipt boundary in this version. The word `verified` in a trust label describes the observation class; it does not prove preventive enforcement.

6.3. Decisions

Decisions are JSON objects tagged by `verdict`:

- `allow`: no additional members.
- `deny`: `reason` and `guard`, both strings.
- `cancelled`: `reason`, a string.
- `incomplete`: `reason`, a string.

An allow decision establishes authorization only inside a coherent `mediated_decision` receipt, under a trusted kernel key. Denied, canceled, and incomplete outcomes do not authorize the call. An incomplete outcome preserves uncertainty about execution; it is not a statement that nothing happened. An observation or advisory receipt never substitutes for an allow decision at a preventive boundary.

6.4. Content Binding

For a mediated value result, the kernel hashes canonical JSON of the evaluated output. For an ordinary, non-redacted call with no output, it hashes the four UTF-8 bytes `null`. For a stream, it first hashes each chunk's canonical data; then it hashes the concatenation of those lowercase hexadecimal digests, in chunk order, without separators. The concatenated values are ASCII hexadecimal strings, not raw 32-octet digests. Stream metadata records chunk hashes, count, and canonical byte count. A delivery-mismatch denial uses the distinct redacted commitment `H(UTF8("chio.delivery-mismatch.redacted.v1") || 0x00 || ASCII(expected_digest))` and omits public stream metadata. A verifier **MUST** select that producer profile before comparing the content binding.

When the kernel holds the evaluated output, it **MUST** recompute its content hash before signing. A pre-filled hash that does not match is rejected. A trusted-body relay can sign a record supplied by a trusted producer when it does not hold that output. That seam preserves producer trust; it does not provide an independent check of the content.

Different observation producers can bind different content preimages. A verifier therefore needs the producer's content profile and the actual content to check `content_hash`; the signature alone does not perform this check.

6.5. Receipt Identifier

Let `J(x)` be the canonical JSON UTF-8 encoding in Section 4.1 and let `H(x)` be lowercase hexadecimal SHA-256. Construct the identity projection `I` from all typed receipt fields except `id`, `algorithm`, `signature`, and the selective-disclosure extension signature. Preserve the omission rules of Section 6.1. The identifier is:

```
id = H(J(I))
```

Thus the kernel key, decision, content binding, action, evidence, attribution, metadata, and tenant are bound into the identifier. Field order in an incoming JSON object does not affect it. Unknown members discarded by a typed parser are outside this projection; consumers cannot infer that such members were authenticated. Extensions that change the projection need a separately defined profile. A base-profile consumer **MUST** reject `bbs_projection_version` or `bbs_signature`, or route the object to the separately defined selective-disclosure profile. This document does not define that extension's projection or signing wrapper.

6.6. Signing

Preparation proceeds in order: validate semantic coherence; bind the signing nonce if the pre-binding identifier is nonempty; compute the content-addressed identifier; then sign the signing body below. The nonce is the trimmed pre-binding identifier. It is placed in `metadata.chio_receipt_signing_nonce`, replacing an existing value. Absent metadata becomes

an object. Non-object metadata is preserved under `original_metadata` before the nonce is inserted. An empty pre-binding identifier does not insert a nonce. A verifier cannot assume every receipt carries one.

Without selective-disclosure extension material, the signing body is exactly the canonical UTF-8 encoding of:

```
{"id": <computed identifier>, "body": <identity projection I>}
```

The angle-bracket notation describes a projection, not a JSON example. The kernel **MUST** reject a signing operation whose actual key differs from the embedded `kernel_key`. The signature covers the signing wrapper (Section 4.5) of that body under `chio.receipt.v1`, not the wire envelope or the identity projection by itself. The optional `algorithm` hint is outside the signed projection. Section 4.4 defines the self-describing signature encodings. Appendix A contains a generated receipt, its exact signing body, and its signing wrapper.

6.7. Receipt Lineage

An ordinary receipt has no top-level DAG ordinal, parent set, or hybrid logical clock. A governed call carries pairwise continuation information inside signed metadata, as specified in Section 8.7. A separate lineage statement can connect parent and child receipts. Multiple verified statements can describe multiple parents; this does not make generic receipt signature verification a DAG validator.

The lineage statement uses camelCase names. Its signed body contains `schema` (`chio.receipt_lineage_statement.v1`), `id`, `parentReceiptId`, `childReceiptId`, `parentRequestId`, `childRequestId`, `parentSessionAnchor`, `childSessionAnchor`, `relationKind`, `evidenceClass`, `issuedAt`, and `kernelKey`; `continuationTokenId` is included when present. The session-anchor reference and governed continuation are defined in Section 8.7. Relations in this document are `local_child` and `continued`. The signature covers the signing wrapper (Section 4.5) of the canonical body, including its identifier, under `chio.receipt_lineage_statement.v1`; it does not use the ordinary receipt signing body or nonce rule. A lineage statement verifier checks the schema and signature. The endpoints' authenticity, authority, and relationship need separate verification.

A parent-set hash helper hashes canonical JSON of a sorted, deduplicated array of receipt identifier strings. That helper does not bind a chain identifier or enforce DAG topology. This document defines no common top-level multi-parent receipt envelope and makes no acyclicity or child-ordinal claim for ordinary receipt verification.

6.8. Child Receipts

A child operation can end without being a mediated tool decision. Its receipt carries `id`, `session_id`, `parent_request_id`, `request_id`, `operation_kind`, `timestamp`, `terminal_state`, `outcome_hash`, `policy_hash`, `kernel_key`, and `signature`. `metadata` and `algorithm` use the ordinary presence rules. Its identifier is a producer-assigned string, not the content address of Section 6.5.

Operation kinds are `tool_call`, `create_message`, `create_elicitation`, `list_roots`, `list_resources`, `read_resource`, `list_resource_templates`, `list_prompts`, `get_prompt`, `complete`, `list_capabilities`, and `heartbeat`. The `terminal_state` is tagged by `state`: `completed`, `cancelled` with a string `reason`, or `incomplete` with a string `reason`.

The child signature covers the signing wrapper (Section 4.5) of the canonical JSON of its body, including `id` and excluding `algorithm` and `signature`, under `chio.child-request-receipt.v1`. It has no ordinary receipt signing body or signing nonce. A successful child outcome hashes canonical JSON of `{"outcome":"result","result":value}`; an error hashes `{"outcome":"error","message":text}`. An ordinary operation error can be a completed child operation. Child completion therefore does not mean that its result succeeded or was authorized. A child verifier **MUST** verify the canonical child body under its embedded signing key, and a consumer accepting it as evidence **MUST** separately authorize that signer and apply any configured cryptographic floor. It **MUST NOT** apply the ordinary receipt content-addressed identifier or signing body to a child receipt. Signature verification does not recompute `outcome_hash`.

6.9. Verification

An ordinary receipt verifier follows this procedure; child receipts use the distinct preimage and checks in Section 6.8:

1. Parse the typed fields and apply the kind, boundary, decision, and trust coherence rules in Section 6.2.
2. Reconstruct `I`, recompute `H(J(I))`, and reject a different `id`.
3. Reconstruct the signing body and its signing wrapper under `chio.receipt.v1`, and verify the signature using the embedded key and signature encoding. Reject an algorithm mismatch between the key and signature. The verifier **MUST** reject a present `algorithm` hint that does not match the signature encoding, whether or not it applies a cryptographic floor; the hint is outside the signed wrapper, so altering it leaves the signature valid. When a cryptographic floor is applied, also enforce the floor.
4. Check whether the kernel key belongs to the verifier's configured trusted set. A self-consistent record signed by an untrusted key is not accepted as evidence of that verifier's kernel.
5. If checking action integrity, recompute `parameter_hash` from `action.parameters`. If checking content integrity, obtain the output and its producer profile, then recompute its content binding.

6. Evaluate the policy identity, time, tenant, attribution, and any lineage evidence against the verifier's own application policy.

Generic signature verification supplies steps 1 through 3. The binding verification profile additionally checks parameter integrity and signer membership. Freshness, external content, and the operational meaning of metadata remain separate checks. A field named `trust_level` or `evidenceClass` never replaces those checks.

7. Budgets and Metering

Budgets mediate admission and reserve accounting headroom before dispatch. They do not, by themselves, establish the amount an external service billed or stop a provider from reporting more than the reserved exposure. A verifier distinguishes an authorization ceiling, a durable reservation, realized accounting, and an external charge.

7.1. Monetary Amounts

A monetary amount is exactly `{"units":integer,"currency":string}`. `units` is an unsigned 64-bit integer in minor units; zero is representable. `currency` denotes an ISO 4217 currency [ISO4217], such as USD with cents or JPY with yen. The wire type rejects unknown members. The typed grant path does not independently validate ISO spelling or the equality of per-call and total currencies. Delegation narrowing does compare currency strings and rejects a changed currency.

A monetary-amount producer **MUST** include both members and encode units as an integer from 0 through 18446744073709551615. A typed monetary-amount decoder **MUST** reject missing members, unknown members, a non-integer units value, or a units value outside that range. The JSON-schema profile additionally requires a nonempty currency string. The typed grant path does not enforce that string's ISO spelling or compare the per-call denomination with the total denomination; deployments that require those properties apply a separate currency policy.

Budget-store additions **MUST** fail on integer overflow rather than wrap. A store whose integer range is narrower **MUST** reject values outside that range. The reference SQLite store supports nonnegative values through 9223372036854775807. These budget-store rules do not describe the saturating totals in auxiliary metering records.

7.2. Grant Ceilings

A tool grant can carry `max_invocations` (unsigned 32-bit integer), `max_cost_per_invocation` (monetary amount), and `max_total_cost` (monetary amount). Each is independently omitted when absent.

A store enforcing a grant's invocation and monetary ceilings **MUST** authorize an admission only when every applicable check succeeds:

- The grant's reserved plus captured invocation count is less than max_invocations. A maximum of zero admits no invocation.
- Requested exposure is no greater than max_cost_per_invocation.

- Existing exposed units plus realized spend plus requested exposure is no greater than max_total_cost.

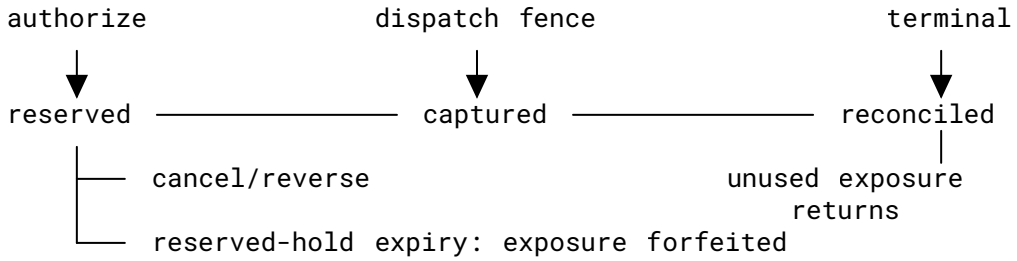
An absent ceiling contributes no check. Equality is permitted for the monetary checks. The kernel requests max_cost_per_invocation.units as exposure, or zero when that ceiling is absent. It does not compute exposure from a realized cost. Therefore max_total_cost alone does not advance monetary accounting and is not an enforced standalone spending ceiling.

An admission in the durable monetary profile **MUST** have a positive per-call exposure. A kernel enforcing that profile **MUST** deny financial tool dispatch when no durable admission covers the selected grant. An explicitly enabled development escape from that check does not establish durable monetary enforcement.

Monetary rows are scoped to the capability identifier and grant index. A delegated token does not automatically share its parent's monetary row.

7.3. Budget Holds

The admission lifecycle is authorize exposure and reserve invocation quotas, capture before dispatch, then reconcile measured spend.



For a positive monetary exposure, a hold store **MUST** reject reconciliation unless the invocation is captured, the monetary state is exposed, the supplied exposure equals the hold's remaining exposure, and realized units do not exceed that exposure. Successful reconciliation removes that exposure and adds realized units; the unused portion returns to headroom. Realized units can be zero.

A hold store **MUST** reject a monetary release unless the release is positive, does not exceed remaining exposure, the monetary state is exposed, and the invocation remains authorized. It **MUST** reject reversal unless the invocation remains authorized and the reversed amount is the complete remaining exposure. Release and reversal therefore cannot cross invocation capture. Cancellation of a captured admission before dispatch is a separate operation.

For an explicitly caller-reserved hold that is still open and whose reserved_until is at or before the reaper's time, the reaper **MUST** capture the invocation if needed and realize the full remaining exposure. It does not refund an abandoned reservation. This expiry rule applies to those reserved holds, not every open inline hold. A zero-exposure invocation reservation settles through capture alone.

On the measured-cost path, the kernel reconciles at no more than the authorized exposure. A missing cost report on that path defaults to the full exposure. If reported cost exceeds a positive exposure, accounting closes at exposure and settlement_status is failed; the financial reported amount can retain the larger cost and the call can remain allowed. The receipt does not prove the external charge was limited. In the reference path with no separate final cost authorization, an adapter that explicitly declares that it does not measure realized cost instead produces a provisional allow with cost_charged 0, settlement_status pending, reversed hold lineage, and no minted nonce. That path does not satisfy the authoritative-spend profile.

7.4. Delegated Budget Shares

The signed token member budget_share_bps is an unsigned 16-bit value from 0 through 10000, omitted when absent. A token verifier **MUST** reject a value above 10000. A token carrying this member **MUST** also carry the attenuation proof required by Section 5.7.

For delegated-token sibling admission, the parent identifier is the last delegation link's capability_id, the child identifier is the token's id, and an absent share is treated as 10000. The registry **MUST** reject an unknown parent, a changed share for an already admitted child, or an existing sibling sum plus the proposed share greater than the registered parent share. Re-admitting the same child and share is idempotent. Each intermediate parent requires its own registration; admitting a child does not automatically register it as a parent.

The registry compares parent and child shares on one basis-point scale. It does not scale max_invocations, max_cost_per_invocation, or max_total_cost, and the share alone does not establish a shared monetary balance. The reference hosted implementation keeps this registry in process-local memory. Hosted admissions use reference-counted holder leases and release the child edge when its last holder releases. A portable verify-only admission can remain in that registry with no holder lease; this does not imply persistence across restart.

7.5. Aggregate Invocation Budgets

The token's `aggregate_invocation_budget` is an object with `scope` (`capability` or `delegation_family`), `max_invocations` (unsigned 32-bit integer), and `root_binding` when needed. Capability scope forbids a root binding and delegation. Delegation-family scope requires one. Neither this object nor its root body has an epoch member.

The signed root binding has `body`, `signature`, and the non-default `algorithm` hint when supplied. The body has:

- `schema`: `chio.aggregate-budget-root.v1`.
- `root_capability_id`, `root_capability_hash`, `root_scope_hash`: strings committing the root token and scope.
- `root_issuer`, `root_subject`: encoded public keys.
- `max_invocations`: unsigned 32-bit integer.
- `root_expires_at`: unsigned 64-bit Unix seconds.

Let C(x) denote the typed canonical UTF-8 JSON bytes in Section 4.1, and H(x) denote SHA-256 rendered as 64 lowercase hexadecimal characters without 0x. A domain string below is UTF-8 followed by one zero octet. Let R be the authenticated direct root token, S = H(C(R.scope)), and N the family's max_invocations.

The root commitment projection is exactly:

```
{
  "root_capability_id": R.id,
  "root_issuer": R.issuer,
  "root_subject": R.subject,
  "root_scope_hash": S,
  "root_issued_at": R.issued_at,
  "root_expires_at": R.expires_at,
  "aggregate_scope": "delegation_family",
  "max_invocations": N
}
```

Here the expressions R.id and similar expressions denote copied values, not JSON strings. root_capability_hash is:

```
H("chio.aggregate-budget-root-commitment.v1" || 0x00 ||
  C(projection))
```

The binding body B is exactly the following projection:

```
{
  "schema": "chio.aggregate-budget-root.v1",
  "root_capability_id": R.id,
  "root_capability_hash": the commitment hash above,
  "root_issuer": R.issuer,
  "root_subject": R.subject,
  "max_invocations": N,
  "root_expires_at": R.expires_at,
  "root_scope_hash": S
}
```

The commitment's root_issued_at and aggregate_scope are not members of B. A root-binding signer **MUST** use the key identified by B.root_issuer and sign the signing wrapper (Section 4.5) of C(B) under `chio.aggregate-budget-root.v1`.

The signature is placed beside body, with a non-default algorithm hint when supplied. A verifier **MUST** reject an unsupported body schema, a present algorithm hint inconsistent with the signature, or a failed signature under B.root_issuer. A verifier comparing the binding with R **MUST** require equality of root_capability_id, root_capability_hash, root_issuer, root_subject, max_invocations, root_expires_at, and root_scope_hash with their recomputed root values.

For the complete binding E, including its signature and any non-default algorithm hint, the descendant marker digest is:

```
H("chio.aggregate-budget-root-binding-digest.v1" || 0x00 || C(E))
```

The shared family-accounting owner is:

```
H("chio.aggregate-budget-family-key.v1" || 0x00 || C(B))
```

The owner hashes the body, while the marker digest hashes the complete binding. Neither the aggregate object nor B has an epoch member.

A capability-scoped aggregate object **MUST** omit root_binding and **MUST NOT** authorize delegation. A delegation-family object **MUST** carry root_binding, and its maximum **MUST** equal the binding body's maximum. An implementation of this aggregate profile **MUST** reject family delegation longer than one hop.

A family-descendant verifier **MUST** authenticate a direct root token with an empty delegation chain and a trusted root issuer. It **MUST** reject the descendant unless all of the following hold:

- The first link's capability_id equals the root id, its delegator equals the root subject, and its scope_hash equals the root scope hash.
- The first link timestamp is at least root.issued_at and less than root.expires_at; the child issued_at is at least the final link timestamp.
- The final delegatee equals the child's subject, the child's expiry is no later than the root expiry, and its scope passes delegable attenuation against the root scope.
- The child has delegation_family scope, the same maximum, and a root binding whose typed canonical JSON is identical to the root's binding.
- Every link carries aggregate_budget with the binding digest and maximum. When an attenuation witness is present, it carries the same marker as aggregateBudget.

An aggregate-feature verifier **MUST** reject aggregate evidence when aggregate_invocation_budget was not negotiated. An enforcement path without the required durable composite admission **MUST** reject the aggregate request rather than infer enforcement from parsing.

Composite quota admission **MUST** check every participating quota before writing any reservation. It **MUST** reject a changed stored maximum, an exhausted reserved-plus-captured count, or a maximum of zero. Its quota set **MUST** be strictly sorted by unique key and contain at most eight entries. Capability aggregate quotas use the capability id as owner under chio.aggregate-capability-invocation.v1; family quotas use the derived family owner under chio.aggregate-family-invocation.v1. Neither key has a grant index. Monetary rows remain per capability and grant.

The reference portable evaluator rejects aggregate invocation enforcement, and the ordinary capability issuance validator does not issue this feature. The supplied v1 token JSON schema also lacks the aggregate marker members in its closed delegation-link and witness definitions. Passing that schema is therefore not acceptance evidence for a family descendant described here. This schema limitation does not remove the signed marker checks.

7.6. Guarantee Levels

Guarantee identifiers and ranks are advisory_posthoc (0), single_node_atomic (1), partition_escrowed (2), and ha_linearizable (3). A guarantee-floor verifier **MUST** reject an unknown requested floor, a missing budget-authority block, an unknown claimed level, or a claimed level ranked below the requested floor.

A producer of budget execution metadata **MUST** obtain the guarantee identifier from its accounting store. The reference local stores return single_node_atomic; the remote trust-control accounting client returns advisory_posthoc. The reference implementation does not supply a store that returns partition_escrowed or ha_linearizable. Recognizing those identifiers does not establish those backings. Likewise, a single_node_atomic label by itself does not establish durable storage or distributed accounting truth.

Hold authority can carry `authority_id`, `lease_id`, and `lease_epoch`. A budget term is `authority_id:lease_epoch`. Local authority uses a kernel-key identifier, lease `single-node`, and epoch 1. A durable serving store checks its ownership fence within the transaction. An epoch on a hold authority is distinct from an aggregate invocation budget.

7.7. Authoritative Spend

An execution nonce has exactly the JSON-schema profile envelope members nonce and signature. The body-only profile is `chio.execution_nonce.v1`; the operation-owned profile is `chio.operation_execution_nonce.v1` (Section 7.7.1). They share this nonce body and binding:

Member	Type	Presence
schema	chio.execution_nonce.v1 or chio.operation_execution_nonce.v1	required
nonce_id	nonempty string	required
issued_at	integer from 0 through 9223372036854775807, Unix seconds	required
expires_at	integer in the same range, Unix seconds	required
bound_to	object in the following table	required
reserved_hold_id	nonempty string	omitted when absent
reserving_request_id	nonempty string	omitted when absent

Table 6

bound_to member	Type	Presence
subject_id	nonempty string	required
request_id	nonempty string	required
capability_id	nonempty string	required
tool_server	nonempty string	required
tool_name	nonempty string	required
parameter_hash	64 lowercase hexadecimal SHA-256 characters	required

Table 7

For this JSON-schema profile, a receiver **MUST** reject missing required members, unknown members at any of the three object levels, and values outside the listed types and ranges. The reference typed compatibility decoder represents the two timestamps as signed 64-bit integers, does not universally reject unknown members, and defaults an omitted request_id to an empty string. Such decoding does not establish schema-profile acceptance; normal execution verification rejects an empty request_id.

A `chio.execution_nonce.v1` signer **MUST** sign the signing wrapper (Section 4.5) of the typed canonical JSON of the complete nonce body, including each optional body member that is present, under `chio.execution_nonce.v1`. The signature is beside nonce, not inside it. Reference kernel issuance uses its Ed25519 key; the schema's additional signature spellings do

not establish an additional nonce issuance backend. Reference nonce identifiers are UUIDv7 strings and the default lifetime is 30 seconds. Enforcement of this body-only profile is an explicit runtime choice and is off by default.

A `chio.execution_nonce.v1` verifier for a presented tool-call request **MUST** reject an unsupported schema, now greater than or equal to expires_at, an empty request_id, a mismatch in any of the six binding fields against the independently derived expected request, or an invalid signature under the kernel key. Before accepting that execution, it **MUST** reserve the nonce identifier exactly once in its replay store and reject a replay or replay-store failure. The consumed marker remains needed through the signed expiry.

Reconciliation by a caller-reserved `chio.execution_nonce.v1` nonce is a distinct path. The reference implementation validates the nonce using its own signed binding, then separately checks the presented arguments hash, the named open hold, the hold's capability, and the reserved monetary currency. It clamps realized units to remaining exposure and closes the hold atomically. Consumption is attempted only after settlement; a failure to write the consumed marker is nonfatal because the closed hold rejects a second settlement. This path does not independently derive a fresh caller subject, request, server, or tool binding.

A consumer recognizing chio.mediated_spend.v1 through the authoritative-spend receipt predicate **MUST** reject the receipt unless every following condition holds:

1. The receipt signature is valid under its embedded kernel_key and that key belongs to the consumer's admitted kernel signer set.
2. receipt_kind is mediated_decision, boundary_class is prevent, trust_level is mediated, the decision is allow, and observation_outcome is absent.
3. metadata.budget_authority contains a nonempty hold_id and mediated_spend.profile equal to chio.mediated_spend.v1.
4. authorize.exposure_units is positive. terminal.disposition is reconciled and terminal.event_id is present.
5. execution_nonce_id equals the presented signed nonce's nonce_id. If that nonce has reserved_hold_id, it equals hold_id.
6. The nonce's capability_id, tool_server, tool_name, and parameter_hash equal the receipt's corresponding call fields.
7. The nonce signature over the signing wrapper of the complete nonce body under chio.execution_nonce.v1 verifies under the same admitted kernel_key.

This predicate does not independently check nonce subject_id, request_id, schema, expiry, replay state, or the guarantee floor. It also does not query a budget store or quorum and does not require a budget_commit_index. Its terminal realized accounting amount is terminal.realized_spend_units, which can differ from the reported financial.cost_charged. Execution-time nonce verification and guarantee-floor checking are separate checks; successful evaluation of this predicate alone does not establish them. This body-only signature predicate does not verify an operation-owned nonce's signing context.

7.7.1. Operation-Owned Execution Nonces

The `chio.operation_execution_nonce.v1` profile uses the same signed nonce envelope. Its signer **MUST** sign the signing wrapper (Section 4.5) under `chio.operation_execution_nonce.v1` of the canonical JSON of an object with exactly these members:

Member	Signed value
operation_id	Independently authenticated admission operation ID string.
nonce	Complete nonce body, including its chio.operation_execution_nonce.v1 schema and all present optional members.

Table 8

The operation ID is signing context; it is not an additional member of the signed-nonce transport envelope. A verifier **MUST** reconstruct it from the trusted admission binding, including the authenticated namespace, capability artifact, request, policy, and effect class. The presented nonce **MUST NOT** select or change that context. Changing the schema label does not translate a signature between the two nonce profiles.

A verifier **MUST** require the operation profile's schema, a valid signature under the independently trusted issuer, and all six request-binding fields to match the independently derived expected request. It **MUST** reject a negative issued_at, issuance after the trusted verification time, expires_at less than or equal to issued_at, and a verification time greater than or equal to expires_at. It **MUST** reject expires_at above `floor(9007199254740991 / 1000)` Unix seconds. The reference implementation bounds both the canonical signing context and serialized reservation material to 16 KiB.

Fresh operation-owned issuance and capture **MUST** require this profile and operation-owned durable nonce custody. Signature verification alone does not reserve a nonce or authorize a dispatch. Historical verification uses independently authenticated operation, request, issuer, and original verification-time evidence; success does not renew execution authority.

The `chio.execution_nonce.v1` replay-store verifier **MUST** reject `chio.operation_execution_nonce.v1` before consuming a replay marker. Fresh operation-owned reservations and capture **MUST** reject `chio.execution_nonce.v1`, even if its signature is valid and a separate replay cache reports it unused.

7.8. Metering Metadata

The receipt's financial and budget_authority metadata are inside the signed receipt projection. A producer of a financial block **MUST** include the required fields below and omit an optional field that has no value.

financial member	Type	Presence	Meaning
grant_index	unsigned 32-bit integer	required	Matching grant index.
cost_charged	unsigned 64-bit integer	required	Recorded financial amount; can exceed ledger-realized units on a reported overrun.
currency	string	required	Recorded denomination. A zero-exposure invocation reconciliation uses an empty string.
budget_remaining	unsigned 64-bit integer	required	Recorded remaining amount. Reconciled paths subtract committed units from the recorded total, saturating at zero; an exhaustion denial can record the total without reading current usage.
budget_total	unsigned 64-bit integer	required	Recorded grant ceiling or path-specific bounded envelope. The inline path uses the maximum u64 value when no total ceiling exists.
delegation_depth	unsigned 32-bit integer	required	Recorded delegation-chain depth.
root_budget_holder	string	required	Recorded holder identifier; inline receipts use the capability issuer key encoding.
settlement_status	string	required	not_applicable, pending, settled, or failed.
payment_reference	string	omitted when absent	Optional external reference; settlement rails are outside this document.
cost_breakdown	JSON value	omitted when absent	Optional itemized report.
oracle_evidence	object	omitted when absent	Optional conversion evidence; verification of that evidence is outside this section.

financial member	Type	Presence	Meaning
attempted_cost	unsigned 64-bit integer	omitted when absent	Attempted amount on a denial.

Table 9

A producer of a full hold-lineage budget_authority block **MUST** use the member names and presence rules below. A label-only denial can carry only guarantee_level, authority_profile, and metering_profile; that reduced block does not satisfy the authoritative-spend profile.

In the full hold-lineage block, authority_profile is `authoritative_hold_event`, metering_profile is `max_cost_preauthorize_then_reconcile_actual`, and guarantee_level is one of the identifiers in Section 7.6.

budget_authority member	Type	Presence in full block
guarantee_level	string	required
authority_profile	string	required
metering_profile	string	required
hold_id	string	required
budget_term	string	omitted when absent
authority	object	omitted when absent
authorize	object	required
invocation_capture	object	omitted when absent
terminal	object	omitted when absent
execution_nonce_id	string	omitted when absent
mediated_spend	object containing profile	omitted when absent; profile is chio.mediated_spend.v1 when included

Table 10

authority member	Type	Presence when authority is included
authority_id	string	required
lease_id	string	required
lease_epoch	unsigned 64-bit integer	required

Table 11

authorize member	Type	Presence
exposure_units	unsigned 64-bit integer	required
committed_cost_units_after	unsigned 64-bit integer	required
event_id	string	omitted when absent
budget_commit_index	unsigned 64-bit integer	omitted when absent

Table 12

terminal member	Type	Presence when terminal is included
disposition	string	required; reference emitted values are reconciled, reversed, and cancelled_before_dispatch
exposure_units	unsigned 64-bit integer	required
realized_spend_units	unsigned 64-bit integer	required
committed_cost_units_after	unsigned 64-bit integer	required
event_id	string	omitted when absent
budget_commit_index	unsigned 64-bit integer	omitted when absent

Table 13

A successful invocation_capture object records invocation_count_after and includes event_id and budget_commit_index when available. An ambiguous-capture denial instead records event_id with invocation_capture_ambiguous and admission_retained set to true. These are distinct projections of capture state.

When a producer links a nonce to this block, it **MUST** include both execution_nonce_id and mediated_spend.profile. Receipt consumers distinguish reported financial amounts from the committed authorize, capture, and terminal lineage. The typed financial and budget_authority decoders do not universally reject unknown members; these metadata blocks are not a strict unknown-member-rejection profile.

Auxiliary metering records can measure compute time, data volume, API cost, warehouse queries, and custom dimensions. They are not automatically attached to kernel receipts and do not substitute for the durable hold contract in this section.

8. Governed Transactions

A governed transaction is a tool call that carries a declared intent: a JSON object that names the call's target and purpose and can bound its cost, name a seller, carry a price quote, and assert the call chain the call belongs to. When a grant requires it, the call also carries an approval, signed by one trusted approver or by a quorum of eligible approvers. The kernel checks the intent against the call and the grant, verifies the approval, classifies the asserted call chain, and records the intent hash and its findings in the receipt.

A `tool_call_request` (Section 10.2) carries these objects in four **OPTIONAL** members, each omitted when absent:

`governed_intent`: The intent (Section 8.2).

`approval_token`: A single approval (Section 8.4).

`approval_tokens`: An array of 1 to 32 approval tokens that vote on a threshold proposal (Section 8.5). It is omitted when empty.

`threshold_approval_proposal`: The proposal that the tokens in `approval_tokens` vote on.

The MCP binding carries the same objects in `_meta` (Section 11.6). An approval binds to the call's request identifier: the `id` of a native `tool_call_request`, or `_meta.chioRequestId` on the MCP binding.

An agent **MUST NOT** send both `approval_token` and a non-empty `approval_tokens`. An agent that sends `threshold_approval_proposal` **MUST** send a non-empty `approval_tokens`, and an agent that sends a non-empty `approval_tokens` **MUST** send `threshold_approval_proposal`. The kernel denies a call that breaks these rules.

8.1. Applying the Checks

The kernel applies the checks in this section to a grant that matches the call (Section 5.8) when the grant carries any of the constraints `governed_intent_required`, `require_approval_above`, `seller_exact`, `minimum_runtime_assurance`, or `minimum_autonomy_tier` (Section 5.3), or when the request carries any of the four members above. When the checks apply, the kernel **MUST** deny a call that carries no `governed_intent`.

The kernel runs the checks for each matching grant in turn, before it runs the guards for that grant. It does not use a grant whose checks fail, and it goes on to the next matching grant. If no matching grant passes, the kernel denies the call. It runs the checks again for the chosen grant immediately before dispatch.

Unless this section says otherwise, a governed validation failure **MUST** prevent dispatch. For authorization artifacts representable by the typed digest structures, the kernel returns a signed denial with decision `deny` and guard `kernel`; the native result is `policy_denied`. Structurally inconsistent approvals can fail receipt-metadata construction and return `internal_error` instead. A fallback error receipt lacks the governed context and is not authenticated under the configured kernel key (Section 10.4).

8.2. Governed Intent

The intent is a JSON object. Its own members and those of `commerce` have snake_case names. The members of `metered_billing`, `call_chain`, and `autonomy` have camelCase names. **OPTIONAL** members are omitted when absent.

Field	Type	Presence	Meaning
`id`	string	**REQUIRED**	Identifier chosen by the agent. The kernel copies it to the receipt.
`server_id`	string	**REQUIRED**	Target tool server.
`tool_name`	string	**REQUIRED**	Target tool.
`purpose`	string	**REQUIRED**	Free text; can be empty.
`max_amount`	MonetaryAmount object (Section 7.1)	**OPTIONAL**	Largest amount the intent authorizes.
`commerce`	object	**OPTIONAL**	Seller context (Table 15).
`metered_billing`	object	**OPTIONAL**	Settlement mode and quote (Section 8.2.3).
`runtime_attestation`	object	**OPTIONAL**	Runtime attestation evidence, in the form defined in Section 12.1.
`call_chain`	object	**OPTIONAL**	Asserted call chain (Table 16).
`autonomy`	object	**OPTIONAL**	Autonomy tier (Section 8.2.2).
`context`	JSON value	**OPTIONAL**	Free-form context. Two keys are reserved (Section 8.7).
`body`	object	**OPTIONAL**	Omitted for a tool call. Its `kind` is `tool_invocation` or `active_response_plan`.

Table 14: Governed Intent Members

Field	Type	Presence	Meaning
`seller`	string	**REQUIRED**	Seller or payee that the call pays.
`shared_payment_token_id`	string	**REQUIRED**	Reference to the payment credential the seller will use.
`settlement_destination_ref`	string	**OPTIONAL**	Reference to the account that receives the payment.

Table 15: Commerce Members

Field	Type	Presence	Meaning
`chainId`	string	**REQUIRED**	Identifier of the call chain.
`parentRequestId`	string	**REQUIRED**	Request identifier of the parent call.
`parentReceiptId`	string	**OPTIONAL**	Receipt identifier of the parent call.
`originSubject`	string	**REQUIRED**	Subject that started the chain.
`delegatorSubject`	string	**REQUIRED**	Subject that delegated to the caller.

Table 16: Call Chain Members

The `autonomy` object has a **REQUIRED** `tier`, one of `direct`, `delegated`, and `autonomous` in increasing order, and an **OPTIONAL** string `delegationBondId`.

The following intent is illustrative; its values come from the tests of the reference implementation.

```
{
  "id": "intent-1",
  "server_id": "srv-pay",
  "tool_name": "charge",
  "purpose": "pay supplier",
  "max_amount": { "units": 500, "currency": "USD" },
  "commerce": {
    "seller": "merchant.example",
    "shared_payment_token_id": "spt_123",
    "settlement_destination_ref": "acct:merchant-primary"
  },
  "metered_billing": {
    "settlementMode": "allow_then_settle",
    "quote": {
      "quoteId": "quote-1",
      "provider": "meter.chio",
      "billingUnit": "1k_tokens",
      "quotedUnits": 12,
      "quotedCost": { "units": 300, "currency": "USD" },
      "issuedAt": 950,
      "expiresAt": 1300
    },
    "maxBilledUnits": 20
  },
  "call_chain": {
    "chainId": "chain-1",
    "parentRequestId": "req-parent-1",
    "parentReceiptId": "rc-parent-1",
    "originSubject": "origin-subject",
    "delegatorSubject": "delegator-subject"
  }
}
```

Its intent hash is `6584f466fdfd57b71861d268d2ffe45a48b179a5999ab9502cb58e935bdb875d`.

8.2.1. Intent Hash

The intent hash is the SHA-256 digest (Section 4.2), as 64 lowercase hexadecimal characters, of the canonical JSON serialization (Section 4.1) of the intent. It has no domain-separation prefix. It covers `context`, including any proof or token carried there.

The kernel computes the hash over the intent as it parsed it. It ignores members that this section does not define, so they are not hashed; an undefined member inside `max_amount` or `verifiedOutcome` makes the request malformed instead. It also omits a `body` whose `kind` is `tool_invocation`. Approval tokens, threshold proposals, and receipts carry this hash.

8.2.2. Intent Checks

When the checks apply, the kernel **MUST** verify the following and **MUST** deny the call if any item fails:

1. `server_id` and `tool_name` equal the target of the call.
2. `body` is absent or has the `kind` `tool_invocation`.
3. If `runtime_attestation` is present, it passes the kernel's local verification of attestation evidence. This holds even when no grant requires attestation.
4. If `call_chain` is present:
   - `chainId`, `parentRequestId`, `originSubject`, and `delegatorSubject` each contain a character other than white space, and so does `parentReceiptId` when present;
   - `parentRequestId` differs from the call's request identifier;
   - if the capability token has a delegation chain (Section 5.5), `delegatorSubject` equals the public key string (Section 4.4) of the `delegator` of the last link, and `originSubject` equals that of the first link;
   - if the kernel evaluates the call within a parent request of the same session, `parentRequestId` equals that parent's request identifier;
   - a proof or token in `context` passes Section 8.7.
5. If `commerce` is present, `seller` and `shared_payment_token_id` each contain a character other than white space, and `max_amount` is present. A `settlement_destination_ref` is not empty, has no leading or trailing white space, has at most 2048 characters, and has no control characters.
6. If `autonomy` is present, a `direct` tier carries no non-whitespace `delegationBondId`. The kernel trims a bond identifier for lookup and treats an empty result as absent. A `delegated` or `autonomous` tier requires `call_chain`, a `runtime_attestation` that the kernel accepts at tier `attested` or `verified` respectively, or higher, and a `delegationBondId` that the kernel resolves to a delegation bond it accepts for the call. Delegation bonds are outside the scope of this document.
7. If `metered_billing` is present, it passes Section 8.2.3.

8. If the grant declares `max_cost_per_invocation` and the intent carries `max_amount`, the two have the same currency, and `max_amount.units` is at least the ceiling's `units`, the amount that the kernel holds before dispatch (Section 7.3).

8.2.3. Metered Billing

The `metered_billing` object carries a price quote for a tool that bills by usage:

Field	Type	Presence	Meaning
`settlementMode`	string	**REQUIRED**	`must_prepay`, `hold_capture`, or `allow_then_settle`.
`quote`	object	**REQUIRED**	The quote (Table 18).
`maxBilledUnits`	integer	**OPTIONAL**	Largest number of units the call can be billed.
`verifiedOutcome`	object	**OPTIONAL**	Reserved. A kernel denies an intent that carries it.

Table 17: Metered Billing Members

Field	Type	Presence	Meaning
`quoteId`	string	**REQUIRED**	Quote identifier.
`provider`	string	**REQUIRED**	Party that issued the quote.
`billingUnit`	string	**REQUIRED**	Unit of `quotedUnits`, for example `1k_tokens`.
`quotedUnits`	integer	**REQUIRED**	Quoted number of units.
`quotedCost`	MonetaryAmount object	**REQUIRED**	Quoted price.
`issuedAt`	integer	**REQUIRED**	Issue time, Unix seconds.
`expiresAt`	integer	**OPTIONAL**	Expiry time, Unix seconds.

Table 18: Quote Members

The kernel **MUST** deny the call unless all of the following hold:

1. `quoteId`, `provider`, and `billingUnit` each contain a character other than white space, and `quotedUnits` is greater than 0.
2. `expiresAt`, when present, is greater than `issuedAt`, and the quote is valid at the current time `now`: `issuedAt <= now`, and `now < expiresAt` when `expiresAt` is present.
3. `maxBilledUnits`, when present, is at least `quotedUnits` and greater than 0.

4. Neither `verifiedOutcome` is present nor `billingUnit` is `verified_outcome`. This version of the protocol does not define outcome-based billing.
5. `quotedCost.currency` equals the currency of `max_amount` when present, and the currency of the grant's `max_cost_per_invocation`, or else of its `max_total_cost`, when the grant declares one.

The modes `hold_capture` and `allow_then_settle` add no checks; the kernel records them in the receipt (Section 8.8). For `must_prepay`, the kernel **MUST** also deny the call in each of these cases:

- the kernel is not configured to authorize payments;
- the grant declares `max_total_cost` but not `max_cost_per_invocation`;
- `quotedCost.units` exceeds the `units` of either ceiling that the grant declares, or `max_amount.units` is below `quotedCost.units`.

The quoted cost of a `must_prepay` intent counts toward `require_approval_above` (Section 8.3). When the kernel answers such a call with an execution nonce instead of executing it (Section 10.2), the call **MUST** carry a singular `approval_token`.

8.3. Intent Constraints

Three grant constraints (Section 5.3) govern intents and approvals. They do not affect whether a grant matches a call; the kernel enforces them in the checks of this section. A delegated grant keeps each of them unchanged (Section 5.6).

```
[
  { "type": "governed_intent_required" },
  { "type": "require_approval_above",
    "value": { "threshold_units": 5000 } },
  { "type": "seller_exact", "value": "merchant.example" }
]
```

`governed_intent_required`: The call **MUST** carry `governed_intent`. The kernel then applies every check in Section 8.2.2.

`require_approval_above`: The call **MUST** carry an approval when its requested amount is greater than or equal to `threshold_units`. If a grant carries several of these constraints, the largest threshold applies. The requested amount, in minor units, is the largest of: the base; `quotedCost.units` when `settlementMode` is `must_prepay`; and 0. The base is the `units` of the grant's `max_cost_per_invocation` when the grant declares it, and otherwise the `units` of the intent's `max_amount` when present. A `threshold_units` of 0 therefore always requires approval. The constraint carries no currency, and a smaller `max_amount` does not lower the requested amount when the grant declares a ceiling.

`seller_exact`: The intent **MUST** carry `commerce`, and `commerce.seller` **MUST** equal the constraint's value exactly, with no case folding or normalization. If a grant carries several of these constraints, the last one applies.

A commerce intent under a grant that charges per call also needs an approval. When the grant's `max_cost_per_invocation` has `units` greater than 0 and the intent carries `commerce`, the intent **MUST** carry `settlement_destination_ref`, and the call **MUST** carry an approval, whatever `require_approval_above` requires.

An approval is a singular token that passes Section 8.4 or a threshold set that passes Section 8.5. The kernel **MUST** deny a call that needs an approval and carries neither. The kernel verifies a presented approval even when no approval is required, and an approval that fails verification denies the call.

The constraints `minimum_runtime_assurance` and `minimum_autonomy_tier` also trigger the checks. For each, the largest value in a grant applies. The first requires a `runtime_attestation` that the kernel accepts at that tier or higher, in the order `none`, `basic`, `attested`, `verified`. The second requires an `autonomy` object whose `tier` is at least the required tier.

8.4. Approval Tokens

An approval token is a signed JSON object that approves one intent for one request by one subject. All member names are snake_case.

Field	Type	Presence	Meaning
`id`	string	**REQUIRED**	Token identifier.
`approver`	string	**REQUIRED**	Public key of the signer (Section 4.4).
`subject`	string	**REQUIRED**	Subject key of the capability token that authorizes the call.
`governed_intent_hash`	string	**REQUIRED**	Intent hash (Section 8.2.1).
`request_id`	string	**REQUIRED**	Request identifier of the call.
`threshold_proposal_hash`	string	**OPTIONAL**	Digest of the proposal that the token votes on (Section 8.5).
`issued_at`	integer	**REQUIRED**	Start of validity, Unix seconds.
`expires_at`	integer	**REQUIRED**	End of validity, exclusive, Unix seconds.
`decision`	string	**REQUIRED**	`approved` or `denied`.
`algorithm`	string	**OPTIONAL**	`ed25519`, `p256`, `p384`, or `hybrid`. Informational; omitted when absent or `ed25519`.
`signature`	string	**REQUIRED**	Signature by `approver` (Section 4.3).

Table 19: Approval Token Members

The approver signs the signing wrapper (Section 4.5) of the canonical JSON of the token without `algorithm` and `signature` under `chio.governed-approval-token.v1`. The token digest is the lowercase hexadecimal SHA-256 digest of the ASCII bytes `chio.governed-approval-token.v1`, one zero byte, and the canonical JSON of the whole token, `signature` included.

The following token is taken from the schema fixtures of the reference implementation. Its hashes and signature are placeholders. It is a vote on a threshold proposal; a singular token omits `threshold_proposal_hash`.

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

{
  "id": "approval-1",
  "approver": \
  "8888888888888888888888888888888888888888888888888888888888888888",
  "subject": \
  "2222222222222222222222222222222222222222222222222222222222222222",
  "governed_intent_hash": \
  "1111111111111111111111111111111111111111111111111111111111111111",
  "request_id": "request-1",
  "threshold_proposal_hash": \
  "9999999999999999999999999999999999999999999999999999999999999999",
  "issued_at": 110,
  "expires_at": 190,
  "decision": "approved",
  "signature": "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa\
  aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa\
  aaaaaaaaaa"
}
```

The kernel **MUST** verify a singular `approval_token` as follows, at the current time `now`, and **MUST** deny the call if a step fails:

1. `issued_at <= now < expires_at`.
2. `request_id` equals the call's request identifier.
3. `governed_intent_hash` equals the intent hash.
4. `subject` equals the `subject` of the capability token.
5. `decision` is `approved`.
6. `approver` is the kernel's own key, a configured capability-authority key, or another issuer key trusted by the kernel (Section 5.8), and the signature verifies.
7. `expires_at - issued_at` is at most 3600 seconds.

The kernel does not check `id` or `threshold_proposal_hash` of a singular token.

A singular token authorizes one dispatch. When it reserves the call for dispatch, the kernel records the tuple of the subject key, the request identifier, and the intent hash until the token's `expires_at`. The kernel **MUST** deny the call as a replay if the tuple is already recorded and has not expired. Because the record is keyed by the tuple and not by the token's `id`, a second token for the same subject, request, and intent is also refused. The kernel releases an owned replay reservation only after definite pre-effect rejection with successful rollback. After acknowledged or ambiguous external payment authorization it retains the marker, even when later checks prevent tool dispatch. Uncertain cleanup can also retain it; a denial alone does not authorize replay. If the kernel cannot record the tuple, for example because its record store is full, it **MUST** deny the call. A kernel **SHOULD** keep these records across restarts; a kernel that loses them can accept the same approval again until it expires.

8.5. Threshold Approval

A threshold approval authorizes a call with a quorum of votes. A proposal, signed by a policy authority, fixes the request, the intent, the subject, the authorizing capability token, the policy, the quorum, the set of eligible approvers, and a deadline. Each vote is an approval token (Section 8.4) whose `threshold_proposal_hash` is the proposal's digest.

The kernel accepts a threshold set only when the feature `threshold_governed_approvals` is supported for the request's origin (Section 5.9). It **MUST** deny the call otherwise.

The kernel takes the threshold requirement for the call from its active policy, by policy hash, tool server, and tool. The policy language is out of scope (Section 1.2). A requirement has:

- a policy hash: 64 lowercase hexadecimal characters, equal to the kernel's active policy hash;
- an eligible set of 1 to 32 approvers, each a JSON object with the string members `identifier` and `public_key`. Identifiers are unique, not empty, free of leading and trailing white space and of control characters; public keys are unique;
- a threshold from 1 to the number of eligible approvers;
- a timeout from 1 to 3600 seconds.

The eligible set digest is the lowercase hexadecimal SHA-256 digest of the ASCII bytes `chio.approver-set.v1`, one zero byte, and the canonical JSON of the array of eligible approvers, sorted by `identifier` and then by `public_key`.

8.5.1. Proposal

A proposal is a JSON object with snake_case members. A receiver rejects a proposal with members that Table 20 does not list.

Field	Type	Presence	Meaning
`schema`	string	**REQUIRED**	`chio.threshold-approval-proposal.v1`.
`proposal_id`	string	**REQUIRED**	Proposal identifier.
`request_id`	string	**REQUIRED**	Request identifier of the call.
`governed_intent_hash`	string	**REQUIRED**	Intent hash (Section 8.2.1).
`subject`	string	**REQUIRED**	Subject key of the capability token.
`authorizing_capability_digest`	string	**REQUIRED**	SHA-256 digest of the canonical JSON of the capability token, signature included, with no prefix.
`policy_hash`	string	**REQUIRED**	Policy hash of the requirement.
`threshold`	integer	**REQUIRED**	Quorum size.
`eligible_set_digest`	string	**REQUIRED**	Eligible set digest.
`proposal_created_at`	integer	**REQUIRED**	Start of the voting window, Unix seconds.
`proposal_deadline`	integer	**REQUIRED**	End of the voting window, exclusive, Unix seconds.
`policy_authority`	string	**REQUIRED**	Public key of the signer.
`algorithm`	string	**OPTIONAL**	As for approval tokens.
`signature`	string	**REQUIRED**	Signature by `policy_authority`.

Table 20: Threshold Proposal Members

The four digest members are 64 lowercase hexadecimal characters. `proposal_id` and `request_id` are not empty and have no leading or trailing white space. `threshold` is greater than 0, and `proposal_created_at` is less than `proposal_deadline`. The policy authority signs the signing wrapper (Section 4.5) of the canonical JSON of the proposal without `algorithm` and `signature` under `chio.threshold-approval-proposal.v1`. The proposal digest is the SHA-256 digest, in lowercase hexadecimal, of the ASCII bytes `chio.threshold-approval-proposal.v1`, one zero byte, and the canonical JSON of the whole proposal, `signature` included.

The following proposal is taken from the schema fixtures of the reference implementation; its digests and signature are placeholders.

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

{
  "schema": "chio.threshold-approval-proposal.v1",
  "proposal_id": "proposal-1",
  "request_id": "request-1",
  "governed_intent_hash": \
  "1111111111111111111111111111111111111111111111111111111111111111",
  "subject": \
  "2222222222222222222222222222222222222222222222222222222222222222",
  "authorizing_capability_digest": \
  "3333333333333333333333333333333333333333333333333333333333333333",
  "policy_hash": \
  "4444444444444444444444444444444444444444444444444444444444444444",
  "threshold": 2,
  "eligible_set_digest": \
  "5555555555555555555555555555555555555555555555555555555555555555",
  "proposal_created_at": 100,
  "proposal_deadline": 200,
  "policy_authority": \
  "6666666666666666666666666666666666666666666666666666666666666666",
  "signature": "77777777777777777777777777777777777777777777777777\
  77777777777777777777777777777777777777777777777777777777777777\
  7777777777"
}
```

8.5.2. Verifying a Threshold Set

The kernel **MUST** verify a threshold set as follows, at the current time `now`, and **MUST** deny the call if a step fails:

1. `approval_tokens` holds 1 to 32 tokens, and the kernel has a threshold requirement for the call.
2. `threshold_approval_proposal` is present, its members are valid as stated above, and `proposal_created_at <= now < proposal_deadline`.
3. `policy_authority` is the kernel's own key, a configured capability- authority key, or another issuer key trusted by the kernel (Section 5.8), and the signature verifies.
4. `request_id`, `governed_intent_hash`, and `subject` match the call as for a singular token. `authorizing_capability_digest` matches the capability token. `policy_hash`, `threshold`, and `eligible_set_digest` equal those of the requirement.
5. `proposal_deadline` equals the smaller of `proposal_created_at` plus the requirement's timeout and the capability token's `expires_at`.
6. Each token is valid at `now`; its `id` is not empty and has no leading or trailing white space; its `request_id`, `governed_intent_hash`, and `subject` match the call; its `decision` is `approved`; and its `threshold_proposal_hash` equals the proposal digest.
7. Each token's `issued_at` is at least `proposal_created_at` and less than `proposal_deadline`, and its `expires_at` is at most `proposal_deadline`.

8. Each token's `approver` is the public key of an eligible approver, and its signature verifies. The kernel's trusted issuer keys do not apply to votes.
9. No two tokens share an `id`, a token digest, or an `approver`. The kernel does not remove duplicates; a duplicate denies the call.
10. The number of approvers is at least `threshold`. Extra valid votes are allowed.

A vote whose `decision` is `denied` fails step 6, so it denies the call rather than counting against the quorum.

The approval set hash identifies the verified set. Its input is a JSON object with the members `token_digests`, an array of the digests of all the tokens sorted in ascending order with no duplicates, and `threshold_proposal_hash`, the proposal digest, together with these members copied from the proposal: `policy_hash`, `threshold`, `eligible_set_digest`, `request_id`, `governed_intent_hash`, `subject`, `authorizing_capability_digest`, `proposal_id`, `proposal_created_at`, and `proposal_deadline`. The hash is the SHA-256 digest, in lowercase hexadecimal, of the ASCII bytes `chio.verified-approval-set.v1`, one zero byte, and the canonical JSON of that object. The order of the tokens in the request does not change it.

A proposal and its votes authorize one call. Before dispatch, the kernel **MUST** record the proposal digest and the approval set hash with its durable record of the call's admission, and **MUST** deny the call if it keeps no durable admission records. It **MUST NOT** record the same proposal, approval set hash, or token digest for a second call. When the kernel resumes an admission it has already recorded, both hashes **MUST** equal the recorded values.

8.5.3. Kernel-Issued Proposals

A kernel also issues proposals itself, when a budget requires cumulative approval (Section 7.3). It signs the proposal with its own key as `policy_authority`, takes `proposal_created_at` from the time it recorded the budget hold, and records the proposal. If the call carries no votes, the kernel does not dispatch it. It signs a receipt whose decision is `deny`, with guard `kernel` and reason `cumulative approval required`, and whose `metadata` has a `threshold_approval` object with the members `proposal_id`, `proposal_hash`, `proposal_deadline`, and `state`, the last with the value `approval_required`. A native agent receives `policy_denied` with guard `approval` (Section 10.4). A later call with votes **MUST** carry the recorded proposal unchanged. This document does not specify how an agent obtains that proposal.

8.6. Provenance Classes

A `call_chain` is the agent's assertion. The intent has no member that states its strength; the kernel assigns an evidence class when it writes the receipt:

`asserted`: The kernel checked only the rules of Section 8.2.2.

`observed`: At least one local evidence source in Table 21 matched the call chain.

`verified`: An upstream proof in `context` passed its checks (Section 8.7).

Source	Condition
`session_parent_request_lineage`	The kernel evaluates the call within a parent request of the same session whose request identifier equals `parentRequestId`.
`local_parent_receipt_linkage`	`parentReceiptId` names a receipt that the kernel holds.
`capability_delegator_subject`	`delegatorSubject` equals the key of the last delegator in the capability token's delegation chain.
`capability_origin_subject`	`originSubject` equals the key of the first delegator.
`upstream_delegator_proof`	A `callChainUpstreamProof` passed its checks.

Table 21: Call-Chain Evidence Sources

When a grant has been selected and its validated evidence is retained for the receipt, the kernel **MUST** assign `verified` if and only if a `callChainUpstreamProof` passed its checks. Otherwise it **MUST** assign `observed` if at least one source matched, and `asserted` if none did. No other evidence raises the class. In particular, a `callChainContinuation` that passes every check adds no source of its own: the kernel records its `tokenId`, and the class stays at `observed` or `asserted` as the other sources decide.

The sources are narrow. `local_parent_receipt_linkage` means only that the identifier resolves to a stored receipt. For a delegated capability token, the rules of Section 8.2.2 already force both subject sources to match, so its call chain is at least `observed`. A receipt for a call that the kernel denied because no grant passed its checks carries the class `asserted`.

8.7. Call-Chain Continuation

Two keys of the intent's `context` carry evidence for a `call_chain`: `callChainUpstreamProof` and `callChainContinuation`. The kernel reads them only when `context` is a JSON object and `call_chain` is present. A key whose value is `null` counts as absent, and a value that does not parse as the object below denies the call. If `callChainContinuation` is present, the kernel validates it and ignores `callChainUpstreamProof`. Both objects have camelCase members, and each is signed by its `signer` over the signing wrapper (Section 4.5) of the canonical JSON of all its members except `signature`, under `chio.call_chain_continuation.v1` for a continuation and `chio.governed-upstream-call-chain-proof.v1` for an upstream proof.

An upstream proof is a statement by the delegator that handed the call chain to the caller. It has the call-chain members of Table 16 together with `signer` and `subject` (public key strings), `issuedAt` and `expiresAt` (integers, Unix seconds), and `signature`. The kernel **MUST** deny the call unless the signature verifies; `issuedAt <= now < expiresAt`; `subject` equals the capability token's `subject`; the capability token has a delegation chain and `signer` equals the `delegator` of its last link; and the five call-chain members equal those of the intent's `call_chain`, with `parentReceiptId` absent from both or equal.

A continuation token carries call-chain context from a parent call to a child call:

Field	Type	Presence	Meaning
`schema`	string	**REQUIRED**	Continuation schema.
`tokenId`	string	**REQUIRED**	Token identifier.
`signer`	string	**REQUIRED**	Public key of the signer.
`subject`	string	**REQUIRED**	Subject key of the child's capability token.
`chainId`, `parentRequestId`, `originSubject`, `delegatorSubject`	string	**REQUIRED**	As in Table 16.
`parentReceiptId`	string	**OPTIONAL**	As in Table 16.
`parentReceiptHash`	string	**OPTIONAL**	SHA-256 digest of the canonical JSON of the parent receipt, signature included, with no prefix.
`parentSessionAnchor`	object	**OPTIONAL**	Reference to the parent's session anchor: `sessionAnchorId` and `sessionAnchorHash`, both strings.
`currentSubject`	string	**REQUIRED**	Public key string of the child's subject.
`parentCapabilityId`	string	**OPTIONAL**	`capability_id` of the last delegation link.
`delegationLinkHash`	string	**OPTIONAL**	SHA-256 digest of the canonical JSON of the last delegation link without its `signature`.
`governedIntentHash`	string	**OPTIONAL**	Intent hash of the child call.
`audience`	object	**OPTIONAL**	`serverId` and `toolName` of the child call, both strings.
`nonce`	string	**OPTIONAL**	Not checked.
`issuedAt`, `expiresAt`	integer	**REQUIRED**	Validity interval, Unix seconds.
`signature`	string	**REQUIRED**	Signature by `signer`.

Table 22: Continuation Token Members

The kernel **MUST** verify a continuation token as follows and **MUST** deny the call if a step fails:

1. `schema` is `chio.call_chain_continuation.v1`, and the signature verifies.
2. `issuedAt <= now < expiresAt`.
3. `subject` equals the capability token's `subject`, and `currentSubject` equals its public key string.
4. `signer` is the kernel's own key, one of its trusted issuer keys (Section 5.8), or the `delegator` of the last link of the capability token's delegation chain.
5. The five call-chain members equal those of the intent's `call_chain`.
6. Each **OPTIONAL** binding that is present matches: `audience` names the call's server and tool; `governedIntentHash` equals the intent hash; `parentCapabilityId` and `delegationLinkHash` match the last delegation link, and a token that carries either one requires a delegated capability token.
7. If `parentReceiptId` names a receipt that the kernel holds, that receipt's signature verifies under the kernel's algorithm policy (Section 6.9). If it names no such receipt, the token carries neither `parentReceiptHash` nor `parentSessionAnchor`. A token without `parentReceiptId` carries no `parentReceiptHash`.
8. `parentReceiptHash`, when present, equals the digest of the parent receipt.
9. `parentSessionAnchor`, when present, equals the kernel's reference for the parent session: that of the live parent session when the kernel evaluates the call within a parent request, and otherwise the `sessionAnchorId` and `sessionAnchorHash` that the parent receipt's `metadata` carries. The kernel denies the call when it has no such reference.

An embedded token's `governedIntentHash` would bind an intent containing that same signed token. This document defines no construction for that self-bound form; senders omit this member on a token embedded in `context`. The kernel does not use `nonce` or `tokenId` to detect replay. It records `tokenId` in the receipt, and the anchor's `sessionAnchorId` when step 9 applied.

This document specifies how a kernel verifies these objects, not when a kernel issues them. Carrying call chains between operators is outside its scope (Section 1.2).

8.8. Governed Context in Receipts

For a normal signed result carrying `governed_intent`, the kernel adds `governed_transaction` to receipt metadata, including on denial. The malformed-authorization error path in Section 8.1 is an exception. Its members are:

Field	Type	Presence	Meaning
`intent_id`, `purpose`, `server_id`, `tool_name`	string	**REQUIRED**	Copied from the intent.
`intent_hash`	string	**REQUIRED**	Intent hash (Section 8.2.1).
`max_amount`, `commerce`	object	**OPTIONAL**	Copied from the intent.
`metered_billing`	object	**OPTIONAL**	`settlementMode`, `quote`, and `maxBilledUnits`, copied from the intent.
`approval`	object	**OPTIONAL**	The approval the call carried.
`runtime_assurance`	object	**OPTIONAL**	Present when the kernel accepted `runtime_attestation`.
`call_chain`	object	**OPTIONAL**	The classified call chain.
`autonomy`	object	**OPTIONAL**	Copied from the intent.
`economic_authorization`	object	**OPTIONAL**	Payment and settlement terms; outside the scope of this document.

Table 23: Governed Transaction Receipt Members

The `approval` object has snake_case members. For a singular token, `token_id` is its `id`, `approver_key` its `approver`, `approval_artifact_digest` its token digest, and `approved` is true when its `decision` is `approved`. For a threshold set, `token_id` is the `proposal_id`, `approver_key` the `policy_authority`, `approval_artifact_digest` the approval set hash, and `approved` is true. The object describes what the call carried. On a receipt whose decision is `allow`, the approval passed verification; on a denial, it might not have.

The `runtime_assurance` object has the camelCase members `schema`, `tier`, `verifier`, `evidenceSha256`, and the **OPTIONAL** `verifierFamily` and `workloadIdentity`.

The `call_chain` object has camelCase members: `evidenceClass` (Section 8.6); `evidenceSources`, an array of the matched sources, omitted when empty; `upstreamProof`, the proof that passed its checks; `continuationTokenId` and `sessionAnchorId` from a continuation token; and the five call-chain members, copied from the intent unchanged.

When the class is `asserted`, the kernel also adds the member `governed_transaction_diagnostics` beside `governed_transaction`. Its member `assertedCallChain` holds a copy of the `call_chain` object. The kernel also adds the diagnostics member when a continuation token supplied a session anchor; its member `lineageReferences` then holds `sessionAnchorId`.

When the grant charges per call and the intent carries `commerce`, the kernel binds the seller, the settlement destination, the intent hash, and the approval's digest together in `economic_authorization`.

9. Receipt Checkpoints

A checkpoint signs a commitment to a local receipt batch and can commit the sequence of checkpoint batch roots. It supports audit evidence and checked continuity over supplied statements. It does not prove that a publisher disclosed every view it signed.

9.1. Checkpoint Statement

The signed object has two required members, `body` and `signature`. A verifier **MUST** reject any unknown member in either the envelope or the body. Issuance and verification use only `chio.checkpoint_statement.v1`.

Body field	Type	Presence
`schema`	String	**REQUIRED**
`checkpoint_seq`	Unsigned 64-bit integer	**REQUIRED**
`batch_start_seq`	Unsigned 64-bit integer	**REQUIRED**
`batch_end_seq`	Unsigned 64-bit integer	**REQUIRED**
`tree_size`	Positive integer	**REQUIRED**
`merkle_root`	32-octet hash	**REQUIRED**
`issued_at`	Unsigned 64-bit integer	**REQUIRED**
`kernel_key`	Public-key string (Section 4.4)	**REQUIRED**
`previous_checkpoint_sha256`	String	**OPTIONAL**
`chain_root`	32-octet hash	**OPTIONAL**

Table 24: Checkpoint Statement Body

Body field	Meaning
`schema`	Exactly `chio.checkpoint_statement.v1`.
`checkpoint_seq`	Positive checkpoint number.
`batch_start_seq`	Positive first claim-log entry in the batch.
`batch_end_seq`	Last claim-log entry in the batch, not less than its first.
`tree_size`	Number of leaves in this batch, not a cumulative log size.
`merkle_root`	Root of the receipt batch tree.
`issued_at`	Positive Unix timestamp in seconds.
`kernel_key`	Key used to verify the statement signature.
`previous_checkpoint_sha256`	SHA-256 of the preceding canonical body, exactly 64 lowercase hexadecimal characters without `0x`.
`chain_root`	Commitment to the checkpoint chain, subject to Section 9.2.

Table 25: Checkpoint Statement Body Meanings

The two **OPTIONAL** fields default to absence. A producer **MUST** omit an absent field, and a verifier **MUST** reject explicit `null` for either. The body has no `log_id`. Producers **MUST** encode `merkle_root` and `chain_root` as `0x` followed by 64 lowercase hexadecimal characters. The compatibility decoder also accepts typed hashes without the `0x` prefix and accepts hexadecimal digits in either case; typed reserialization restores the producer form. Key and signature input compatibility follows Section 4.4.

A verifier **MUST** reject an unsupported schema, a zero checkpoint number, zero batch start, inverted range, zero tree size, zero timestamp, or `tree_size` unequal to `batch_end_seq - batch_start_seq + 1`. It **MUST** reject a present predecessor digest that does not have the exact form in the table. Count arithmetic that overflows is invalid.

The signer **MUST** sign the signing wrapper (Section 4.5) of the canonical typed encoding of `body` alone, without the envelope signature, under `chio.checkpoint_statement.v1`. A verifier **MUST** reconstruct that typed body, serialize it according to Section 4.1, build its signing wrapper, and verify the signature using `kernel_key` over the wrapper. It does not require the received JSON bytes to equal the signed body bytes. The reference issuer uses Ed25519; the generic verifier dispatches on matching encoded key and signature suites. Its ordinary Ed25519 check does not imply the additional weak-key checks of a strict-verification profile.

`tree_size` is represented as a machine-sized unsigned integer in the reference implementation. Checkpoint sequence numbers used as chain-tree sizes must also fit that implementation's addressable size. These are implementation bounds; the typed integer compatibility rules in Section 4.1 also apply to checkpoint bodies.

9.2. Merkle Trees

For the local claim-log checkpointing profile, a checkpoint issuer **MUST** select a gap-free inclusive range ordered by claim-log entry sequence. For each `tool_receipt` or `child_receipt`, leaf data **MUST** be the canonical typed JSON of the full verified signed receipt, including its signature and kernel key. Other entry kinds are invalid for this batch projection. Receipt identifiers alone are not the leaf data.

For leaf data `b`, an implementation **MUST** compute its leaf hash as `SHA-256(0x00 || b)`. It **MUST** compute a parent as `SHA-256(0x01 || left || right)`, where each child is its raw 32-octet hash. For a subtree of `n > 1` leaves, the recursive split is the largest power of two strictly less than `n`, as in [RFC9162]. An unpaired rightmost node advances unchanged. A single-leaf root is its leaf hash; an empty receipt batch is invalid.

The checkpoint-chain leaf is canonical JSON of exactly `checkpoint_seq`, `batch_start_seq`, `batch_end_seq`, and `merkle_root`. Its canonical member order is `batch_end_seq`, `batch_start_seq`, `checkpoint_seq`, `merkle_root`. It excludes every other checkpoint field. Its hash is `SHA-256(0x00 || canonical_chain_leaf_bytes)`. An issuer carrying `chain_root` **MUST** compute it by combining the already hashed chain leaves for checkpoint 1 through `checkpoint_seq`, in sequence order, with the parent hash above, without hashing them as leaves again. The chain tree's size is the checkpoint count.

A checkpoint 1 producer **MUST** carry a `chain_root` equal to its own chain-leaf hash. A detached later statement can pass standalone validation without a chain root; a verifier **MUST** reject its use as either endpoint in a predecessor pair. For the local store profile, the first persisted checkpoint **MUST** have checkpoint number 1, batch start 1, and no predecessor digest. Those store-genesis rules are separate from standalone statement validation.

A local store accepting a checkpoint **MUST** verify the covered receipts, require every receipt's `kernel_key` to equal the checkpoint's `kernel_key`, and require the rebuilt batch's leaf count and root to equal the signed `tree_size` and `merkle_root`. A mixed-signer receipt batch is invalid for this store profile.

9.3. Predecessor and Consistency Proofs

A predecessor-pair verifier **MUST** validate both statements and their signatures. It **MUST** require the successor's checkpoint number to equal the predecessor's plus 1, its batch start to equal the predecessor's batch end plus 1, and its `previous_checkpoint_sha256` to equal the SHA-256 digest of the predecessor's canonical typed body, excluding signature. Overflow is invalid. Both statements **MUST** carry chain roots. Predecessor linkage alone does not prove Merkle prefix extension or require increasing timestamps.

The consistency record is unsigned. Its fields are:

Field	Type	Parsing presence/default
`schema`	String	**REQUIRED**
`log_id`	String	**REQUIRED**
`from_checkpoint_seq`	Unsigned 64-bit integer	**REQUIRED**
`to_checkpoint_seq`	Unsigned 64-bit integer	**REQUIRED**
`from_checkpoint_sha256`	String	**REQUIRED**
`to_checkpoint_sha256`	String	**REQUIRED**
`from_log_tree_size`	Unsigned 64-bit integer	**REQUIRED**
`to_log_tree_size`	Unsigned 64-bit integer	**REQUIRED**
`appended_entry_start_seq`	Unsigned 64-bit integer	**REQUIRED**
`appended_entry_end_seq`	Unsigned 64-bit integer	**REQUIRED**
`from_chain_root`	Hash	**REQUIRED**
`to_chain_root`	Hash	**REQUIRED**
`chain_proof_hashes`	Array of hashes	**REQUIRED**
`from_leaf_inclusion`	Merkle inclusion object	**REQUIRED**
`to_leaf_inclusion`	Merkle inclusion object	**REQUIRED**

Table 26

Field	Meaning
`schema`	Exactly `chio.checkpoint_consistency_proof.v1`.
`log_id`	Derived log identifier of both endpoints.
`from_checkpoint_seq`	Earlier checkpoint number.
`to_checkpoint_seq`	Immediately following checkpoint number.
`from_checkpoint_sha256`	Earlier canonical body digest, 64 lowercase hex characters without `0x`.
`to_checkpoint_sha256`	Later canonical body digest, in the same form.
`from_log_tree_size`	Earlier endpoint's `batch_end_seq`.
`to_log_tree_size`	Later endpoint's `batch_end_seq`.
`appended_entry_start_seq`	Later endpoint's `batch_start_seq`.
`appended_entry_end_seq`	Later endpoint's `batch_end_seq`.
`from_chain_root`	Earlier signed chain root.
`to_chain_root`	Later signed chain root.
`chain_proof_hashes`	RFC 9162 consistency path.
`from_leaf_inclusion`	Earlier endpoint's own chain leaf at its final position.
`to_leaf_inclusion`	Later endpoint's own chain leaf at its final position.

Table 27

All fifteen fields are required, non-null, and have no parsing defaults. A verifier **MUST** reject missing or unknown members of the outer record. Producers **MUST** include `chain_proof_hashes`, including an empty array when the path is empty. The nested inclusion objects have the format in Section 9.4 and accept unknown members in the reference typed parser. Parsing acceptance does not establish a valid proof.

The log identifier is `local-log-` followed by 64 lowercase hexadecimal characters: SHA-256 of the raw Ed25519 key bytes, or of the UTF-8 bytes of the normalized prefixed public-key encoding for another suite. This proof profile supports adjacent checkpoints under one derived log identifier. A consistency verifier **MUST** reject a pair with different derived identifiers, and **MUST** compare every metadata field against the values recomputed from the signed bodies.

For verification, both signed statements **MUST** carry chain roots. The verifier **MUST** require the record's roots to equal them and both endpoint inclusion objects to be present. For each endpoint, it **MUST** require an inclusion tree size equal to that endpoint's checkpoint number, an index equal to that number minus 1, and a path proving that endpoint's own chain-leaf hash against its signed chain root.

The verifier **MUST** establish the earlier prefix by one of these methods:

- Genesis: the earlier endpoint is checkpoint 1.
- A chain root previously accepted for the earlier endpoint: it equals the earlier signed root.
- The ordered chain-leaf hashes from checkpoint 1 through the earlier endpoint, including it: their count equals its checkpoint number and their root equals its signed chain root.

An application using a previously accepted root or prefix supplies it as verifier input; the unsigned proof record cannot establish that trust. A mid-chain pair without such input is invalid. The verifier **MUST** then verify the Merkle consistency path using the two checkpoint numbers as tree sizes and the two signed chain roots, following [RFC9162], Section 2.1.4.2. Structural input failures are errors; unsuccessful metadata, root, inclusion, anchor, or path comparisons are failed proofs.

A verifier **MUST** reject any consistency-proof schema other than `chio.checkpoint_consistency_proof.v1`.

9.4. Inclusion Proofs

The Merkle inclusion object has three required members:

Field	Type	Meaning
`tree_size`	Positive integer	Number of leaves in the tree being proved.
`leaf_index`	Non-negative integer	Zero-based leaf position.
`audit_path`	Array of 32-octet hashes	Sibling hashes in order from leaf toward root.

Table 28

`audit_path` has no parsing default and remains present when empty. The compatibility decoder accepts unknown members. Counts and indices use machine-sized unsigned integers in the reference implementation. Hash serialization and compatibility parsing follow Section 9.1.

An inclusion verifier **MUST** reject a zero tree size or an index not less than that size. It **MUST** compute the leaf hash from the specified bytes and combine the supplied siblings using Section 9.2. A carried rightmost node consumes no sibling at that level. It **MUST** reject a path with missing or unused hashes and accept membership only when the computed root equals the expected root.

The evidence-export wrapper is unsigned and has no schema identifier. It has these five required members, none with a parsing default:

Field	Type	Meaning
`checkpoint_seq`	Unsigned 64-bit integer	Referenced checkpoint number.
`receipt_seq`	Unsigned 64-bit integer	Receipt's claim-log entry sequence claimed by the wrapper.
`leaf_index`	Non-negative integer	Receipt's claimed position in this batch.
`merkle_root`	32-octet hash	Advertised batch root.
`proof`	Merkle inclusion object	Nested path and tree parameters.

Table 29

Its compatibility decoder accepts unknown members. Basic wrapper verification **MUST** require matching wrapper and nested indices and verify the canonical receipt bytes against the expected root supplied by the caller. It does not independently authenticate the wrapper's sequence metadata or advertised root.

An evidence-package verifier **MUST** resolve the referenced checkpoint and receipt, compare the wrapper root to the signed checkpoint root, require the wrapper index to be below the checkpoint's batch size, and require `receipt_seq` to fall within its inclusive batch range. It **MUST** reject two proofs for the same receipt sequence, verify the canonical receipt bytes against the signed root, and require the number of exported tool receipts without successful proofs to equal the manifest's uncheckpointed count.

That package profile does not independently compare the nested tree size with the signed batch size, or the index with the claimed sequence offset. A consumer claiming positional membership **MUST** additionally require `proof.tree_size == checkpoint.body.tree_size` and `leaf_index == receipt_seq - checkpoint.body.batch_start_seq`, after checking the sequence range. These are additional positional-profile checks; they are not implicit in basic wrapper or generic package verification.

The current export emits tool-receipt proofs. Child receipts can be batch leaves, while child-receipt inclusion-proof export remains outside this export profile. Uncovered tool receipts are reported explicitly; absence of a proof establishes no membership.

9.5. Checkpoint Verification

A consumer that treats a checkpoint as evidence from an expected kernel **MUST** check its `kernel_key` against a separately configured trusted-key policy. Signature validity under a key carried in the statement does not establish that key's authority. This is a consumer-policy check; the reference standalone and checkpoint-set APIs do not take a trusted-key input. A stricter consumer profile can also require strict signature verification and signer validity intervals.

A checkpoint-set verifier for this profile **MUST** validate every statement and signature, reject duplicate checkpoint numbers and observed conflicting statements, and require each predecessor digest to resolve within the supplied set and pass predecessor validation. A checkpoint without a predecessor **MUST** be checkpoint 1 with batch start 1. A cited predecessor absent from the set is invalid. The reference set verifier requires the complete prefix through checkpoint 1 and has no separately pinned-boundary input; the pair-level anchor interface in Section 9.3 is a distinct verification path.

Scoped evidence exports retain the prefix through the newest checkpoint covering the selected receipts. Inclusion and consistency verification uses the roots in the validated signed bodies, rather than relying on an unsigned advertised root. Consumer signer policy determines which of those signed bodies is accepted as evidence from the intended kernel.

9.6. Claim Limits

The checkpoint surface provides local signed audit evidence and verified continuity for the supplied, anchored views. It does not establish public append-only publication, complete receipt-family sequencing, or strong non-repudiation. A verifier can detect conflicting statements that it receives; it cannot detect a hidden competing view from these bytes alone. External witnessing and publication policies need separate protocols and qualification and are outside this document.

10. Native Transport

The native transport carries messages between an agent and a kernel over a reliable, ordered, bidirectional byte stream, such as a pair of pipes, a TCP connection, or a Unix domain socket. The agent sends agent messages, the kernel sends kernel messages, and each message travels in one frame.

A native session lasts as long as the byte stream. The transport has no initialization exchange and no in-band version negotiation: both parties use version `chio-wire-v1`, agreed out of band (Section 13). The kernel binds each session to one agent key, which it establishes when the session opens by means outside this document. It denies a call whose capability token has a different subject (Section 5.8).

The native transport provides no confidentiality, integrity protection, or peer authentication of its own. It relies on the underlying byte stream for them (Section 15).

10.1. Native Framing

Each frame is a length followed by a payload (Figure 4).

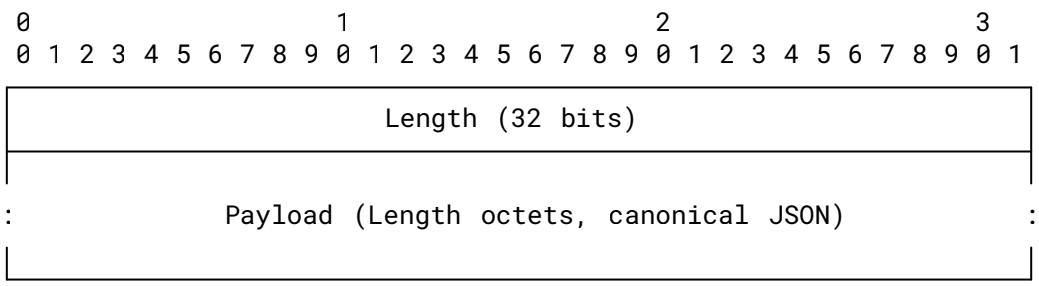


Figure 4: Native Frame

Length: An unsigned 32-bit integer in network byte order that gives the number of octets in Payload. It does not count its own four octets.

Payload: One agent message or kernel message (Section 10.2), encoded as canonical JSON (Section 4.1).

The maximum payload length is 16,777,216 octets. A sender **MUST NOT** send a frame with a longer payload, and a receiver **MUST** reject a frame whose Length exceeds 16,777,216 (Section 10.5). A payload of exactly 16,777,216 octets is permitted.

A sender **MUST** encode each message as canonical JSON and **MUST** put exactly one message in each frame. A sender that fails to encode a message sends no part of its frame. A receiver parses the payload as JSON and does not check that it is canonical. A zero-length payload is valid framing but never a valid message.

For example, the message `{"type":"heartbeat"}` is 20 octets long, so its frame is the octets 0x00 0x00 0x00 0x14 followed by those 20 octets.

10.2. Native Messages

Every message is a JSON object whose `type` member names its kind (Table 30). A sender **MUST NOT** include members that this document does not define for the message. The kernel ignores such members when it receives them.

Sender	type	Other members	Purpose
agent	`tool_call_request`	Table 31	Request one tool call
agent	`list_capabilities`	none	Request the session's tokens
agent	`heartbeat`	none	Liveness probe
kernel	`tool_call_chunk`	Table 35	One piece of streamed output
kernel	`tool_call_response`	Table 33	Terminal outcome of a call
kernel	`capability_list`	`capabilities`	The session's tokens
kernel	`capability_revoked`	`id`	Notice of a revoked capability
kernel	`heartbeat`	none	Reply to `heartbeat`

Table 30: Native Message Types

A `tool_call_request` has the members in Table 31. A sender omits an **OPTIONAL** member that has no value and omits `approval_tokens` when it is empty.

Field	Type	Presence
`type`	string	**REQUIRED**
`id`	string	**REQUIRED**
`capability_token`	object	**REQUIRED**
`server_id`	string	**REQUIRED**
`tool`	string	**REQUIRED**
`params`	JSON value	**REQUIRED**
`governed_intent`	object	**OPTIONAL**
`approval_token`	object	**OPTIONAL**
`approval_tokens`	array of object	CONDITIONAL
`threshold_approval_proposal`	object	CONDITIONAL
`supplemental_authorization`	object	**OPTIONAL**
`execution_nonce`	object	**OPTIONAL**

Table 31: tool_call_request Members

Field	Meaning
`type`	`tool_call_request`
`id`	Request identifier chosen by the agent
`capability_token`	Capability token (Section 5.1) that authorizes the call
`server_id`	Identifier of the target tool server
`tool`	Name of the tool on that server
`params`	Arguments for the tool
`governed_intent`	Governed transaction intent (Section 8.2)
`approval_token`	One approval token (Section 8.4)
`approval_tokens`	1 to 32 approval tokens; present exactly when `threshold_approval_proposal` is (Section 8.5)
`threshold_approval_proposal`	Present exactly when `approval_tokens` is (Section 8.5)
`supplemental_authorization`	Opaque authorization extension (Section 7)
`execution_nonce`	Execution nonce that the kernel returned for this `id` (Section 7.7)

Table 32: tool_call_request Members Meanings

The agent **MUST NOT** reuse an `id` within a session, except to retry a call with the execution nonce that the kernel returned for that `id`. The kernel answers a reused `id` with `internal_error` (Section 10.4).

The kernel denies a request that carries both `approval_token` and a non-empty `approval_tokens`, a `threshold_approval_proposal` without `approval_tokens`, or `approval_tokens` without a `threshold_approval_proposal`. It reports `policy_denied` with `guard` set to `session_authorization`.

No member of `tool_call_request` carries a sender proof. A kernel therefore denies every native call for which a matching grant requires one (Section 5.4). No member carries model metadata either, so a model constraint that requires metadata (Section 5.3) is never satisfied on this transport.

The following request is shown with whitespace added and with its capability token abbreviated to the `id` member. Appendix A gives a complete frame.

```
{
  "capability_token": {"id": "cap-1"},
  "id": "req-001",
  "params": {"text": "hello"},
  "server_id": "srv",
  "tool": "echo",
  "type": "tool_call_request"
}
```

The kernel answers each `list_capabilities` with one `capability_list`, whose `capabilities` member is an array of the capability tokens that the kernel issued for the session. The kernel does not remove expired or revoked tokens from the list, and it sends an empty array if it cannot produce the list. A kernel **MAY** also send `capability_list` unprompted, for example as its first message, to deliver the tokens it issued when the session opened.

The kernel answers each `heartbeat` with one `heartbeat`. It sends no `heartbeat` unprompted, and this document defines no heartbeat interval or timeout.

A kernel **MAY** send `capability_revoked`, whose `id` member is the identifier of a revoked capability. A kernel does not otherwise notify the agent of a revocation: the agent learns of it when a call that depends on the revoked capability is denied (Section 12.4).

10.3. Tool Call Results

For each `tool_call_request`, the kernel sends zero or more `tool_call_chunk` messages and then one `tool_call_response`. Each carries the request's `id`, and the agent correlates them by that value. Table 33 lists the members of `tool_call_response`.

Field	Type	Presence	Meaning
`type`	string	**REQUIRED**	`tool_call_response`
`id`	string	**REQUIRED**	The `id` of the request
`result`	object	**REQUIRED**	Outcome of the call (Table 34)
`receipt`	object	**REQUIRED**	Receipt for the call (Section 6.1)
`execution_nonce`	object	**OPTIONAL**	Nonce to present when retrying this `id` (Section 7.7); omitted when absent

Table 33: tool_call_response Members

The `result` member is an object whose `status` member selects its other members (Table 34). The "Decision" column gives the decision that the receipt records (Section 6.3).

status	Other members	Decision	Meaning
`ok`	`value` (JSON value)	`allow`	The call completed. `value` is the output, or `null` if there is none.
`stream_complete`	`total_chunks` (integer)	`allow`	The call completed after streaming `total_chunks` chunks.
`cancelled`	`reason` (string), `chunks_received` (integer)	`cancelled`	The call was canceled.
`incomplete`	`reason` (string), `chunks_received` (integer)	`incomplete`	The call ended before completion.
`err`	`error` (object, Section 10.4)	`deny`	The kernel denied the call or failed to evaluate it.

Table 34: Tool Call Results

In `cancelled` and `incomplete` results, `chunks_received` is the number of chunks that the kernel sent for the call.

A `tool_call_chunk` carries one piece of streamed output (Table 35).

Field	Type	Presence	Meaning
`type`	string	**REQUIRED**	`tool_call_chunk`
`id`	string	**REQUIRED**	The `id` of the request
`chunk_index`	integer	**REQUIRED**	Position of the chunk, starting at 0
`data`	JSON value	**REQUIRED**	Content of the chunk

Table 35: tool_call_chunk Members

The kernel sends the chunks of a call in order, with `chunk_index` values 0, 1, 2, and so on without gaps, and it sends all of them before the call's `tool_call_response`. A kernel can collect the whole stream before it sends the first chunk, so an agent cannot assume that chunks arrive while the tool runs. Each chunk travels in its own frame, and the frame limit (Section 10.1) applies to each one. A kernel **MAY** bound the duration, total size, or chunk count of a stream. When it stops a stream at such a bound, it sends the chunks it kept and ends the call as `incomplete`.

Every `tool_call_response` carries a receipt, including a response that reports a denial, a cancellation, an incomplete call, or an evaluation error. Normal evaluation signs with the kernel's configured receipt key (Section 6.6). The reference native error handler can instead sign a diagnostic receipt with a fresh key after an evaluation error; that key is not authenticated to the session kernel. If even that signing fails, no response is sent. A consumer **MUST NOT** treat a diagnostic receipt under an untrusted key as evidence of its expected kernel. An agent **SHOULD** verify a receipt (Section 6.9) before it relies on it.

10.4. Native Errors

An `err` result carries an error object in its `error` member. The object's `code` member names the error, and for some codes a `detail` member adds information (Table 36). A code without detail is serialized with no `detail` member. The native transport carries no numeric error code; Section 14.2 maps each `code` to the registry code shown.

code	detail	Meaning	Registry
`capability_denied`	string	The capability token is invalid for the call, for example because its signature does not verify	2100
`capability_expired`	none	The token is outside its validity interval	2101
`capability_revoked`	none	The token, or a capability in its delegation chain, is revoked	2102
`policy_denied`	object	A check of the kernel or a guard denied the call	3100
`tool_server_error`	string	The tool server failed	5100
`internal_error`	string	The kernel failed to evaluate the call	6100

Table 36: Native Error Codes

The `detail` of `policy_denied` has two string members: `guard`, which names the check that denied the call, and `reason`, a description for people. The kernel uses these `guard` values:

`kernel`: The kernel's own checks or policy denied the call.

`approval`: The call needs an approval that the request did not carry (Section 8.4).

`session_authorization`: The request's approval members conflict (Section 10.2).

The codes `capability_denied`, `capability_expired`, `capability_revoked`, and `tool_server_error` identify specific failures, and a kernel **SHOULD** use them for those failures. A kernel **MAY** instead report any denial as `policy_denied` with `guard` set to `kernel` and a `reason` that names the failure, such as "capability has expired", and any evaluation failure as `internal_error`. An agent can therefore receive either form for the same condition.

The following `err` result reports an expired token in the second form.

```
{
  "error": {
    "code": "policy_denied",
    "detail": {
      "guard": "kernel",
      "reason": "capability has expired"
    }
  },
  "status": "err"
}
```

10.5. Receiver Behavior

A receiver reads a frame in two steps: it reads the four octets of Length, then Length octets of payload. It handles failures as follows:

- If the stream ends before all four octets of Length arrive, the connection is closed and no message is delivered.
- If the stream ends after Length but before the full payload arrives, the connection is closed and the partial payload is discarded.
- If Length exceeds 16,777,216, the receiver **MUST** reject the frame without reading or allocating its payload.
- If the payload is not exactly one JSON object that forms a valid message of the kind the receiver expects (for example, it is empty, it is not JSON, a **REQUIRED** member is missing, a member has the wrong type, or `type` is unknown), the receiver **MUST** reject the frame.

The transport defines no marker from which a receiver could find the next frame boundary, and a receiver **MUST NOT** attempt to resynchronize. After any of these failures, and after a failure to send, the kernel ends the session: it reads no further frames, sends no further messages, and closes the session. It sends the peer no error message first. Neither party resumes a session. A peer that wants to continue opens a new transport, which begins a new session.

Signed objects travel inside messages, including capability tokens (in `tool_call_request` and `capability_list`) and receipts (in `tool_call_response`), approval artifacts, and execution nonces. Each artifact defines its own authenticated projection and signing input (Section 4.6). A party that forwards an artifact **MUST** preserve that projection and its signature. It **MAY** re-encode the same typed values when this produces the same signing input. Unknown JSON members are not thereby authenticated.

11. Hosted MCP Binding

A hosted edge is an MCP server [MCP] whose tool calls a kernel mediates. It uses the MCP Streamable HTTP transport: the client sends each JSON-RPC 2.0 message [JSON-RPC] in an HTTP POST [RFC9110], and the edge answers requests with server-sent events [SSE]. The edge supports MCP version 2025-11-25, which it selects as Section 13 specifies.

A client never handles capability tokens. For each MCP session the edge opens a kernel session, creates an agent key, and obtains capability tokens for that key under its issuance policy. For each tool call it selects one of those tokens (Section 11.6).

11.1. Endpoints and Headers

The edge serves one endpoint path, `/mcp`, with the methods in Table 37.

Method	Use	Success response
POST	Send one JSON-RPC request, notification, or response	200 with an event stream for a request; 202, or 200 with an event stream, otherwise
GET	Open the notification stream (Section 11.5)	200 with an event stream
DELETE	End the session	204

Table 37: Hosted Endpoint Methods

Table 38 lists the request headers that the edge examines.

Header	Sent with	Rule
`Authorization`	every request	A `Bearer` credential (Section 11.2)
`Origin`	any request	If present, its host is `localhost`, `127.0.0.1`, or `::1`
`Accept`	POST, GET	POST lists `application/json` and `text/event-stream`; GET lists `text/event-stream`
`Content-Type`	POST	Begins with `application/json`
`MCP-Session-Id`	every request after `initialize`	The session identifier
`MCP-Protocol-Version`	any request in a session	If present, equals the session's version
`Last-Event-ID`	GET	Replay cursor (Section 11.5)
`DPoP`	any request	Refused with 401 (Section 11.2)

Table 38: Hosted Request Headers

The edge limits the request rate of each client address and answers a request over the limit with 429 and a `Retry-After` header. It then processes a POST in this order, stopping at the first failure:

1. An `Origin` header whose host is not a loopback name or address: 403.

2. Admission fails (Section 11.2): 401.
3. `Accept` lacks either media type: 406. `Content-Type` does not begin with `application/json`: 415.
4. `Content-Length` is not an integer: 400, with JSON-RPC code -32600. The body exceeds 8,388,608 octets: 413, with JSON-RPC code -32600. The body is not JSON, or reading it fails: 400, with JSON-RPC code -32700.
5. If `method` is `initialize`, the edge continues as Section 11.3 describes.
6. `MCP-Session-Id` is missing, empty, or has leading or trailing whitespace: 400, with JSON-RPC code -32600. The session is unknown: 404.
7. For a session in a terminal state (Section 11.4), the edge checks credential continuity (403 on failure) and then answers 410.
8. Otherwise, a `MCP-Protocol-Version` header that differs from the session's version: 400. Credential continuity fails: 403. The session is not `ready`: 409.

Where a step gives a JSON-RPC code, the response body is a JSON object with members `jsonrpc` and `error`, and no `id` member. Other error responses carry a plain-text body.

A POST that carries a request, meaning a message with both `id` and `method`, receives an event stream. The stream starts with a priming event that has an event identifier, a `retry` field of 1000 milliseconds, and empty data. It ends after the event that carries the response to the request. The edge handles the requests of a session one at a time: a request POST waits until the stream of the previous request has ended. A POST that carries a notification or a response receives any resulting output of the session as an event stream, or 202 if there is none. The edge answers 202 at once if a request stream of the session is open.

A DELETE is checked for Origin and admission as a POST is. It then needs an `MCP-Session-Id` of a known session (400 or 404 otherwise, with plain-text bodies) and credential continuity (403). The edge answers 410 for a session already in a terminal state. Otherwise it moves the session to `deleted` and answers 204.

11.2. Session Admission

Every request, `initialize` included, **MUST** carry an `Authorization` header with the `Bearer` scheme and a non-empty token. The edge answers a request without one, or with a token that it does not accept, with 401 and a `WWW-Authenticate` challenge. If the edge publishes OAuth protected resource metadata [RFC9728], the challenge is `Bearer` with a `resource_metadata` parameter that gives the metadata URL, and a `scope` parameter when scopes are configured. Otherwise the challenge is `Bearer` alone.

The edge accepts tokens in one configured mode:

- a static token, which has to equal the configured value;

- a JWT [RFC7519], signed with EdDSA, RS256, RS384, RS512, PS256, PS384, PS512, ES256, or ES384 under a configured key or key set. The edge rejects the token before its `nbf` time or at or after its `exp` time when those claims are present. When the corresponding expected issuer, audience, or protected resource is configured, it checks `iss`, the audience or `resource` claim; it also checks configured scopes;
- a token that the edge validates by OAuth token introspection [RFC7662].

The edge has no unauthenticated mode.

A JWT or introspected token can bind the request to a sender through members of its `cnf` claim. For `x5t#S256` and `chioAttestationSha256`, the request **MUST** carry the header `x-chio-mtls-thumbprint-sha256` or `x-chio-runtime-attestation-sha256`, respectively, with a value equal to the claim. A failed check is 401. This edge refuses Chio DPoP sender binding: it **MUST** reject with 401 a token whose `cnf` carries `chioSenderKey` and any request that carries a `DPoP` header, and **MUST NOT** admit either as a bearer.

The edge binds the authentication context of `initialize` to the session. Every later request of the session **MUST** present the same context: the same `Origin` value or none, and either the same static token or, for a JWT or introspected token, the same principal, issuer, subject, audience, scopes, and identity claims. When the edge recorded a fingerprint of the token, the request **MUST** carry the same token. The edge answers a mismatch with 403.

11.3. Initialization

A client opens a session as follows:

1. The client sends an `initialize` request in a POST without an `MCP-Session-Id` header. The edge answers 400, with JSON-RPC code -32600, if the message has no `id` or if the POST carries an `MCP-Session-Id` header with any value. It creates no session in either case.
2. The edge creates a session in state `initializing`, with a new session identifier, a new kernel session, an agent key, and capability tokens for that key. It generates the session identifier from fresh randomness; clients treat it as opaque.
3. The edge answers with an event stream that holds a priming event and then the `initialize` response. If the response is a result, the edge sets `MCP-Session-Id` on the HTTP response, records `result.protocolVersion` as the session's version, and moves the session to `ready`. If the response is an error, for example for an unsupported version (Section 13), the HTTP response has no `MCP-Session-Id` and the edge discards the session.
4. The client sends `notifications/initialized` in a POST with the session identifier. The edge then activates the kernel session. Until it does, the edge answers `tools/list`, `tools/call`, and the other operations of a session with JSON-RPC error -32002. It answers `ping` at any time.

Every `initialize` creates a new session, and the edge never re-initializes an existing one. A client that sets `toolCallChunkNotifications` within `capabilities.experimental.chioToolStreaming` to `true` in `initialize` receives streamed

tool output as notifications (Section 11.6). The result's `capabilities.experimental` member advertises that feature as `chioToolStreaming` and describes version selection as `chioProtocol` (Section 13).

11.4. Session States

Figure 5 shows the states of a hosted session.

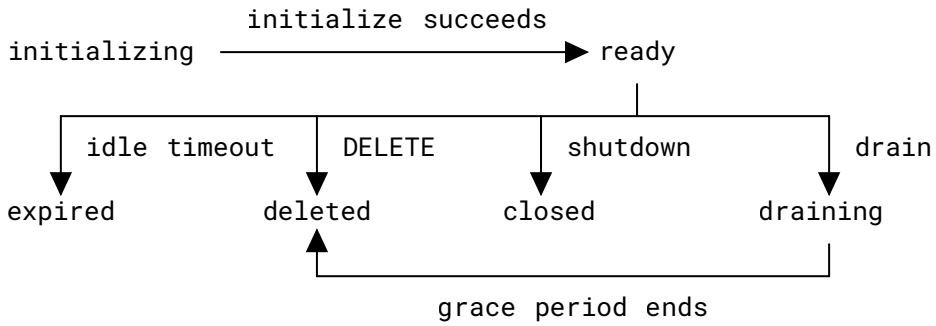


Figure 5: Hosted Session States

A `ready` session becomes `expired` when no request has been accepted for the idle period, which is 15 minutes by default. Each POST or GET that the edge accepts restarts the idle period; an open stream does not. An operator can drain a `ready` session, which then becomes `deleted` when a grace period ends, 5 seconds by default. A DELETE moves any session that is not in a terminal state to `deleted`, and an operator shutdown moves it to `closed`.

The states `deleted`, `expired`, and `closed` are terminal, and a terminal session never changes state. The edge answers a request to an `initializing` or `draining` session with 409 and a request to a terminal session with 410. It keeps a record of a terminal session for a retention period, 30 minutes by default, and then answers 404.

The edge does not resume a terminal session. A client continues after 410 or 404 only by opening a new session. While a session is `ready`, a client that loses a connection can keep sending requests with the same session identifier and credentials. A response lost with its stream cannot be recovered; only notifications can be replayed (Section 11.5).

11.5. Notification Stream and Replay

A client opens the notification stream of a session with a GET. The GET needs an `MCP-Session-Id` header (400, with JSON-RPC code -32600, otherwise) and an `Accept` header that lists `text/event-stream` (406 otherwise). The edge applies the Origin, admission, version, continuity, and state checks of a POST. A session has at most one notification stream: the edge answers a second GET with 409 while the first stream is open. The stream carries only notifications, meaning messages

with `method` and without `id`. It has no priming event and ends when its output channel closes or the client disconnects. A logical session-state transition does not guarantee immediate closure of an already open stream.

While a notification stream is open, the edge sends notifications only on it. While none is open, it sends them on the stream of the current request POST. A request stream also carries the edge's requests to the client, which have both `method` and `id`.

Each event has an identifier of the form `<session-id>-<n>`, where `<n>` is a decimal counter. The counter starts at 1 and counts every event of the session on every stream, priming events included. The counter is allocated when the edge creates an event, so its value does not establish delivery order across streams or buffered events. For a non-priming event, `data` is one JSON-RPC message; a priming event has empty data. The edge sets no `event` field. The edge retains the last 64 notifications for replay and retains no other events. Delivery on an open stream is best effort: the edge drops events, without notice, for a reader that falls too far behind.

A GET with a `Last-Event-ID` header asks the edge to replay the notifications that followed that event. The edge splits the value at its last hyphen. The part before it has to equal the session identifier, and the part after it, the cursor, has to be an unsigned decimal integer. The edge answers 409 and opens no stream if the value is malformed or names another session, if it retains no notifications, or if the cursor is below one less than the counter of the oldest retained notification or above that of the newest. Otherwise it sends each retained notification with a larger counter and then continues with live notifications, without duplicates. After a 409, a client can open a stream without `Last-Event-ID` and accept that it missed notifications.

An edge can serve several sessions through one connection to an upstream tool server. In that configuration it delivers every notification from the upstream server into every live session's queue. Session workers filter resource updates by subscription and elicitation completion by session context, and ignore unsupported notifications.

11.6. Tool Calls

A `tools/call` request names the tool in `params.name` and passes `params.arguments`, which defaults to an empty object. The edge answers JSON-RPC error -32602 if `name` is absent or names a tool it does not expose. Its `tools/list` lists only the tools that the session's capability tokens authorize.

The client sends no capability token. The edge selects the first of the session's tokens whose grants match the tool's server and name, the arguments, and the model metadata (Section 11.7). If no token matches, the edge returns a result with `isError` set to `true` and does not submit the call to the kernel, so no receipt records the call. Otherwise the edge submits the call to the kernel with the selected token, and the kernel evaluates it as it evaluates a native call (Section 5.8). The submission carries no sender proof: the kernel denies a call for which a matching grant requires one (Section 5.4).

The members of `params._meta` in Table 39 carry the other inputs of a native request (Table 31). `_meta` has to be an object, and each member has to have the structure of its native equivalent; the edge answers -32602 otherwise. The edge also answers -32602 to a member name without its `chio` prefix (for example `approvalToken`) and to `route_selection`.

Member	Native equivalent
`chioRequestId`	`id`
`chioModelMetadata`	none (Section 11.7)
`chioExecutionNonce`	`execution_nonce`
`chioGovernedIntent`	`governed_intent`
`chioApprovalToken`	`approval_token`
`chioApprovalTokens`	`approval_tokens`
`chioThresholdApprovalProposal`	`threshold_approval_proposal`
`chioSupplementalAuthorization`	`supplemental_authorization`
`chioRouteSelection`	none

Table 39: Chio Members of _meta in tools/call

`chioRequestId` is a non-empty string of at most 2048 UTF-8 octets, with no leading or trailing whitespace and no control characters. The kernel uses it as the request identifier. Without it, the edge uses the request identifier bound in a presented execution nonce, or assigns a new identifier to each call. A request that carries approval members or `chioSupplementalAuthorization` **MUST** carry `chioRequestId`, and the approval members have to follow the combination rules of Section 10.2; the edge answers -32602 otherwise. The kernel records `chioRouteSelection`, an object, in the receipt's metadata as `route_selection`.

The following request is illustrative.

```
{
  "jsonrpc": "2.0",
  "id": 7,
  "method": "tools/call",
  "params": {
    "name": "echo",
    "arguments": {"text": "hi"},
    "_meta": {
      "chioRequestId": "req-7",
      "chioModelMetadata": {
        "model_id": "m-1",
        "safety_tier": "standard",
        "provider": "p"
      }
    }
  }
}
```

The edge returns the kernel's outcome as an MCP tool result:

- For an allowed call, `isError` defaults to `false`. A tool output already shaped as an MCP result can retain an existing `isError: true`. An object value becomes `structuredContent` plus a text rendering in `content`. A string value becomes one text item. A value that already has the members of a tool result keeps its existing fields. Missing `isError` defaults to `false`; missing `content` is synthesized only when `structuredContent` is present.
- For a denied, canceled, or incomplete call, and for a failure of the kernel to evaluate the call, `isError` is `true`. An ordinary error has one text item that states the reason. A retained streamed error instead describes the stream delivery in `content` and records the reason in `structuredContent.chioToolStream.reason`.
- For a call that needs a URL elicitation, the edge returns JSON-RPC error -32042 with `data.elicitations`.

The result does not contain the receipt. Receipt-bearing evaluation paths sign and retain receipts as in the native integration. Kernel evaluation errors and duplicate or stale request rejection can produce an MCP error without a receipt. An edge configured with a trust-control service appends each receipt to it, where the operator can query it (Section 12.3). When the kernel issues an execution nonce, the edge returns it in `result._meta.chioExecutionNonce`.

If the client enabled chunk notifications at initialization, the edge sends one `notifications/chio/tool_call_chunk` notification per chunk before the result. Its `params` members are `requestId` (the JSON-RPC `id` of the call), `chunkIndex`, `totalChunks`, and `chunk`. The result's `structuredContent.chioToolStream` then has `mode` set to `notification_stream`. Otherwise a retained stream's `chioToolStream` has `mode` set to `collapsed_result` and carries every chunk in `chunks`. In both cases `chioToolStream` also carries `totalChunks` and `terminalState`, which is `completed` or `incomplete` in retained streamed results. Cancellation is an exception: the edge returns a reason-only error result and omits the stream summary and collapsed chunks. Chunk notifications can already have been queued before that cancellation result.

11.7. Model Metadata

A client can describe the model on whose behalf it calls a tool, in the `chioModelMetadata` member of `_meta` (Table 40).

Field	Type	Presence	Meaning
`model_id`	string	**REQUIRED**	Model identifier
`safety_tier`	string	**OPTIONAL**	`low`, `standard`, `high`, or `restricted`
`provider`	string	**OPTIONAL**	Provider name
`provenance_class`	string	**OPTIONAL**	Replaced by the edge

Table 40: Model Metadata

The edge has not verified this metadata, so it **MUST** set `provenance_class` to `asserted`, whatever value the client sent. The other provenance classes, `observed` and `verified`, are not available to a client. The kernel evaluates model constraints (Section 5.3) against the metadata; those constraints do not consider the provenance class. The kernel records the metadata in the receipt's `metadata.model_metadata` member, where `provenance_class` is always present. For the request above, the receipt records:

```
{
  "model_metadata": {
    "model_id": "m-1",
    "provenance_class": "asserted",
    "provider": "p",
    "safety_tier": "standard"
  }
}
```

12. Trust-Control Interface

The trust-control service offers an HTTP interface [RFC9110] whose paths begin with `/v1` (Section 13). Request and response bodies are JSON objects with camelCase member names. Nested capability tokens, receipts, and attestation statements keep their own member names. This section specifies four endpoints: issuance, delegated issuance, receipt query, and revocation.

A caller authenticates with an `Authorization` header that carries the `Bearer` scheme and the service token. The service compares the token with its configured value in constant time and answers a missing or wrong token with 401 and `WWW-Authenticate: Bearer`. The receipt query also accepts tenant read tokens (Section 12.3). The nodes of a clustered service can authenticate issuance requests to each other by a mechanism outside the scope of this document; a clustered service also adds members that describe the node that handled a request.

An error response has the body `{"error": "<message>"}`, and its status code gives the class of error. The service rejects a body or query string that does not parse into the structure defined here with a 4xx status, which can precede authentication and can carry a different body. It answers 503 when it sheds load.

12.1. Capability Issuance

`POST /v1/capabilities/issue` issues a direct capability token, one with an empty delegation chain. Table 41 lists the request members. A 200 response has one member, `capability`, which holds the token (Section 5.1).

Field	Type	Presence	Meaning
`subjectPublicKey`	string	**REQUIRED**	Subject key (Section 4.4)
`scope`	object	**REQUIRED**	Requested scope (Section 5.2)
`ttlSeconds`	integer	**REQUIRED**	Requested lifetime in seconds
`runtimeAttestation`	object	**OPTIONAL**	Runtime attestation statement; omitted when absent

Table 41: Issuance Request Members

The service signs the token with its authority key. The token's `subject` is `subjectPublicKey`, its `issued_at` is the current time, and its `expires_at` is `issued_at` plus `ttlSeconds`, saturating at the unsigned 64-bit maximum (2^64 - 1). Its scope is the requested scope, to whose grants the service's issuance policy can add constraints. The service answers:

- 400 if `subjectPublicKey` is not a valid key, or if `runtimeAttestation` has a malformed `workload_identity` or one that conflicts with a SPIFFE URI in `runtime_identity`;
- 403 if its issuance policy denies the request, including when it rejects the runtime attestation;
- 409 if the authority backend is not configured or its configuration conflicts with the issuance mode;
- 500 for any other failure.

A runtime attestation statement is a normalized result from an attestation verifier, with members `schema`, `verifier`, `tier` (`none`, `basic`, `attested`, or `verified`), `issued_at`, `expires_at`, `evidence_sha256`, and the **OPTIONAL** `runtime_identity`, `workload_identity`, and `claims`. The service rejects a statement outside its validity interval (`issued_at` <= now < `expires_at`). Configured trust and assurance policies determine acceptance and effective tier; without trust rules, untrusted evidence can be downgraded to `none`. That effective tier selects any configured scope and lifetime ceilings. Attestation evidence formats and their appraisal are outside the scope of this document.

12.2. Delegated Issuance

`POST /v1/federation/capabilities/issue` issues a capability token to the subject of an Agent Passport presentation, optionally within a delegation policy signed by another party. Passports and presentations are outside the scope of this document; this section specifies the delegation policy and the checks that bound the issued token. Only the service token authenticates this endpoint.

Field	Type	Presence	Meaning
`presentation`	object	**REQUIRED**	Passport presentation; its format is outside this document
`expectedChallenge`	object	**REQUIRED**	Challenge that the presentation answers; its format is outside this document, except its `verifier` member, a string that names the verifying service
`capability`	object	**REQUIRED**	Requested token: `scope` (object, Section 5.2) and `ttl` (integer, seconds)
`admissionPolicy`	object	**OPTIONAL**	Admission policy for enterprise identities; outside this document
`enterpriseIdentity`	object	**OPTIONAL**	Enterprise identity of the subject; outside this document
`delegationPolicy`	object	**OPTIONAL**	Signed delegation policy (Table 43)
`upstreamCapabilityId`	string	**OPTIONAL**	Parent capability that the delegation policy names

Table 42: Delegated Issuance Request Members

A `delegationPolicy` has two members: `body` (Table 43) and `signature`, the signature of `signerPublicKey` over the canonical encoding of `body` (Section 4.6).

Field	Type	Presence	Meaning
`schema`	string	**REQUIRED**	`chio.federated-delegation-policy.v1`
`issuer`	string	**REQUIRED**	Party that grants the delegation
`partner`	string	**REQUIRED**	Party that receives it
`verifier`	string	**REQUIRED**	Service that may issue under the policy
`signerPublicKey`	string	**REQUIRED**	Key that signs the policy (Section 4.4)
`createdAt`	integer	**REQUIRED**	Start of validity, in Unix seconds
`expiresAt`	integer	**REQUIRED**	End of validity, in Unix seconds
`ttlSeconds`	integer	**REQUIRED**	Largest lifetime of an issued token
`scope`	object	**REQUIRED**	Largest scope of an issued token
`purpose`	string	**OPTIONAL**	Stated purpose
`parentCapabilityId`	string	**OPTIONAL**	Parent capability to which the policy is bound

Table 43: Delegation Policy Body Members

The service **MUST** perform these checks, in this order, and reject the request at the first that fails:

1. If the service has a configured public URL, `expectedChallenge`'s `verifier` equals it (400).
2. If `delegationPolicy` is present: its `schema` is the value above, `createdAt` <= `expiresAt`, `ttlSeconds` is greater than 0, the signature verifies, `createdAt` <= now <= `expiresAt`, and its `verifier` equals the challenge's `verifier` and the configured public URL, if any (400 for each).
3. If `delegationPolicy` is present, the requested token is within its ceiling: the requested scope is a subset of the policy's `scope` (Section 5.6), `ttl` <= `ttlSeconds`, and now plus `ttl`, saturating at the unsigned 64-bit maximum, is at most `expiresAt` (403).
4. `upstreamCapabilityId`, if present, equals the policy's `parentCapabilityId`, and a policy `parentCapabilityId` is present only with `upstreamCapabilityId` (400).
5. The presentation verifies against the challenge and the service's verifier policy, by rules outside this document (400, 403, or 409).
6. If `upstreamCapabilityId` is present, the service holds a record of that capability imported from the operator that issued it (404), the policy's `signerPublicKey` is the key that shared the record, and the requested scope and expiry fit within the parent's (403). Importing such records is outside the scope of this document.
7. If `delegationPolicy` is present, `signerPublicKey` is one of the keys that the service's capability authority trusts (403).

The service then issues a direct token from its authority key to the presentation's subject, with the requested `scope` and `ttl`, under its issuance policy (403 if that policy denies it). With a delegation policy, the service records the policy as the lineage parent of the new token (409 if it has no receipt store) and returns the identifier of that record. The delegation appears in the service's records, not in the token's delegation chain (Section 5.5).

A 200 response has the members in Table 44.

Field	Type	Presence	Meaning
`subject`	string	**REQUIRED**	The presentation's subject as a `did:chio` identifier (Section 4.7)
`subjectPublicKey`	string	**REQUIRED**	The subject key
`verification`	object	**REQUIRED**	Result of verifying the presentation; outside this document
`capability`	object	**REQUIRED**	The issued token
`enterpriseIdentityProvenance`	object	**OPTIONAL**	Outside this document
`enterpriseAudit`	object	**OPTIONAL**	Outside this document
`delegationAnchorCapabilityId`	string	CONDITIONAL	Present when `delegationPolicy` was: identifier of the recorded lineage parent

Table 44: Delegated Issuance Response Members

12.3. Receipt Query

`GET /v1/receipts/query` returns the stored receipts that match its query parameters (Table 45). Every parameter is **OPTIONAL**, and the filters that are present all apply.

Parameter	Type	Selects receipts
`capabilityId`	string	whose `capability_id` equals the value
`toolServer`	string	whose `tool_server` equals the value
`toolName`	string	whose `tool_name` equals the value
`outcome`	string	whose decision is the value: `allow`, `deny`, `cancelled`, or `incomplete`
`since`	integer	whose `timestamp` is at or after the value (Unix seconds)
`until`	integer	whose `timestamp` is at or before the value
`minCost`	integer	whose charged cost, in minor units (Section 7.1), is at least the value
`maxCost`	integer	whose charged cost is at most the value
`costCurrency`	string	whose charged cost is in this currency ([ISO4217])
`agentSubject`	string	whose subject key equals the value; the capability's subject key is used only when the receipt subject is absent
`cursor`	integer	after this position (Section 12.3.1)
`limit`	integer	up to this many; default 50, clamped to the range 1 to 200

Table 45: Receipt Query Parameters

The service **MUST** reject the query with 400 if `minCost` exceeds `maxCost`, if `minCost` or `maxCost` is present without `costCurrency`, or if `costCurrency` is not exactly three uppercase ASCII letters. It answers an `outcome` outside the four values with 500. A receipt without cost data matches no cost filter.

A caller authenticated with the service token reads all receipts. The service **MAY** also accept read tokens that are each bound to one tenant; a query authenticated with such a token returns only receipts whose `tenant_id` is that tenant.

12.3.1. Paging

The service orders matching receipts by their position in its store, ascending, and returns those whose position is greater than `cursor`. The response has three members:

`totalCount`: The number of receipts that match the filters, regardless of `cursor` and `limit`.

`nextCursor`: The position of the last receipt returned when the page is full, and `null` otherwise.

`receipts`: An array of receipt objects (Section 6.1).

A client pages by passing `nextCursor` as `cursor` until `nextCursor` is `null`; the last page can be empty. Positions are not otherwise exposed. The service verifies the signature of each receipt before it returns it, and it fails the query with 500 if a stored receipt does not verify. A query that matches nothing returns:

```
{"nextCursor": null, "receipts": [], "totalCount": 0}
```

12.4. Revocation

`POST /v1/revocations` revokes a capability. Its request body has one **REQUIRED** member, `capabilityId`, a string that identifies the capability. Only the service token authenticates this endpoint. The service records the identifier as revoked without checking that it names an issued capability, so a caller can revoke an identifier before any token uses it. It answers 200 only after the revocation is visible in its store, 409 if it has no revocation store, and 500 if the store fails. A 200 response has these members:

`capabilityId`: The identifier from the request.

`revoked`: `true`.

`newlyRevoked`: `false` if the identifier was already revoked, and `true` otherwise.

```
{"capabilityId": "cap-1", "newlyRevoked": true, "revoked": true}
```

`GET /v1/revocations`, authenticated by the service token, reports revocation state. With the parameter `capabilityId`, its response includes `capabilityId` and a boolean `revoked` for that identifier. The response also has `configured` (boolean), `backend` (string), `count` (integer), and `revocations`, an array of up to `limit` objects with members `capabilityId` and `revokedAt` (Unix seconds).

A kernel learns of a revocation only when it checks. Before it dispatches a call, the kernel checks the revocation state of the token's `id` and of each capability in the token's delegation chain (Section 5.8). A kernel configured with a trust-control service makes each check a `GET /v1/revocations` request with `capabilityId` set to the identifier and `limit` set to 1, reads `revoked`, and keeps no cache. The kernel **MUST** deny the call, with a signed receipt, if any identifier is revoked or if a check fails. The trust-control service does not notify kernels of revocations. The reference native driver does not notify agents; the optional `capability_revoked` message remains a permitted wire allocation (Section 10.2).

13. Versioning and Negotiation

Each wire surface has its own version selector. A version match on one surface does not establish a match on another surface.

For the native transport, this document defines `chio-wire-v1`. The agent and kernel establish that value out of band. Native messages carry no in-band version field. A kernel that cannot accept the agreed version closes or resets the transport. Registry code 1000 classifies that out-of-band incompatibility; there is no native numeric-error frame or implemented in-band version exchange. There is no native downgrade exchange.

The hosted edge selects by exact match from its supported set. This document defines MCP version `2025-11-25`. The client sends `params.protocolVersion` in `initialize`; the response carries `result.protocolVersion`. The client carries the selected value in `MCP-Protocol-Version` on subsequent requests. The hosted edge rejects an initialization-time mismatch with JSON-RPC code -32600 and Chio code 1000. An omitted initialize selector uses the compatibility default. A later HTTP version-header mismatch produces HTTP 400 with a plain-text body, not that JSON-RPC error; an omitted header is tolerated. Session admission and state handling are specified in Section 11.

The trust-control interface uses the path prefix `/v1`. A client selects that prefix explicitly. There is no downgrade negotiation for that interface. Future prefixes are separate interface versions.

14. Error Model

Errors describe a failed operation, not proof that a side effect did not occur. In particular, an incomplete outcome records uncertainty about the tool's terminal state. Before repeating an operation with side effects, an agent determines whether it completed or uses the tool's own idempotency contract.

14.1. Error Registry

The numeric registry has eight categories: `protocol`, `auth`, `capability`, `guard`, `budget`, `tool`, `internal`, and `transaction`. Section 17.1 gives every initial allocation. The transaction entries reserve identifiers used by related evidence profiles; this document does not define those profiles.

A transient flag describes whether a changed condition can permit a later attempt. It is not permission to repeat the same request without change. The retry strategy is part of each allocation. For example, `auth_missing_or_invalid` requires fresh credentials, and `budget_exhausted` requires a budget change or reconciliation. A revoked token stays revoked even though a newly issued token can authorize a later call. Tool and internal failures use bounded backoff.

14.2. Surface Mappings

A native error is a `tool_call_response` whose result has `status: err`. The nested error has `code` and, for variants carrying diagnostic text, `detail`. The native names are `capability_denied`, `capability_expired`, `capability_revoked`, `policy_denied`, `tool_server_error`, and `internal_error`. They map to registry codes 2100, 2101, 2102, 3100, 5100, and 6100 respectively. A registry allocation does not add a variant to the closed native error union.

Hosted errors use JSON-RPC and, where supplied, a Chio code in error data. The registered mappings are -32600 for unsupported version and invalid request shape, and -32002 for a session that is not initialized. Authentication failures can occur at HTTP session admission before a JSON-RPC response exists. A client keeps HTTP status, JSON-RPC code, and Chio code distinct.

15. Security Considerations

This section describes the threats that Chio addresses, the controls that address them, and the risks that remain. It follows the order in which authority moves: issuance, delivery to the kernel, admission, dispatch to a tool server, and the receipt.

15.1. Assets and Trust Boundaries

The assets are capability tokens and delegation state, session identifiers and sender bindings, the authenticity of the kernel, the confinement of tool execution, the integrity of receipts and of the decisions they record, and the availability of the kernel and its services.

Authority crosses five boundaries: from the trust-control service to the agent at issuance, from the agent to the kernel over the native transport or the MCP binding, inside the kernel at admission, from the kernel to a tool server at dispatch, and from the kernel to any verifier through the receipt. The kernel is the component that must be trusted at every boundary except the last, where the receipt lets a verifier check the kernel's statements with the kernel's public key.

15.2. Transport Security

Table 46 states the transport requirements for each surface. TLS supplies channel protection; [RFC8446] defines TLS 1.3.

Surface	TLS	Mutual TLS	Without transport security
Native transport	**REQUIRED** across hosts or untrusted networks	**REQUIRED** when the peer's identity is part of the authorization decision	Conformant only over a same-host socket or loopback, for development
MCP binding	**REQUIRED** for any non-loopback deployment	**REQUIRED** when the session's sender binding is an mTLS certificate thumbprint	Conformant only on loopback or in a test harness
Trust-control interface	**REQUIRED** for any non-loopback deployment	**REQUIRED** for service-to-service deployments that rely on transport identity	Conformant only on loopback
Kernel to tool server	Provided by mutual TLS when networked	**REQUIRED** over TCP across processes or hosts	Not conformant for production over a network

Table 46: Transport Security Requirements

A deployment that is missing a transport property that Table 46 requires **MUST** deny the request or restrict itself to a local development configuration. A deployment **MUST NOT** claim confidentiality, peer authentication, or replay resistance for traffic it carries in plaintext across a network.

A hosted edge can run behind a proxy that terminates TLS and passes the client certificate's thumbprint to the edge in a request header. The edge then trusts that header. An operator **MUST** ensure that only the proxy can reach the edge, because a client that reaches the edge directly can set the header itself.

15.3. Token Theft and Replay

A capability token that is not sender-constrained is a bearer token for its validity interval: anyone who obtains it can present it. Signatures, validity intervals, and revocation bound the damage, and a sender proof (Section 5.4) removes it for grants that require one.

The native transport carries no sender proof (Section 10.2), and it has no anti-replay marker of its own: a captured frame that carries a bearer token can be replayed until the token expires or is revoked. The MCP binding can bind a session to a sender key or a certificate thumbprint (Section 11). Operators **SHOULD** issue tokens with the shortest validity interval that the task allows, **SHOULD** require sender constraint for sensitive or cross-host flows where the surface supports it, and **SHOULD** revoke a token as soon as its holder is suspected of compromise (Section 12.4).

A kernel records the nonces of sender proofs it has accepted, and a kernel that restarts or fails over without that record accepts a replayed proof again within the proof's lifetime. Deployments that run several kernel instances, or restart them, **SHOULD** keep that record in shared, durable storage.

15.4. Kernel Impersonation and Receipt Trust

A receipt carries the public key that signed it (`kernel_key`), so a valid signature proves only that the holder of that key signed the receipt. A verifier **MUST** check that `kernel_key` is a key it trusts for the kernel that it expects to have made the decision, using a key it obtained out of band, before it treats a receipt as evidence. Receipt verification therefore depends on how a deployment distributes kernel keys; this document does not define a public transparency system for them (Section 9.6).

In the same way, a kernel accepts a capability token only from an issuer in its configured set of trusted issuer keys (Section 5.8). Distributions **SHOULD** pin or securely provision kernel keys, issuer keys, and certificates, rather than learning them from the traffic they protect.

Capability tokens carry no audience. Any kernel that trusts a token's issuer accepts the token for the grants it names. Operators that run kernels in different trust domains **SHOULD** use a distinct issuer key for each domain, so that a token issued for one domain is not accepted in another.

15.5. Delegation Abuse

An attacker who holds a delegated token can try to widen its scope, truncate its lineage, or claim a parent it does not hold. The attenuation proof and the chain-binding rule (Section 5.6, Section 5.7) make a claimed parent scope verifiable against the trust root or the previous delegation link, and delegated issuance at the trust-control service cannot exceed its signed delegation policy (Section 12.2).

The kernel checks the revocation state of a token and of each ancestor that its delegation chain names (Section 5.8), so revoking a token denies every descendant whose chain names it. Operators **SHOULD** revoke the parent token when they suspect a compromise anywhere below it.

The kernel enforces delegated budget shares against the state it has recorded about sibling tokens (Section 7.4). Two kernels that do not share that state each admit children up to the full share, so a deployment that spreads one delegation family across kernel instances needs a single authority for that state to keep the bound.

15.6. Signature Algorithm Downgrade

Keys and signatures identify their own algorithm (Section 4.4), and a verifier rejects an `algorithm` hint that disagrees with the encoded signature. A verifier that requires post-quantum protection **MUST** enforce a minimum accepted signature suite and **MUST** reject a classical signature where that minimum requires a hybrid one (Section 4.3). A verifier that enforces no minimum accepts classical and hybrid signatures alike, and so does not protect against an attacker who can produce a classical signature for a key the verifier trusts.

15.7. Canonicalization

Signatures cover the signing wrapper of canonical bytes (Section 4.5, Section 4.1), so two parties that parse the same bytes differently can disagree about what was signed. JSON objects with duplicate member names are the usual source of such disagreement. Verifiers **SHOULD** reject a signed object that contains duplicate member names rather than resolving them. Intermediaries **MUST** preserve every member and value in the artifact's typed signing projection when forwarding it. They **MAY** change insignificant JSON whitespace and member order, since verification reconstructs canonical bytes (Section 10.5). They **MUST NOT** add unauthenticated members and represent them as part of the signed statement.

15.8. Tool Server Confinement

The kernel decides whether a call reaches a tool server, and it checks the server, the tool, and the arguments against the grant before dispatch. It does not constrain what the tool server does once it runs. A tool server **MUST** be treated as less trusted than the kernel unless it runs in the same reviewed binary and privilege domain. Operators **SHOULD** confine tool servers with operating-system or container isolation, least-privilege file system access, and outbound network controls. A compromised tool server keeps whatever host privileges it was given.

15.9. Resource Exhaustion

A kernel rejects native frames larger than the maximum payload length before allocating a buffer for them (Section 10.1). The MCP binding limits each session to one notification stream and never resumes a session that has reached a terminal state (Section 11.4). Deployments **SHOULD** apply rate, concurrency, and time limits at the hosted edge and the trust-control service, and **SHOULD** bound per-session buffers and queues. An authenticated caller can still spend its own budget and queue share; budgets bound the cost of that, not its occurrence.

15.10. Budgets and Approvals

A budget hold is reserved before dispatch and captured once (Section 7.3); a kernel denies a request that tries to capture a hold a second time. Approval tokens and threshold approvals are single use (Section 8.4, Section 8.5), and a kernel records the ones it has accepted. As with sender proofs, loss of a singular approval's replay record can allow replay within its validity interval, so deployments relying on singular approvals **SHOULD** keep that record in durable storage. Threshold approval **MUST** use the durable admission replay reservation in Section 8.5.2 and **MUST** deny admission when that durable reservation is unavailable.

Threshold approval counts distinct keys. Distinct keys are not distinct people or organizations: an operator who configures an eligible key set decides how independent the approvers are.

15.11. Checkpoint Equivocation

A kernel signs its own checkpoints, so a kernel whose key is compromised, or a dishonest operator, can sign two checkpoints that disagree and show each to a different verifier. Consistency proofs let a verifier detect the disagreement when it holds both checkpoints (Section 9.3); nothing in this document makes a verifier hold both. Section 9.6 states what a verified checkpoint establishes.

15.12. Provenance Inflation

Callers can supply context that the kernel cannot check, such as model metadata and call-chain information. The kernel records such context as `asserted` (Section 8.6, Section 11.7). A report or export **MUST NOT** present `asserted` context as `observed` or `verified`.

15.13. Attestation

Attestation evidence can inform issuance (Section 12.1), but it does not authorize a call by itself. A profile that binds an attestation digest to a token **MUST** also bind the request to the token's sender, by a sender proof or by mutual TLS continuity, on the same request.

15.14. Clock Skew

A kernel treats a token as expired when its clock reaches `expires_at` and as not yet valid while its clock is before `issued_at`, with no tolerance for skew (Section 5.8). An issuer whose clock runs ahead of the kernel's produces tokens that the kernel rejects at first, and a kernel whose clock runs behind accepts tokens after they expire. Issuers and kernels need synchronized clocks.

16. Privacy Considerations

Blocking dispatch does not remove attempted arguments from a denial receipt. A pre-invocation guard can deny an attempt while its raw arguments remain in signed audit evidence; protect receipt storage and read access independently of tool admission.

Receipts are designed to be kept and shown to other parties, and they contain more than a verdict. A receipt records the tool server and tool, the capability identifier, the call's parameters as the agent sent them together with their hash, a hash of the output, the guards that were evaluated and their evidence, the chain of actors that the kernel attributes the call to, the time, and, in multi-tenant deployments, the tenant (Section 6.1). A receipt has no dedicated output field. Free-form metadata can still contain output or other sensitive data.

Parameters are the most sensitive part. Because a receipt signs the parameters that the agent sent, anyone who can read the receipt can read them, including secrets or personal data that the agent placed there. Agents and tool designers **SHOULD** keep secrets and personal data out of tool arguments where the tool allows it. Deployments that handle regulated data **SHOULD** evaluate a guard that detects and redacts or blocks sensitive values before the call, and **SHOULD** restrict who can query receipts.

Leaving the output out of a receipt does not hide a predictable output. The `content_hash` of a value result is an unsalted SHA-256 digest of its canonical JSON (Section 6.4). The per-chunk digests in stream metadata and a child receipt's `outcome_hash` (Section 6.8) are computed the same way. The signing nonce (Section 6.6) is placed in metadata and does not enter these digests. A reader who can enumerate the plausible outputs of a call can hash each candidate and compare: a boolean, an approval verdict, a status code, or a short identifier is recovered this way, even when the arguments carry nothing sensitive. The same reader can also tell whether two receipts record identical outputs. A receipt therefore discloses any low-entropy output to everyone who can read it.

Deployments that share receipts beyond the parties entitled to the output need to account for this. Mitigations include limiting such receipts to that audience, designing tools whose outputs carry enough entropy to resist enumeration, and a future profile that binds a salted or keyed commitment to the output instead of the plain digest, with the salt or key disclosed only to parties entitled to check the content. Apart from the delivery-mismatch commitment (Section 6.4), which does not bind the delivered output, this document defines no such commitment.

Receipts are linkable. The capability identifier and the subject key connect every call made under one token, and a delegation chain reveals who delegated to whom. A party that sees many receipts can build a profile of an agent's activity and of the organization behind it.

The `redaction_mode` field records how much of a receipt's detail was redacted when it was signed or exported. It describes the receipt; it does not by itself remove anything.

A trust-control service limits receipt queries by tenant (Section 12.3). A tenant sees its own receipts; an administrator sees all of them. Checkpoints commit to batches of receipts from every tenant that a kernel serves, so the size and timing of a batch can reveal aggregate activity across tenants to anyone who can see the checkpoint.

When MCP logging is enabled, a hosted edge can send the client log messages that name denied tools and carry error text (Section 11). Operators **SHOULD** review what those messages reveal before enabling them for untrusted clients.

17. IANA Considerations

This document requests four registries under a new "Chio Protocol" registry group. Each uses Specification Required as defined in [RFC8126]. Registration takes effect through IANA; the initial allocations below are requests, not assertions that IANA has assigned these values.

String values in the message and artifact registries are case-sensitive ASCII names of 1 to 96 octets using lowercase letters, digits, underscore, hyphen, and period. Signature-suite names additionally permit plus. These are proposed registration rules; compatibility decoders can accept other key spellings as specified in Section 4.4.

A registration identifies its value, name, description, and a stable specification. The designated experts check uniqueness, complete encoding and verification rules, and whether the specification states its security and interoperability consequences. A registration does not establish cryptographic suitability or implementation support. An update retains the meaning of previously allocated values; a change that breaks that meaning receives a new value.

17.1. Chio Error Codes

The registration template is: numeric code; symbolic name; category; transient flag; retry strategy and guidance; native error name, if any; JSON-RPC code, if any; and reference. Codes are decimal integers from 0 through 2147483647. Negative JSON-RPC codes belong to their separate namespace and are not values in this registry. Experts check that the retry guidance distinguishes a changed condition from repeating an unsafe side effect. The initial allocations follow. Core allocations reference this document. Values 7100 through 7119 are reserved for related transaction profiles, rather than allocated by this document; their diagnostic names below describe current implementation usage. A future Specification Required registration supplies a permanent public specification before assigning any reserved value.

1000 (`protocol_version_unsupported`): Category `protocol`; transient `false`; retry `do_not_retry_until_version_change`. JSON-RPC code: -32600. Retry only after selecting a supported protocol version.

1001 (`session_not_initialized`): Category `protocol`; transient `false`; retry `reinitialize`. JSON-RPC code: -32002. Open a new session and rerun initialize before retrying the operation.

1002 (`invalid_request_shape`): Category `protocol`; transient `false`; retry `do_not_retry`. JSON-RPC code: -32600. Correct the request shape before retrying.

1100 (`auth_missing_or_invalid`): Category `auth`; transient `true`; retry `retry_after_refresh`. Acquire valid credentials or refresh the token before retrying.

2100 (`capability_denied`): Category `capability`; transient `false`; retry `do_not_retry`. Native name: `capability_denied`. Do not retry the same capability without correcting signature, scope, or subject binding.

2101 (`capability_expired`): Category `capability`; transient `false`; retry `retry_after_reissue`. Native name: `capability_expired`. Issue or obtain a fresh capability before retrying.

2102 (`capability_revoked`): Category `capability`; transient `false`; retry `retry_after_reissue`. Native name: `capability_revoked`. Obtain a newly issued capability; the revoked capability must not be retried.

3100 (`guard_denied`): Category `guard`; transient `false`; retry `do_not_retry`. Native name: `policy_denied`. Adjust the request or policy inputs before retrying.

4100 (`budget_exhausted`): Category `budget`; transient `true`; retry `retry_after_budget_change`. Retry only after budget replenishment, grant widening, or billing reconciliation.

5100 (`tool_server_error`): Category `tool`; transient `true`; retry `retry_with_backoff`. Native name: `tool_server_error`. Use bounded exponential backoff unless upstream tooling documents a permanent failure.

6100 (`internal_error`): Category `internal`; transient `true`; retry `retry_with_backoff`. Native name: `internal_error`. Retry with bounded backoff and escalate if the condition persists.

7100 (`transaction_passport_schema_unsupported`): Category `transaction`; transient `false`; retry `do_not_retry`. Related-profile diagnostic name: `transaction_passport_schema_unsupported`. Regenerate the passport with a registered transaction passport schema.

7101 (`transaction_passport_hash_mismatch`): Category `transaction`; transient `false`; retry `do_not_retry`. Related-profile diagnostic name: `transaction_passport_hash_mismatch`. Rebuild the passport root with matching evidence graph, claim set, and policy digests.

7102 (`transaction_graph_not_closed`): Category `transaction`; transient `false`; retry `do_not_retry`. Related-profile diagnostic name: `transaction_graph_not_closed`. Include every required evidence, claim-set, policy, and receipt node in the transaction graph.

7103 (`transaction_graph_cycle`): Category `transaction`; transient `false`; retry `do_not_retry`. Related-profile diagnostic name: `transaction_graph_cycle`. Emit an acyclic evidence graph whose dependency edges can be topologically verified.

7104 (`transaction_required_claim_missing`): Category `transaction`; transient `false`; retry `do_not_retry`. Related-profile diagnostic name: `transaction_required_claim_missing`. Add verified evidence for the required claim or remove the unsupported requirement.

7105 (`transaction_artifact_hash_mismatch`): Category `transaction`; transient `false`; retry `do_not_retry`. Related-profile diagnostic name: `transaction_artifact_hash_mismatch`. Regenerate the artifact set with matching transaction evidence digests.

7106 (`transaction_identity_not_bound`): Category `transaction`; transient `false`; retry `do_not_retry`. Related-profile diagnostic name: `transaction_identity_not_bound`. Bind identity evidence to the transaction passport subject and evidence graph.

7107 (`transaction_authorization_not_bound`): Category `transaction`; transient `false`; retry `do_not_retry`. Related-profile diagnostic name: `transaction_authorization_not_bound`. Bind capability, policy, approval, or guard evidence to the governed transaction.

7108 (`transaction_receipt_uncheckpointed`): Category `transaction`; transient `false`; retry `do_not_retry`. Related-profile diagnostic name: `transaction_receipt_uncheckpointed`. Provide checkpointed receipt or inclusion evidence for the transaction receipt.

7109 (`transaction_runtime_proof_rejected`): Category `transaction`; transient `false`; retry `do_not_retry`. Related-profile diagnostic name: `transaction_runtime_proof_rejected`. Regenerate runtime security, parity, lease, nonce, revocation, sandbox, ack, and terminal receipt evidence.

7110 (`transaction_buyer_review_rejected`): Category `transaction`; transient `false`; retry `do_not_retry`. Related-profile diagnostic name: `transaction_buyer_review_rejected`. Regenerate buyer review evidence that matches the transaction passport and verifier policy.

7111 (`transaction_settlement_unverified`): Category `transaction`; transient `false`; retry `do_not_retry`. Related-profile diagnostic name: `transaction_settlement_unverified`. Include settlement evidence bound to the transaction order, amount, currency, rail, and receipt lineage.

7112 (`transaction_dispute_unbound`): Category `transaction`; transient `false`; retry `do_not_retry`. Related-profile diagnostic name: `transaction_dispute_unbound`. Bind dispute, refund, remediation, and settlement reversal evidence to the transaction passport and receipts.

7113 (`transaction_transparency_preview_not_allowed`): Category `transaction`; transient `false`; retry `do_not_retry`. Related-profile diagnostic name: `transaction_transparency_preview_not_allowed`. Provide verified transparency inclusion evidence or remove the transparency claim.

7114 (`transaction_claim_rejected`): Category `transaction`; transient `false`; retry `do_not_retry`. Related-profile diagnostic name: `transaction_claim_rejected`. Regenerate the family evidence so every required claim verifies.

7115 (`transaction_artifact_schema_invalid`): Category `transaction`; transient `false`; retry `do_not_retry`. Related-profile diagnostic name: `transaction_artifact_schema_invalid`. Regenerate the artifact against its registered schema.

7116 (`transaction_signature_invalid`): Category `transaction`; transient `false`; retry `do_not_retry`. Related-profile diagnostic name: `transaction_signature_invalid`. Re-sign the proof bundle with a key the verifier trusts.

7117 (`transaction_verifier_feature_unsupported`): Category `transaction`; transient `false`; retry `do_not_retry`. Related-profile diagnostic name: `transaction_verifier_feature_unsupported`. Pin the required trust context for the verifier, or remove the unsupported feature from the proof bundle.

7118 (`transaction_negative_case_mismatch`): Category `transaction`; transient `false`; retry `do_not_retry`. Related-profile diagnostic name: `transaction_negative_case_mismatch`. Regenerate the negative case so it fails with its declared failure code, or correct the declared code.

7119 (`transaction_verifier_context_invalid`): Category `transaction`; transient `false`; retry `do_not_retry`. Related-profile diagnostic name: `transaction_verifier_context_invalid`. Correct the pinned verifier trust context (keys, digests, authority documents, or settlement settings) and rerun the verifier.

17.2. Chio Signature Suites

The registration template is: suite name; public-key encoding; signature encoding; message preparation and hash; verification rule; and reference. Experts check that key and signature encodings are unambiguous and that composite suites specify every component and require all components to verify. Section 4.3 and Section 4.4 define the initial values:

- `ed25519`: Ed25519, bare hexadecimal key and signature.
- `p256`: ECDSA P-256 with SHA-256; SEC1 uncompressed key and DER signature.
- `p384`: ECDSA P-384 with SHA-384; SEC1 uncompressed key and DER signature.
- `ed25519+mldsa65`: Ed25519 and ML-DSA-65, composite encoding.
- `p256+mldsa65`: ECDSA P-256 and ML-DSA-65, composite encoding.
- `p384+mldsa65`: ECDSA P-384 and ML-DSA-65, composite encoding.

The envelope hint `hybrid` identifies a composite. It is not a seventh suite and does not select its components. The composite's encoded algorithm set selects them.

17.3. Chio Native Message Types

The registration template is: `type` value; direction (agent to kernel or kernel to agent); payload fields and types; terminal or streaming behavior; and reference. Experts check that state transitions, correlation, and failure behavior are defined. Initial values are:

Value	Direction	Reference
`tool_call_request`	Agent to kernel	Section 10.2
`list_capabilities`	Agent to kernel	Section 10.2
`heartbeat`	Both directions	Section 10.2
`tool_call_chunk`	Kernel to agent	Section 10.2
`tool_call_response`	Kernel to agent	Section 10.2
`capability_list`	Kernel to agent	Section 10.2
`capability_revoked`	Kernel to agent	Section 10.2

Table 47: Initial Native Message Types

The shared `heartbeat` allocation defines a payload in each direction. Registering a new type does not make old decoders accept it. A future wire version or explicit extension agreement defines its availability.

17.4. Chio Signed Artifact Schemas

The registration template is: schema identifier; artifact role; exact signed projection; canonical encoding; verification procedure; and reference. The schema identifier is the `schema_id` of the signing wrapper (Section 4.5), which is the only signature domain separation. Experts check that a verifier cannot confuse artifacts with different authority or trust roles. A registry identifier can name a projection without appearing as a member in its wire envelope. In particular, receipt records carry no top-level `schema` member.

Initial values in the scope of this document are:

- `chio.capability.v1`: capability token (Section 5).
- `chio.receipt.v1`: receipt signing projection (Section 6).
- `chio.aggregate-budget-root.v1`: aggregate budget root (Section 7.5).
- `chio.execution_nonce.v1`: authoritative execution nonce (Section 7.7).
- `chio.operation_execution_nonce.v1`: operation-owned execution nonce with an operation-bound signing context (Section 7.7.1).
- `chio.threshold-approval-proposal.v1`: threshold proposal (Section 8.5).
- `chio.checkpoint_statement.v1`: signed receipt checkpoint (Section 9.1).

Approval tokens, delegation links, and child receipts do not gain a new schema member from this registration. Unsigned proof records do not become signed artifacts through inclusion in a registry.

18. Implementation Status

This section is to be removed before publishing as an RFC.

This section is included according to [RFC7942] and is removed before publication as an RFC. Its purpose is to inform discussion. It is not an IETF endorsement, an interoperability certification, or evidence of independent implementations.

The Chio Rust implementation is available at https://github.com/backbay-labs/chio, under the Apache-2.0 license. The contact is Connor Whelan, connor@backbay.io. It implements capability tokens, signed receipts, framed native messages, hosted MCP, trust-control endpoints, governed transactions, budget accounting, and receipt checkpoints. Cryptographic suite and operational support depend on the enabled build features and configuration.

The inspected implementation profile is leader-local single-writer trust-control with eventual repair; hosted admission on one node or a dedicated session owner; atomic monetary accounting in one store with bounded clustered overrun; and signed local checkpoint evidence. Static bearer,

tokens without sender bindings, and shared-owner hosted authentication are compatibility modes, not the stronger operational profile. Neither checkpoint signatures nor these configurations establish public transparency or distributed spend linearizability.

TLS and proxy-only reachability requirements are deployment duties. The hosted listeners themselves accept plain TCP; operators provide the required transport protection. The local token issuer pairs attestation confirmation with a sender key or certificate binding, but imported JWT and introspection paths can accept attestation-only confirmation. Such imports require an issuer or deployment policy enforcing Section 15.13; that property is not a universal edge check.

The portable core and the hosted kernel have different enforcement surfaces. The portable core handles tool grants and refuses constraints that require unavailable runtime evidence. The hosted kernel supplies resource and prompt matching, revocation state, sender-constraint verification, budget holds, and governed transaction validation. A producer or consumer claiming conformance states its supported profile; parsing an artifact does not establish enforcement of its constraints.

The implementation departs from two requirements of this document. Both are tracked for alignment and are recorded here, not as protocol behavior. First, it checks `scope_hash` on delegation links only when a token carries an attenuation proof, nonempty `scope_attenuations`, or a budget share (Section 5.7). For a plain delegation chain its compatibility path verifies link signatures, key connectivity, timestamps, and depth, and accepts links that omit `scope_hash`, so it admits tokens that a conforming verifier rejects (work item DELEG-SCOPE-HASH). The binding vector `valid_delegated_capability` (Appendix B) has such a link; its corpus result does not cover this requirement. Second, its ordinary receipt signature check does not compare a present `algorithm` hint with the signature encoding; only its floor-aware receipt verifier does. A receipt whose unsigned hint was altered therefore passes the ordinary check, although Section 6.9 requires rejecting it (work item RCPT-ALG-HINT).

The native conformance suite is in `tests/conformance/native/`. Its six scenario descriptors cover capability validation, delegation attenuation, receipt integrity, revocation propagation, proof of possession, and governed transaction enforcement. The runner supports three driver modes: deterministic `artifact` validation, native framed `stdio` exchange with an external executable, and a test-only `http` bridge exposing `POST /chio-conformance/v1/invoke`. That bridge is a conformance harness contract, not a fourth production wire surface.

Binding vectors are described in Appendix B. Passing these vectors establishes the exercised encoding and validation cases; it does not certify budget durability, deployment isolation, checkpoint publication, or end-to-end behavior of a third-party tool.

19. References

19.1. Normative References

[FIPS180-4] National Institute of Standards and Technology, "Secure Hash Standard (SHS)", FIPS PUB 180-4, DOI 10.6028/NIST.FIPS.180-4, August 2015, <https://doi.org/10.6028/NIST.FIPS.180-4>.

[FIPS186-5] National Institute of Standards and Technology, "Digital Signature Standard (DSS)", FIPS PUB 186-5, DOI 10.6028/NIST.FIPS.186-5, February 2023, <https://doi.org/10.6028/NIST.FIPS.186-5>.

[FIPS204] National Institute of Standards and Technology, "Module-Lattice-Based Digital Signature Standard", FIPS PUB 204, DOI 10.6028/NIST.FIPS.204, August 2024, <https://doi.org/10.6028/NIST.FIPS.204>.

[ISO4217] International Organization for Standardization, "Codes for the representation of currencies", ISO 4217:2015, August 2015, <https://www.iso.org/iso-4217-currency-codes.html>.

[JSON-RPC] JSON-RPC Working Group, "JSON-RPC 2.0 Specification", January 2013, <https://www.jsonrpc.org/specification>.

[MCP] Model Context Protocol, "Model Context Protocol Specification, Version 2025-11-25", 25 November 2025, <https://modelcontextprotocol.io/specification/2025-11-25>.

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <https://www.rfc-editor.org/rfc/rfc2119>.

[RFC3279] Bassham, L., Polk, W., and R. Housley, "Algorithms and Identifiers for the Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile", RFC 3279, DOI 10.17487/RFC3279, May 2002, <https://www.rfc-editor.org/rfc/rfc3279>.

[RFC7515] Jones, M., Bradley, J., and N. Sakimura, "JSON Web Signature (JWS)", RFC 7515, DOI 10.17487/RFC7515, May 2015, <https://www.rfc-editor.org/rfc/rfc7515>.

[RFC7519] Jones, M., Bradley, J., and N. Sakimura, "JSON Web Token (JWT)", RFC 7519, DOI 10.17487/RFC7519, May 2015, <https://www.rfc-editor.org/rfc/rfc7519>.

[RFC7662] Richer, J., Ed., "OAuth 2.0 Token Introspection", RFC 7662, DOI 10.17487/RFC7662, October 2015, <https://www.rfc-editor.org/rfc/rfc7662>.

[RFC8032] Josefsson, S. and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)", RFC 8032, DOI 10.17487/RFC8032, January 2017, <https://www.rfc-editor.org/rfc/rfc8032>.

[RFC8126] Cotton, M., Leiba, B., and T. Narten, "Guidelines for Writing an IANA Considerations Section in RFCs", BCP 26, RFC 8126, DOI 10.17487/RFC8126, June 2017, <https://www.rfc-editor.org/rfc/rfc8126>.

[RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, DOI 10.17487/RFC8174, May 2017, <https://www.rfc-editor.org/rfc/rfc8174>.

[RFC8259] Bray, T., Ed., "The JavaScript Object Notation (JSON) Data Interchange Format", STD 90, RFC 8259, DOI 10.17487/RFC8259, December 2017, <https://www.rfc-editor.org/rfc/rfc8259>.

[RFC8446] Rescorla, E., "The Transport Layer Security (TLS) Protocol Version 1.3", RFC 8446, DOI 10.17487/RFC8446, August 2018, <https://www.rfc-editor.org/rfc/rfc8446>.

[RFC8785] Rundgren, A., Jordan, B., and S. Erdtman, "JSON Canonicalization Scheme (JCS)", RFC 8785, DOI 10.17487/RFC8785, June 2020, <https://www.rfc-editor.org/rfc/rfc8785>.

[RFC9110] Fielding, R., Ed., Nottingham, M., Ed., and J. Reschke, Ed., "HTTP Semantics", STD 97, RFC 9110, DOI 10.17487/RFC9110, June 2022, <https://www.rfc-editor.org/rfc/rfc9110>.

[RFC9162] Laurie, B., Messeri, E., and R. Stradling, "Certificate Transparency Version 2.0", RFC 9162, DOI 10.17487/RFC9162, December 2021, <https://www.rfc-editor.org/rfc/rfc9162>.

[RFC9449] Fett, D., Campbell, B., Bradley, J., Lodderstedt, T., Jones, M., and D. Waite, "OAuth 2.0 Demonstrating Proof of Possession (DPoP)", RFC 9449, DOI 10.17487/RFC9449, September 2023, <https://www.rfc-editor.org/rfc/rfc9449>.

[RFC9728] Jones, M.B., Hunt, P., and A. Parecki, "OAuth 2.0 Protected Resource Metadata", RFC 9728, DOI 10.17487/RFC9728, April 2025, <https://www.rfc-editor.org/rfc/rfc9728>.

[SEC1] Standards for Efficient Cryptography Group, "SEC 1: Elliptic Curve Cryptography, Version 2.0", May 2009, <https://www.secg.org/sec1-v2.pdf>.

[SSE] WHATWG, "HTML Living Standard, Section 9.2: Server-sent events", <https://html.spec.whatwg.org/multipage/server-sent-events.html>.

19.2. Informative References

[A2A] Linux Foundation A2A Project, "Agent2Agent (A2A) Protocol Specification", <https://a2a-protocol.org/latest/specification/>.

[BISCUIT] Eclipse Biscuit, "Biscuit, a bearer token with offline attenuation and decentralized verification", <https://doc.biscuitsec.org/reference/specifications>.

[I-D.ietf-wimse-arch-08] Salowey, J. A., Rosomakho, Y., and H. Tschofenig, "Workload Identity in a Multi System Environment (WIMSE) Architecture", Work in Progress, Internet-Draft, draft-ietf-wimse-arch-08, 6 July 2026, <https://datatracker.ietf.org/doc/html/draft-ietf-wimse-arch-08>.

[MACAROONS] Birgisson, A., Politz, J. G., Erlingsson, U., Taly, A., Vrable, M., and M. Lentczner, "Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud", Network and Distributed System Security Symposium NDSS 2014, February 2014, <https://www.ndss-symposium.org/ndss2014/programme/macaroons-cookies-contextual-caveats-decentralized-authorization-cloud/>.

[OID4VCI] OpenID Foundation, "OpenID for Verifiable Credential Issuance 1.0", 16 September 2025, <https://openid.net/specs/openid-4-verifiable-credential-issuance-1_0.html>.

[OID4VP] OpenID Foundation, "OpenID for Verifiable Presentations 1.0", 9 July 2025, <https://openid.net/specs/openid-4-verifiable-presentations-1_0.html>.

[RFC7942] Sheffer, Y. and A. Farrel, "Improving Awareness of Running Code: The Implementation Status Section", BCP 205, RFC 7942, DOI 10.17487/RFC7942, July 2016, <https://www.rfc-editor.org/rfc/rfc7942>.

[RFC8693] Jones, M., Nadalin, A., Campbell, B., Ed., Bradley, J., and C. Mortimore, "OAuth 2.0 Token Exchange", RFC 8693, DOI 10.17487/RFC8693, January 2020, <https://www.rfc-editor.org/rfc/rfc8693>.

[RFC8792] Watsen, K., Auerswald, E., Farrel, A., and Q. Wu, "Handling Long Lines in Content of Internet-Drafts and RFCs", RFC 8792, DOI 10.17487/RFC8792, June 2020, <https://www.rfc-editor.org/rfc/rfc8792>.

[RFC9334] Birkholz, H., Thaler, D., Richardson, M., Smith, N., and W. Pan, "Remote ATtestation procedureS (RATS) Architecture", RFC 9334, DOI 10.17487/RFC9334, January 2023, <https://www.rfc-editor.org/rfc/rfc9334>.

[RFC9396] Lodderstedt, T., Richer, J., and B. Campbell, "OAuth 2.0 Rich Authorization Requests", RFC 9396, DOI 10.17487/RFC9396, May 2023, <https://www.rfc-editor.org/rfc/rfc9396>.

[RFC9635] Richer, J., Ed. and F. Imbault, "Grant Negotiation and Authorization Protocol (GNAP)", RFC 9635, DOI 10.17487/RFC9635, October 2024, <https://www.rfc-editor.org/rfc/rfc9635>.

[RFC9943] Birkholz, H., Delignat-Lavaud, A., Fournet, C., Deshpande, Y., and S. Lasker, "An Architecture for Trustworthy and Transparent Digital Supply Chains", RFC 9943, DOI 10.17487/RFC9943, June 2026, <https://www.rfc-editor.org/rfc/rfc9943>.

[UCAN] UCAN Working Group, "User Controlled Authorization Network (UCAN) Specification", <https://github.com/ucan-wg/spec>.

[VC-DATA-MODEL-2.0] World Wide Web Consortium, "Verifiable Credentials Data Model v2.0", 15 May 2025, <https://www.w3.org/TR/vc-data-model-2.0/>.

Appendix A. Examples

The examples in this appendix are built from two cases of the Chio binding vector corpus, `capability/v1.json` case `valid_no_delegation_chain` and `receipt/v1.json` case `allow_receipt`, so every key, hash, and signature in them is exact. Lines longer than 69 characters are folded as described in [RFC8792], and each folded block begins with the header line that document defines.

A.1. Capability Token

The following case is `capability/v1.json` case `valid_no_delegation_chain` from the Chio binding vector corpus. It is a directly issued token with no delegation chain and one tool grant, pretty-printed here for reading. Capability tokens are defined in Section 5.

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

{
  "expires_at": 1710000800,
  "id": "cap-bindings-direct",
  "issued_at": 1710000200,
  "issuer": "66be7e332c7a453332bd9d0a7f7db055f5c5ef1a06ada66d98b39fb\
      6810c473a",
  "schema": "chio.capability.v1",
  "scope": {
    "grants": [
      {
        "constraints": [
          {
            "type": "path_prefix",
            "value": "/workspace/"
          }
        ],
        "max_invocations": 3,
        "operations": [
          "invoke",
          "read_result"
        ],
        "server_id": "srv-files",
        "tool_name": "file_read"
      }
    ]
  },
  "signature": "423e1a5ce4c07d2ef4aa8c11c065b1d778dba80d4e91242ca59c\
      0a431529965bc6a332f9e431d4fd11c68b0431fde84b48991476dd5bea9137\
      d8c8c8471c6804",
  "subject": "0b513ad9b4924015ca0902ed079044d3ac5dbec2306f06948c10da\
      8eb6e39f2d"
}
```

The signing body of the token is its canonical form without the `signature` member. The member `"schema":"chio.capability.v1"` sorts between `issuer` and `scope`. The signing body is the `capability_signing_body_canonical_json` value of the case:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

{"expires_at":1710000800,"id":"cap-bindings-direct","issued_at":\
1710000200,"issuer":\
"66be7e332c7a453332bd9d0a7f7db055f5c5ef1a06ada66d98b39fb6810c473a",\
"schema":"chio.capability.v1","scope":{"grants":[{"constraints":\
[{"type":"path_prefix","value":"/workspace/"}],"max_invocations":3,\
"operations":["invoke","read_result"],"server_id":"srv-files",\
"tool_name":"file_read"}]},"subject":\
"0b513ad9b4924015ca0902ed079044d3ac5dbec2306f06948c10da8eb6e39f2d"}
```

The signing wrapper (Section 4.5) of this body under the schema id `chio.capability.v1` is these 88 octets, in hex:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

6368696f2e7369676e696e672d777261707065722e7631005d8d33c3d56ece64cca5\
a6fa61a763d04ce26ebfd9235ec1f75e81b28a6a535f89dfefe2ef217fd1b424ff7e\
d5825d3920ae3258f527b25fc9c573a994bbdba3
```

The `signature` member is an Ed25519 signature over the wrapper that verifies under the key in `issuer`.

A.2. Tool Call Request Frame

The following request is built from `capability/v1.json` case `valid_no_delegation_chain` and `receipt/v1.json` case `allow_receipt` of the Chio binding vector corpus. Its `capability_token` is the token of Appendix A.1, and its `server_id`, `tool`, and `params` are the `tool_server`, `tool_name`, and `action.parameters` members of the receipt of Appendix A.3. Its `id` is illustrative. The request is not a case of the corpus, and its canonical form is:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

{"capability_token":{"expires_at":1710000800,"id":\
"cap-bindings-direct","issued_at":1710000200,"issuer":\
"66be7e332c7a453332bd9d0a7f7db055f5c5ef1a06ada66d98b39fb6810c473a",\
"schema":"chio.capability.v1","scope":{"grants":[{"constraints":\
[{"type":"path_prefix","value":"/workspace/"}],"max_invocations":3,\
"operations":["invoke","read_result"],"server_id":"srv-files",\
"tool_name":"file_read"}]},"signature":"423e1a5ce4c07d2ef4aa8c11c065\
b1d778dba80d4e91242ca59c0a431529965bc6a332f9e431d4fd11c68b0431fde84b\
48991476dd5bea9137d8c8c8471c6804","subject":\
"0b513ad9b4924015ca0902ed079044d3ac5dbec2306f06948c10da8eb6e39f2d"},\
"id":"req-001","params":{"mode":"read","path":\
"/workspace/docs/roadmap.md"},"server_id":"srv-files","tool":\
"file_read","type":"tool_call_request"}
```

In the native transport (Section 10), a message is sent as a frame: a four-octet length in big-endian order, followed by the canonical form as the payload. The payload here is 758 octets long, so the length prefix is:

```
000002f6
```

The first 32 octets of the payload are the ASCII text `{"capability_token":{"expires_at`. In hex they are:

```
7b226361706162696c6974795f746f6b656e223a7b22657870697265735f6174
```

A.3. Receipt

The following case is `receipt/v1.json` case `allow_receipt` from the Chio binding vector corpus. It records an allow decision for a call with the server, tool, and parameters of Appendix A.2, and it is pretty-printed here for reading. Its `capability_id` is `cap-bindings-001`. The capability and receipt cases of the corpus are independent, so that value is not the `id` of the token in Appendix A.1. Receipts are defined in Section 6.

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

{
```

```
  "action": {
    "parameter_hash": "035350b0e6a021a4c924a149111944945e94d12f6d8a5\
       e24e4461e7009ebf929",
    "parameters": {
      "mode": "read",
      "path": "/workspace/docs/roadmap.md"
    }
  },
  "boundary_class": "prevent",
  "capability_id": "cap-bindings-001",
  "content_hash": "4062edaf750fb8074e7e83e0c9028c94e32468a8b6f161477\
     4328ef045150f93",
  "decision": {
    "verdict": "allow"
  },
  "evidence": [
    {
      "details": "path allowed",
      "guard_name": "ForbiddenPathGuard",
      "verdict": true
    },
    {
      "details": "no secrets detected",
      "guard_name": "SecretLeakGuard",
      "verdict": true
    }
  ],
  "id": "c9909f733d5fb367922293f0167d06db715393be114f212096cf2866f29\
     433ea",
  "kernel_key": "ea4a6c63e29c520abef5507b132ec5f9954776aebebe7b92421\
     eea691446d22c",
  "metadata": {
    "surface": "bindings-vectors",
    "version": 1
  },
  "policy_hash": "policy-bindings-v1",
  "receipt_kind": "mediated_decision",
  "redaction_mode": "none",
  "signature": "67e3bfef7a6f3da592811e088d396f3186fbc1533632b273fbd3\
     a2f6ce60eb099418d256853398661d802d179ea869acb584bed757681f501f\
     2c40fd4fde790a",
  "timestamp": 1710000200,
  "tool_name": "file_read",
  "tool_origin": "caller_executed",
  "tool_server": "srv-files",
  "trust_level": "mediated"
}
```

The signing body of the receipt is the canonical form of a JSON object with two members: `body`, which is the receipt without its `id` and `signature` members, and `id`, which is the SHA-256 digest of the canonical form of `body`, in lowercase hex. The `receipt_body_canonical_json` value of the case is the canonical form of the receipt without its `signature` member. The signing body is:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

{"body":{"action":{"parameter_hash":\
"035350b0e6a021a4c924a149111944945e94d12f6d8a5e24e4461e7009ebf929",\
"parameters":{"mode":"read","path":"/workspace/docs/roadmap.md"}},\
"boundary_class":"prevent","capability_id":"cap-bindings-001",\
"content_hash":\
"4062edaf750fb8074e7e83e0c9028c94e32468a8b6f1614774328ef045150f93",\
"decision":{"verdict":"allow"},"evidence":[{"details":\
"path allowed","guard_name":"ForbiddenPathGuard","verdict":true},\
{"details":"no secrets detected","guard_name":"SecretLeakGuard",\
"verdict":true}],"kernel_key":\
"ea4a6c63e29c520abef5507b132ec5f9954776aebebe7b92421eea691446d22c",\
"metadata":{"surface":"bindings-vectors","version":1},"policy_hash":\
"policy-bindings-v1","receipt_kind":"mediated_decision",\
"redaction_mode":"none","timestamp":1710000200,"tool_name":\
"file_read","tool_origin":"caller_executed","tool_server":\
"srv-files","trust_level":"mediated"},"id":\
"c9909f733d5fb367922293f0167d06db715393be114f212096cf2866f29433ea"}
```

The signing wrapper (Section 4.5) of this body under the schema id `chio.receipt.v1` is these 88 octets, in hex:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

6368696f2e7369676e696e672d777261707065722e76310030a0a0cb00102ac64808\
db450924b39544cc41b773e6d2b7f0d899021a6c2ae16d0752cbe80e4a4f8204063c\
d20e9a17bbcb5c5ce16d3ea122fd05cfe80365ee
```

The `signature` member is an Ed25519 signature over the wrapper that verifies under the key in `kernel_key`. The `parameter_hash` member of `action` is the SHA-256 digest of the canonical form of its `parameters` member, in the same encoding.

Appendix B. Test Vectors

The cases in this appendix come from the files `canonical/v1.json`, `hashing/v1.json`, `signing/v1.json`, `capability/v1.json`, and `receipt/v1.json` of the Chio binding vector corpus, and all keys in them are test keys derived from seeds that the corpus publishes. Lines longer than 69 characters are folded as described in [RFC8792], and each folded block begins with the header line that document defines.

B.1. Canonical JSON

Each case gives a JSON text and its canonical form, which Section 4 defines.

The following case is `canonical/v1.json` case `object_key_sorting` from the Chio binding vector corpus. Members are sorted by key. The input is:

```
{"z":1,"a":2,"m":3}
```

The canonical form is:

```
{"a":2,"m":3,"z":1}
```

The following case is `canonical/v1.json` case `utf16_key_ordering` from the Chio binding vector corpus. Keys are compared as UTF-16 code units, so the key U+10000, which the input writes as a surrogate pair, sorts before the key U+E000, which is the reverse of their order as UTF-8 octets. The input is:

```
{"\ue000":1,"\ud800\udc00":2}
```

The canonical form holds characters outside ASCII, so it is shown as its 18 octets, in hex:

```
7b 22 f0 90 80 80 22 3a 32 2c 22 ee 80 80 22 3a 31 7d
```

The following case is `canonical/v1.json` case `nested_structures` from the Chio binding vector corpus. Members are sorted at every level of nesting, and the order of array elements is unchanged. The input is:

```
================ NOTE: '\' line wrapping per RFC 8792 ================

{"tool":"read","params":{"path":"/tmp/demo","flags":["read",\
"text"]},"enabled":true}
```

The canonical form is:

```
================ NOTE: '\' line wrapping per RFC 8792 ================

{"enabled":true,"params":{"flags":["read","text"],"path":\
"/tmp/demo"},"tool":"read"}
```

The following case is `canonical/v1.json` case `number_formatting` from the Chio binding vector corpus. Numbers are written as ECMAScript writes them: `1.0` becomes `1`, negative zero becomes `0`, and large and small magnitudes use exponent forms such as `1e+21` and `1e-7`. The input is:

```
{"whole":1.0,"small":1e-7,"big":1e21,"negative_zero":-0.0}
```

The canonical form is:

```
{"big":1e+21,"negative_zero":0,"small":1e-7,"whole":1}
```

B.2. SHA-256 Digests

Each case gives the octets of a UTF-8 string and their SHA-256 digest [FIPS180-4], written as 64 lowercase hex digits.

The following case is `hashing/v1.json` case `abc_lowercase` from the Chio binding vector corpus. The input is the ASCII string `abc`, which is three octets:

```
61 62 63
```

The digest is:

```
ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
```

The following case is `hashing/v1.json` case `unicode_utf8` from the Chio binding vector corpus. The input is the word `chio`, a space, and the character U+1F510, which is 9 octets in UTF-8:

```
63 68 69 6f 20 f0 9f 94 90
```

The digest is:

```
5c57c7deea3998c13730797e592abe72cae55130e8da2923de6374b75b83101b
```

The following case is `hashing/v1.json` case `json_canonical_form` from the Chio binding vector corpus. The input is a canonical JSON text:

```
{"a":1,"b":[2,3]}
```

The digest is:

```
efbd0040190fb0871831e606c581f8a66db79d8e2bb836745a70051306956070
```

B.3. Ed25519 Signatures

In each case the signature is an Ed25519 signature [RFC8032] over the signing wrapper (Section 4.5) of the canonical form of the JSON input under the schema id of the case.

The following case is `signing/v1.json` case `valid_wrapped_json_message` from the Chio binding vector corpus. The input is:

```
{"z":1,"a":2}
```

Its canonical form is:

```
{"a":2,"z":1}
```

The signing wrapper of the canonical form under the schema id `chio.receipt.v1` is these 88 octets, in hex:

```
================ NOTE: '\' line wrapping per RFC 8792 ================

6368696f2e7369676e696e672d777261707065722e76310030a0a0cb00102ac64808\
db450924b39544cc41b773e6d2b7f0d899021a6c2ae1c2985c5ba6f7d2a55e768f92\
490ca09388e95bc4cccb9fdf11b15f4d42f93e73
```

The Ed25519 public key is:

```
fd1724385aa0c75b64fb78cd602fa1d991fdebf76b13c58ed702eac835e9f618
```

The signature is:

```
================ NOTE: '\' line wrapping per RFC 8792 ================

f2ae2e1de317ecd0b73d822e527642282965a7b72904e0dde67ca53e324860c67398\
8a1e690dc0a73acef7d1e08348566af7af01cf7ca3c08901b1618c02e50e
```

Verification is expected to succeed.

The following case is `signing/v1.json` case `tampered_wrapped_json_message` from the Chio binding vector corpus. It reuses the signature of case `valid_wrapped_json_message` for a message whose value of `z` differs. The input is:

```
{"z":2,"a":2}
```

Its canonical form is:

```
{"a":2,"z":2}
```

The signing wrapper of the canonical form under the schema id `chio.receipt.v1` is these 88 octets, in hex:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

6368696f2e7369676e696e672d777261707065722e76310030a0a0cb00102ac64808\
db450924b39544cc41b773e6d2b7f0d899021a6c2ae1800990570eb5beee61ae3dab\
26f990a9fab09bc6c4a8a65f164bdb7b8e9c5d76
```

The Ed25519 public key is:

```
fd1724385aa0c75b64fb78cd602fa1d991fdebf76b13c58ed702eac835e9f618
```

The signature is:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

f2ae2e1de317ecd0b73d822e527642282965a7b72904e0dde67ca53e324860c67398\
8a1e690dc0a73acef7d1e08348566af7af01cf7ca3c08901b1618c02e50e
```

Verification is expected to fail.

The following case is `signing/v1.json` case `substituted_schema_id` from the Chio binding vector corpus. It reuses the signature and message of case `valid_wrapped_json_message` under another schema id. The input is:

```
{"z":1,"a":2}
```

Its canonical form is:

```
{"a":2,"z":1}
```

The signing wrapper of the canonical form under the schema id `chio.capability.v1` is these 88 octets, in hex:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

6368696f2e7369676e696e672d777261707065722e7631005d8d33c3d56ece64cca5\
a6fa61a763d04ce26ebfd9235ec1f75e81b28a6a535fc2985c5ba6f7d2a55e768f92\
490ca09388e95bc4cccb9fdf11b15f4d42f93e73
```

The Ed25519 public key is:

```
fd1724385aa0c75b64fb78cd602fa1d991fdebf76b13c58ed702eac835e9f618
```

The signature is:

```
================ NOTE: '\' line wrapping per RFC 8792 ================

f2ae2e1de317ecd0b73d822e527642282965a7b72904e0dde67ca53e324860c67398\
8a1e690dc0a73acef7d1e08348566af7af01cf7ca3c08901b1618c02e50e
```

Verification is expected to fail.

B.4. Capability Token Verification

Each case gives the signing body of a token, its signing wrapper under the schema id `chio.capability.v1`, the value of its `signature` member, the Unix time at which the token is verified, and the result that verification is expected to produce. The signing body is the canonical form of the token without its `signature` member, including its member `"schema":"chio.capability.v1"`, as in Appendix A.1. It is the `capability_signing_body_canonical_json` value of the case.

The expected result covers the signature of the token, the signatures, key connectivity, and timestamps of its delegation links, and its validity period. It is not the outcome of the full verification in Section 5.8, which also requires the final delegatee to be the token subject and applies issuer trust and chain binding.

The following case is `capability/v1.json` case `valid_delegated_capability` from the Chio binding vector corpus. Its signature verifies, its delegation link verifies, and the verification time falls within its validity period. Its final delegatee is not its subject, so full verification rejects it. Its signing body is:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

{"delegation_chain":[{"capability_id":"cap-bindings-valid",\
"delegatee":\
"91a28a0b74381593a4d9469579208926afc8ad82c8839b7644359b9eba9a4b3a",\
"delegator":\
"66be7e332c7a453332bd9d0a7f7db055f5c5ef1a06ada66d98b39fb6810c473a",\
"signature":"c76b203b6fc0defbbadee97399d5ab4e1ff3fcc9ece8feaf0ec594d\
1c55d0d5ffe7c9a375695e049fe7c3ec3105c669ee1bc9937cffbd5f526fd9669c8c\
add01","timestamp":1710000250}],"expires_at":1710000800,"id":\
"cap-bindings-valid","issued_at":1710000200,"issuer":\
"66be7e332c7a453332bd9d0a7f7db055f5c5ef1a06ada66d98b39fb6810c473a",\
"schema":"chio.capability.v1","scope":{"grants":[{"constraints":\
[{"type":"path_prefix","value":"/workspace/"}],"max_invocations":3,\
"operations":["invoke","read_result"],"server_id":"srv-files",\
"tool_name":"file_read"}]},"subject":\
"0b513ad9b4924015ca0902ed079044d3ac5dbec2306f06948c10da8eb6e39f2d"}
```

Its signing wrapper is:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

6368696f2e7369676e696e672d777261707065722e7631005d8d33c3d56ece64cca5\
a6fa61a763d04ce26ebfd9235ec1f75e81b28a6a535f780e709fe51d7b1a83237ab8\
5327477ee99ff906a6cf8f8822952e25cd352ed9
```

The value of the `signature` member is:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

b27fe85e65d1ad63e44f268e0213fbc68c33e3e77fe33f16696630467096b8d70639\
b15cf6e20d8a53782311d286051584f258e58d5e13a6f796166bda954d0c
```

Verification at Unix time 1710000400 is expected to produce:

```
{
  "delegation_chain_shape_valid": true,
  "signature_valid": true,
  "time_status": "valid",
  "time_valid": true
}
```

The following case is `capability/v1.json` case `expired_capability` from the Chio binding vector corpus. Its signature verifies, and it expired at 1710000100, before the verification time. Its signing body is:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

{"expires_at":1710000100,"id":"cap-bindings-expired","issued_at":\
1710000000,"issuer":\
"66be7e332c7a453332bd9d0a7f7db055f5c5ef1a06ada66d98b39fb6810c473a",\
"schema":"chio.capability.v1","scope":{"grants":[{"constraints":\
[{"type":"path_prefix","value":"/workspace/"}],"max_invocations":3,\
"operations":["invoke","read_result"],"server_id":"srv-files",\
"tool_name":"file_read"}]},"subject":\
"0b513ad9b4924015ca0902ed079044d3ac5dbec2306f06948c10da8eb6e39f2d"}
```

Its signing wrapper is:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

6368696f2e7369676e696e672d777261707065722e7631005d8d33c3d56ece64cca5\
a6fa61a763d04ce26ebfd9235ec1f75e81b28a6a535f91d5a61c8dd6bf456e674026\
2865e2c75be1687e7edd2dd6e8689d4a0cccf32f
```

The value of the `signature` member is:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

972dd504920dbd8bacdc75a48c1e49f882b9fa154b2683674ddf791f7fba34e54e66\
bebbadb9affdf58f6127c9374be379d770123d2b6a93671ec86c16ea330f
```

Verification at Unix time 1710000400 is expected to produce:

```
{
  "delegation_chain_shape_valid": true,
  "signature_valid": true,
  "time_status": "expired",
  "time_valid": false
}
```

The following case is `capability/v1.json` case `broken_delegation_chain_signature` from the Chio binding vector corpus. Its signature verifies, and the signature of its delegation link does not. Its signing body is:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

{"delegation_chain":[{"capability_id":"cap-bindings-broken-chain",\
"delegatee":\
"91a28a0b74381593a4d9469579208926afc8ad82c8839b7644359b9eba9a4b3a",\
"delegator":\
"66be7e332c7a453332bd9d0a7f7db055f5c5ef1a06ada66d98b39fb6810c473a",\
"signature":"e5031fd5c95e1cb79201c9750831e6604ee65fa451202f0ef540e9e\
30f7bc8e73ca877424e7ca3121db39cbb57e97827878a8656b077a7c084f9bfeeb84\
66f0e","timestamp":1710000301}],"expires_at":1710000800,"id":\
"cap-bindings-broken-chain","issued_at":1710000200,"issuer":\
"66be7e332c7a453332bd9d0a7f7db055f5c5ef1a06ada66d98b39fb6810c473a",\
"schema":"chio.capability.v1","scope":{"grants":[{"constraints":\
[{"type":"path_prefix","value":"/workspace/"}],"max_invocations":3,\
"operations":["invoke","read_result"],"server_id":"srv-files",\
"tool_name":"file_read"}]},"subject":\
"0b513ad9b4924015ca0902ed079044d3ac5dbec2306f06948c10da8eb6e39f2d"}
```

Its signing wrapper is:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

6368696f2e7369676e696e672d777261707065722e7631005d8d33c3d56ece64cca5\
a6fa61a763d04ce26ebfd9235ec1f75e81b28a6a535fde390c726ab7f60e895388d4\
43fb50623c54b89aa76327ac7fa9c0c1f6d280e0
```

The value of the `signature` member is:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

c9f6927713d2f2289cfbf5507b008e7b17b88ad74b4ce9f8b5e71da2419fc2afc963\
9a233a046e8b908e997b7f94f3e8c55cc77b12a9410d525ea43a600b530a
```

Verification at Unix time 1710000400 is expected to produce:

```
{
  "delegation_chain_shape_valid": false,
  "signature_valid": true,
  "time_status": "valid",
  "time_valid": true
}
```

B.5. Receipt Verification

Each case gives the signing body of a receipt, its signing wrapper under the schema id `chio.receipt.v1`, the value of its `signature` member, and the result that verification is expected to produce. The signing body is the canonical form of a JSON object with two members: `body`, which is the receipt without its `id` and `signature` members, and `id`, which is the SHA-256 digest of the canonical form of `body`, in lowercase hex, as in Appendix A.3. The `receipt_body_canonical_json` value of each case is the canonical form of the receipt without its `signature` member.

The following case is `receipt/v1.json` case `allow_receipt` from the Chio binding vector corpus. It records an allow decision. Its signing body is:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

{"body":{"action":{"parameter_hash":\
"035350b0e6a021a4c924a149111944945e94d12f6d8a5e24e4461e7009ebf929",\
"parameters":{"mode":"read","path":"/workspace/docs/roadmap.md"}},\
"boundary_class":"prevent","capability_id":"cap-bindings-001",\
"content_hash":\
"4062edaf750fb8074e7e83e0c9028c94e32468a8b6f1614774328ef045150f93",\
"decision":{"verdict":"allow"},"evidence":[{"details":\
"path allowed","guard_name":"ForbiddenPathGuard","verdict":true},\
{"details":"no secrets detected","guard_name":"SecretLeakGuard",\
"verdict":true}],"kernel_key":\
"ea4a6c63e29c520abef5507b132ec5f9954776aebebe7b92421eea691446d22c",\
"metadata":{"surface":"bindings-vectors","version":1},"policy_hash":\
"policy-bindings-v1","receipt_kind":"mediated_decision",\
"redaction_mode":"none","timestamp":1710000200,"tool_name":\
"file_read","tool_origin":"caller_executed","tool_server":\
"srv-files","trust_level":"mediated"},"id":\
"c9909f733d5fb367922293f0167d06db715393be114f212096cf2866f29433ea"}
```

Its signing wrapper is:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

6368696f2e7369676e696e672d777261707065722e76310030a0a0cb00102ac64808\
db450924b39544cc41b773e6d2b7f0d899021a6c2ae16d0752cbe80e4a4f8204063c\
d20e9a17bbcb5c5ce16d3ea122fd05cfe80365ee
```

The value of the `signature` member is:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

67e3bfef7a6f3da592811e088d396f3186fbc1533632b273fbd3a2f6ce60eb099418\
d256853398661d802d179ea869acb584bed757681f501f2c40fd4fde790a
```

With no trusted signers configured, verification is expected to produce:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

{
  "authorized": false,
  "boundary_class": "prevent",
  "decision": "allow",
  "ok": false,
  "parameter_hash_valid": true,
  "receipt_id_valid": true,
  "receipt_kind": "mediated_decision",
  "result": "Authorized",
  "signature_valid": true,
  "signer_key_hex": "ea4a6c63e29c520abef5507b132ec5f9954776aebebe7b9\
      2421eea691446d22c",
  "signer_trusted": false,
  "trust_level": "mediated"
}
```

The following case is `receipt/v1.json` case `deny_receipt` from the Chio binding vector corpus. It records a deny decision by the guard `ForbiddenPathGuard` for the path `/etc/shadow`. Its signing body is:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

{"body":{"action":{"parameter_hash":\
"d6f55d803ff027795f4a4c43fd8f6d3119051f82b7e3e1a13b1713fd37ea7b09",\
"parameters":{"mode":"read","path":"/etc/shadow"}},"boundary_class":\
"prevent","capability_id":"cap-bindings-001","content_hash":\
"4062edaf750fb8074e7e83e0c9028c94e32468a8b6f1614774328ef045150f93",\
"decision":{"guard":"ForbiddenPathGuard","reason":\
"path is forbidden","verdict":"deny"},"evidence":[{"details":\
"path allowed","guard_name":"ForbiddenPathGuard","verdict":true},\
{"details":"no secrets detected","guard_name":"SecretLeakGuard",\
"verdict":true}],"kernel_key":\
"ea4a6c63e29c520abef5507b132ec5f9954776aebebe7b92421eea691446d22c",\
"metadata":{"surface":"bindings-vectors","version":1},"policy_hash":\
"policy-bindings-v1","receipt_kind":"mediated_decision",\
"redaction_mode":"none","timestamp":1710000200,"tool_name":\
"file_read","tool_origin":"caller_executed","tool_server":\
"srv-files","trust_level":"mediated"},"id":\
"56d5644db5c53d0bb837bf615a60e541724858757aee1b8259a25816b75bb682"}
```

Its signing wrapper is:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

6368696f2e7369676e696e672d777261707065722e76310030a0a0cb00102ac64808\
db450924b39544cc41b773e6d2b7f0d899021a6c2ae1db46cf3bbd040d678b4ba94c\
f9036efac42c2fbc1442a619c9acb12095c72857
```

The value of the `signature` member is:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

d3b96d12fb74aa7f05c46adc2d62942caa5f92a65716ba79e9f3b6c6179f54e5221e\
7531beb9bb17fcbf8ab6a409c994caba1158d904054d371386117b8e7008
```

With no trusted signers configured, verification is expected to produce:

```
=============== NOTE: '\' line wrapping per RFC 8792 ================

{
  "authorized": false,
  "boundary_class": "prevent",
  "decision": "deny",
  "ok": false,
  "parameter_hash_valid": true,
  "receipt_id_valid": true,
  "receipt_kind": "mediated_decision",
  "result": "Denied",
  "signature_valid": true,
  "signer_key_hex": "ea4a6c63e29c520abef5507b132ec5f9954776aebebe7b9\
     2421eea691446d22c",
  "signer_trusted": false,
  "trust_level": "mediated"
}
```

Appendix C. Relationship to Other Work

This appendix compares Chio with adjacent work. A comparison states where the problems overlap; it does not claim wire compatibility unless it says so.

C.1. Agent Communication Protocols

The Model Context Protocol [MCP] defines how a client discovers and calls the tools, resources, and prompts of a server. The Agent2Agent protocol [A2A] defines how agents exchange tasks and messages. Chio does not replace either. The MCP binding in Section 11 makes a Chio kernel an MCP server whose tool calls it mediates, so an unmodified MCP client can make basic tool calls through a kernel under a compatible admission profile. Chio sender constraints and governed metadata require a client that supplies those extensions.

C.2. Delegated Authorization

OAuth 2.0 Token Exchange [RFC8693], Rich Authorization Requests [RFC9396], and GNAP [RFC9635] address aspects of authorization. A Chio capability token plays a related role. Token Exchange conveys delegated authority, RAR describes fine-grained requested authorization, and GNAP negotiates grants. Chio uses signed delegation links and scope witnesses with verifier-owned issuer and root inputs; its current authenticated attenuation profile accepts one hop.

Token signing still requires a trusted issuer. A kernel checks that authority and records receipt-bearing outcomes. Chio defines its own capability and receipt objects and does not claim GNAP compatibility.

DPoP [RFC9449] binds an OAuth access token to a key held by the client. The sender proof in Section 5.4 has the same goal for capability tokens. It uses a Chio-specific proof format and is not RFC 9449 on the native transport.

C.3. Capability Tokens

Macaroons [MACAROONS], Biscuit [BISCUIT], and UCAN [UCAN] are bearer or key-bound capability tokens that support offline attenuation. Chio shares their model of narrowing authority by adding restrictions. Its authenticated attenuation profile binds a one-hop delegation to scope hashes, with a trusted token issuer and verifier-owned root inputs. Its kernel also applies the qualified sibling-budget checks in Section 7 and records receipt-bearing mediated outcomes. These features do not establish a uniqueness claim against other capability systems.

C.4. Workload Identity and Attestation

The WIMSE architecture [I-D.ietf-wimse-arch-08] addresses identity for workloads that call one another across systems, and RATS [RFC9334] defines how a verifier appraises attestation evidence. A Chio subject is a public key. A deployment can bind that key to a workload identity or accept attestation evidence at issuance (Section 12), but attestation does not by itself authorize a call (Section 15).

C.5. Signed Evidence and Transparency

SCITT [RFC9943] defines signed statements and transparency services that record them. Chio receipts and checkpoints address a related problem: they let a party check what a kernel authorized and executed. Chio is not a SCITT profile, and Section 9 states the limits of the claims that checkpoints support.

C.6. Verifiable Credentials

W3C Verifiable Credentials [VC-DATA-MODEL-2.0], OpenID for Verifiable Credential Issuance [OID4VCI], and OpenID for Verifiable Presentations [OID4VP] carry portable claims about a subject. Chio's portable identity credentials for agents, which the reference implementation issues through a bounded OID4VCI-compatible profile and presents through an OID4VP-style profile, are outside the scope of this document.

C.7. Candidate Companion Documents

The reference implementation carries mechanisms that this document does not specify. Each could be specified in its own document:

- federation: standing relationships between operators, including peer handshakes, key pinning, and cross-operator delegation;

- portable identity and reputation credentials for agents;
- settlement of obligations recorded in receipts, and payment channels;
- public anchoring of checkpoints;
- mediation of arbitrary HTTP APIs through a local evaluation service;
- selective disclosure of receipt fields.

Appendix D. Document History

This section is to be removed before publishing as an RFC.

draft-whelan-chio-protocol-00: Initial version.

Acknowledgments

Chio builds on earlier work on capability-based authorization, in particular macaroons, Biscuit, and UCAN, and on the IETF's work on proof-of-possession tokens, workload identity, and transparency services.

Author's Address

Connor Whelan
Backbay Industries
Email: connor@backbay.io